

Introduction to Compilers

Chapter 1 – Introduction

1.1 What is a compiler?

A **compiler** translates a program in a **source language** to a program in a **target language**. The most well known form of a compiler is one that translates a high-level language like C into the native assembly language of a machine so that it can be executed. And of course there are compilers for other languages like C++, Java, C#, and Rust, and many others.

The same techniques used in a traditional compiler are also used in any kind of program that processes a language. For example, a typesetting program like T_EX translates a manuscript into a Postscript document. A graph-layout program like Dot consumes a list of nodes and edges and arranges them on a screen. A web browser translates an HTML document into an interactive graphical display. To write programs like these, you need to understand and use the same techniques as in traditional compilers.

Compilers exist not only to *translate* programs, but also to *improve* them. A compiler assists a programmer by finding errors in a program at compile time, so that the user does not have to encounter them at runtime. Usually, a more strict language results in more compile-time errors. This makes the programmer's job harder, but makes it more likely that the program is correct. For example, the Ada language is infamous among programmers as challenging to write without compile-time errors, but once working, is trusted to run safety-critical systems such as the Boeing 777 aircraft.

A compiler is distinct from an **interpreter**, which reads in a program and then executes it directly, without emitting a translation. This is also sometimes known as a **virtual machine**. Languages like Python and Ruby are typically executed by an interpreter that reads the source code directly.

Compilers and interpreters are closely related, and it is sometimes possible to exchange one for the other. For example, Java compilers translate Java source code into Java **bytecode**, which is an abstract form of assembly language. Some implementations of the Java Virtual Machine work as interpreters that execute one instruction at a time. Others work by translating the bytecode into local machine code, and then running the machine code directly. This is known as **just in time compiling** or **JIT**.

1.2 Why should you study compilers?

You will be a better programmer. A great craftsman must understand his or her tools, and a programmer is no different. By understanding more deeply how a compiler translates your program into machine language, you will become more skilled at writing effective code and debugging it when things go wrong.

You can create tools for debugging and translating. If you can write a parser for a given language, then you can write all manner of supporting tools that help you (and others) debug your own programs. An integrated development environment like Eclipse incorporates parsers for languages like Java, so that it can highlight syntax, find errors without compiling, and connect code to documentation as you write.

You can create new languages. A surprising number of problems are made easier by expressing them compactly in a custom language. (These are sometimes known as **domain specific languages** or simply **little languages**.) By learning the techniques of compilers, you will be able to implement little languages and avoid some pitfalls of language design.

You can contribute to existing compilers. While it's unlikely that you will write the next great C compiler (since we already have several), language and compiler development does not stand still. Standards development results in new language features; optimization research creates new ways of improving programs; new microprocessors are created; new operating systems are developed; and so on. All of these developments require the continuous improvement of existing compilers.

You will have fun while solving challenging problems. Isn't that enough?

1.3 What's the best way to learn about compilers?

The best way to learn about compilers is to *write your own compiler* from beginning to end. While that may sound daunting at first, you will find that this complex task can be broken down into several stages of moderate complexity. The typical undergraduate computer science student can write a complete compiler for a simple language in a semester, broken down into four or five independent stages.

1.4 What language should I use?

Without question, you should use the C programming language and the X86 assembly language, of course!

Ok, maybe the answer isn't quite that simple. There is an ever-increasing number of programming languages that all have different strengths and weaknesses. Java is simple, consistent, and portable, albeit not high performance. Python is easy to learn and has great library support, but is weakly typed. Rust offers exceptional static type-safety, but is not (yet)

widely used. It is quite possible to write a compiler in nearly any language, and you could use this book as a guide to do so.

However, we really think that you should learn C, write a compiler in C, and use it to compile a C-like language which produces assembly for a widely used processor, like X86 or ARM. Why? Because it is important for you to learn the ins and outs of technologies that are in wide use, and not just those that are abstractly beautiful.

C is the most widely-used portable language for low-level coding (compilers, libraries, and kernels) and it is also small enough that one can learn how to compile every aspect of C in a single semester. True, C presents some challenges related to type safety and pointer use, but these are manageable for a project the size of a compiler. There are other languages with different virtues, but none as simple and as widely used as C. Once you write a C compiler, then you are free to design your own (better) language.

Likewise, the X86 has been the most widely deployed computer architecture in desktops, servers, and laptops for several decades. While it is considerably more complex than other architectures like MIPS or SPARC or ARM, one can quickly learn the essential subset of instructions necessary to build a compiler. Of course, ARM is quickly catching up as a popular architecture in the mobile, embedded, and low power space, so we have included a section on that as well.

That said, the principles presented in this book are widely applicable. If you are using this as part of a class, your instructor may very well choose a different compilation language and different target assembly, and that's fine too.

1.5 How is this book different from others?

Most books on compilers are very heavy on the abstract theory of scanners, parsers, type systems, and register allocation, and rather light on how the design of a language affects the compiler and the runtime. Most are designed for use by a graduate survey of optimization techniques.

This book takes a broader approach by giving a lighter dose of optimization, and introducing more material on the process of engineering a compiler, the tradeoffs in language design, and considerations for interpretation and translation.

You will also notice that this book doesn't contain a whole bunch of fiddly paper-and-pencil assignments to test your knowledge of compiler algorithms. (Ok, there are a few of those in Chapters 3 and 4.) If you want to test your knowledge, then write some working code. To that end, the exercises at the end of each chapter ask you to take the ideas in the chapter, and either explore some existing compilers, or write parts of your own. If you do all of them in order, you will end up with a working compiler, summarized in the final appendix.

1.6 What other books should I read?

For general reference on compilers, I suggest the following books:

- **Charles N. Fischer, Ron K. Cytron, and Richard J. LeBlanc Jr, “Crafting a Compiler”, Pearson, 2009.**
This is an excellent undergraduate textbook which focuses on object-oriented software engineering techniques for constructing a compiler, with a focus on generating output for the Java Virtual Machine.
- **Christopher Fraser and David Hanson, “A Retargetable C Compiler: Design and Implementation”, Benjamin/Cummings, 1995.**
Also known as the “LCC book”, this book focuses entirely on explaining the C implementation of a C compiler by taking the unusual approach of embedding the literal code into the textbook, so that code and explanation are intertwined.
- **Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman, “Compilers: Principles, Techniques, and Tools”, Addison Wesley, 2006.** *Affectionately known as the “dragon book”, this is a comprehensive treatment of the theory of compilers from scanning through type theory and optimization at an advanced graduate level.*

Ok, what are you waiting for? Let’s get to work.

Chapter 2 – A Quick Tour

2.1 The Compiler Toolchain

A compiler is one component in a **toolchain** of programs used to create executables from source code. Typically, when you invoke a single command to compile a program, a whole sequence of programs are invoked in the background. Figure 2.1 shows the programs typically used in a Unix system for compiling C source code to assembly code.

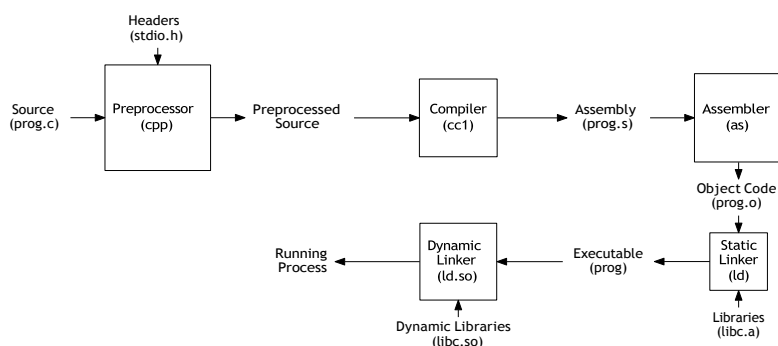


Figure 2.1: A Typical Compiler Toolchain

- The **preprocessor** prepares the source code for the compiler proper. In the C and C++ languages, this means consuming all directives that start with the `#` symbol. For example, an `#include` directive causes the preprocessor to open the named file and insert its contents into the source code. A `#define` directive causes the preprocessor to substitute a value wherever a macro name is encountered. (Not all languages rely on a preprocessor.)
- The **compiler** proper consumes the clean output of the preprocessor. It scans and parses the source code, performs typechecking and

other semantic routines, optimizes the code, and then produces assembly language as the output. This part of the toolchain is the main focus of this book.

- The **assembler** consumes the assembly code and produces **object code**. Object code is “almost executable” in that it contains raw machine language instructions in the form needed by the CPU. However, object code does not know the final memory addresses in which it will be loaded, and so it contains gaps that must be filled in by the linker.
- The **linker** consumes one or more object files and library files and combines them into a complete, executable program. It selects the final memory locations where each piece of code and data will be loaded, and then “links” them together by writing in the missing address information. For example, an object file that calls the `printf` function does not initially know the address of the function. An empty (zero) address will be left where the address must be used. Once the linker selects the memory location of `printf`, it must go back and write in the address at every place where `printf` is called.

In Unix-like operating systems, the preprocessor, compiler, assembler, and linker are historically named `cpp`, `cc1`, `as`, and `ld` respectively. The user-visible program `cc` simply invokes each element of the toolchain in order to produce the final executable.

2.2 Stages Within a Compiler

In this book, our focus will be primarily on the compiler proper, which is the most interesting component in the toolchain. The compiler itself can be divided into several stages:

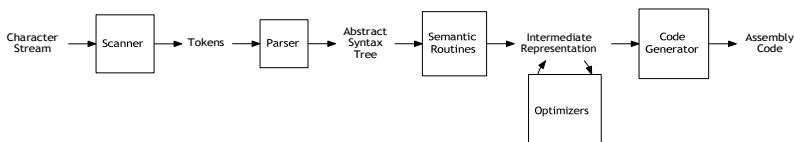


Figure 2.2: The Stages of a Unix Compiler

- The **scanner** consumes the plain text of a program and groups together individual characters to form complete **tokens**. This is much like grouping characters into words in a natural language.

- The **parser** consumes tokens and groups them together into complete statements and expressions, much like words are grouped into sentences in a natural language. The parser is guided by a **grammar** which states the formal rules of composition in a given language. The output of the parser is an **abstract syntax tree (AST)** that captures the grammatical structures of the program. The AST also remembers where in the source file each construct appeared, so it is able to generate targeted error messages, if needed.
- The **semantic routines** traverse the AST and derive additional meaning (semantics) about the program from the rules of the language and the relationship between elements of the program. For example, we might determine that `x + 10` is a `float` expression by observing the type of `x` from an earlier declaration, then applying the language rule that addition between `int` and `float` values yields a float. After the semantic routines, the AST is often converted into an **intermediate representation (IR)** which is a simplified form of assembly code suitable for detailed analysis. There are many forms of IR which we will discuss in Chapter 8.
- One or more **optimizers** can be applied to the intermediate representation, in order to make the program smaller, faster, or more efficient. Typically, each optimizer reads the program in IR format, and then emits the same IR format, so that each optimizer can be applied independently, in arbitrary order.
- Finally, a **code generator** consumes the optimized IR and transforms it into a concrete assembly language program. Typically, a code generator must perform **register allocation** to effectively manage the limited number of hardware registers, and **instruction selection** and **sequencing** to order assembly instructions in the most efficient form.

2.3 Example Compilation

Suppose we wish to compile this fragment of code into assembly:

```
height = (width+56) * factor(foo);
```

The first stage of the compiler (the scanner) will read in the text of the source code character by character, identify the boundaries between symbols, and emit a series of **tokens**. Each token is a small data structure that describes the nature and contents of each symbol:

id:height	=	(	id:width	+	int:56	)	*	id:factor	(	id:foo	)	;
-----------	---	---	----------	---	--------	---	---	-----------	---	--------	---	---

At this stage, the purpose of each token is not yet clear. For example, `factor` and `foo` are simply known to be identifiers, even though one is

the name of a function, and the other is the name of a variable. Likewise, we do not yet know the type of `width`, so the `+` could potentially represent integer addition, floating point addition, string concatenation, or something else entirely.

The next step is to determine whether this sequence of tokens forms a valid program. The parser does this by looking for patterns that match the **grammar** of a language. Suppose that our compiler understands a language with the following grammar:

Grammar G_1

1. $\text{expr} \rightarrow \text{expr} + \text{expr}$
2. $\text{expr} \rightarrow \text{expr} * \text{expr}$
3. $\text{expr} \rightarrow \text{expr} = \text{expr}$
4. $\text{expr} \rightarrow \text{id} (\text{expr})$
5. $\text{expr} \rightarrow (\text{expr})$
6. $\text{expr} \rightarrow \text{id}$
7. $\text{expr} \rightarrow \text{int}$

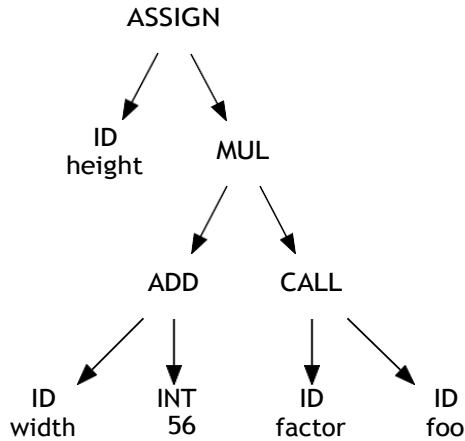
Each line of the grammar is called a rule, and explains how various parts of the language are constructed. Rules 1-3 indicate that an expression can be formed by joining two expressions with operators. Rule 4 describes a function call. Rule 5 describes the use of parentheses. Finally, rules 6 and 7 indicate that identifiers and integers are atomic expressions.¹

The parser looks for sequences of tokens that can be replaced by the left side of a rule in our grammar. Each time a rule is applied, the parser creates a node in a tree, and connects the sub-expressions into the **abstract syntax tree (AST)**. The AST shows the structural relationships between each symbol: addition is performed on `width` and `56`, while a function call is applied to `factor` and `foo`.

With this data structure in place, we are now prepared to analyze the meaning of the program. The **semantic routines** traverse the AST and derive additional meaning by relating parts of the program to each other and to the definition of the programming language. An important component of this process is **typechecking**, in which the type of each expression is determined, and checked for consistency with the rest of the program. To keep things simple here, we will assume that all of our variables are plain integers.

To generate linear intermediate code, we perform a post-order traversal of the AST and generate an IR instruction for each node in the tree. A typical IR looks like an abstract assembly language, with load/store instructions, arithmetic operations, and an infinite number of registers. For example, this is a possible IR representation of our example program:

¹The careful reader will note that this example grammar has ambiguities. We will discuss that in some detail in Chapter 4.

**Figure 2.3: Example AST**

```

LOAD $56      -> r1
LOAD width   -> r2
IADD r1, r2   -> r3
ARG foo
CALL factor   -> r4
IMUL r3, r4   -> r5
STOR r5       -> height

```

Figure 2.4: Example Intermediate Representation

The intermediate representation is where most forms of optimization occur. Dead code is removed, common operations are combined, and code is generally simplified to consume fewer resources and run more quickly. Finally, the intermediate code must be converted to the desired assembly code. Figure 2.5 shows X86 assembly code that is one possible translation of the IR given above. Note that the assembly instructions do not necessarily correspond one-to-one with IR instructions.

A well-engineered compiler is highly modular, so that common code elements can be shared and combined as needed. To support multiple languages, a compiler can provide distinct scanners and parsers, each emitting the same intermediate representation. Different optimization techniques can be implemented as independent modules (each reading and writing the same IR) so that they can be enabled and disabled indepen-

```
MOVQ    width, %rax    # load width into rax
ADDQ    $56, %rax      # add 56 to rax
MOVQ    %rax, -8(%rbp)  # save sum in temporary
MOVQ    foo, %edi       # load foo into arg 0 register
CALL    factor          # invoke factor, result in rax
MOVQ    -8(%rbp), %rbx  # load sum into rbx
IMULQ   %rbx           # multiply rbx by rax
MOVQ    %rax, height   # store result into height
```

Figure 2.5: Example Assembly Code

dently. A retargetable compiler contains multiple code generators, so that the same IR can be emitted for a variety of microprocessors.

2.4 Exercises

1. Determine how to invoke the preprocessor, compiler, assembler, and linker manually in your local computing environment. Compile a small complete program that computes a simple expression, and examine the output at each stage. Are you able to follow the flow of the program in each form?
2. Determine how to change the optimization level for your local compiler. Find a non-trivial source program and compile it at multiple levels of optimization. How does the compile time, program size, and run time vary with optimization levels?
3. Search the internet for the formal grammars for three languages that you are familiar with, such as C++, Ruby, and Rust. Compare them side by side. Which language is inherently more complex? Do they share any common structures?

Chapter 3 – Scanning

3.1 Kinds of Tokens

Scanning is the process of identifying **tokens** from the raw text source code of a program. At first glance, scanning might seem trivial – after all, identifying words in a natural language is as simple as looking for spaces between letters. However, identifying tokens in source code requires the language designer to clarify many fine details, so that it is clear what is permitted and what is not.

Most languages will have tokens in these categories:

- **Keywords** are words in the language structure itself, like `while` or `class` or `true`. Keywords must be chosen carefully to reflect the natural structure of the language, without interfering with the likely names of variables and other identifiers.
- **Identifiers** are the names of variables, functions, classes, and other code elements chosen by the programmer. Typically, identifiers are arbitrary sequences of letters and possibly numbers. Some languages require identifiers to be marked with a **sigil** (like the dollar sign in Perl) to clearly distinguish identifiers from keywords.
- **Numbers** could be formatted as integers, or floating point values, or fractions, or in alternate bases such as binary, octal or hexadecimal. Each format should be clearly distinguished, so that the programmer does not confuse one with the other.
- **Strings** are literal character sequences that must be clearly distinguished from keywords or identifiers. Strings are typically quoted with single or double quotes, but also must have some facility for containing quotations, newlines, and unprintable characters.
- **Comments** and **whitespace** are used to format a program to make it visually clear, and in some cases (like Python) are significant to the structure of a program.

When designing a new language, or designing a compiler for an existing language, the first job is to state precisely what characters are permitted in each type of token. Initially, this could be done informally by stating,

```

token_t scan_token( FILE *fp ) {
    int c = fgetc(fp);
    if(c=='*') {
        return TOKEN_MULTIPLY;
    } else if(c=='!') {
        char d = fgetc(fp);
        if(d=='=') {
            return TOKEN_NOT_EQUAL;
        } else {
            ungetc(d,fp);
            return TOKEN_NOT;
        }
    } else if(isalpha(c)) {
        do {
            char d = fgetc(fp);
        } while(isalnum(d));
        ungetc(d,fp);
        return TOKEN_IDENTIFIER;
    } else if ( . . . ) {
        . . .
    }
}

```

Figure 3.1: A Simple Hand Made Scanner

for example, “An identifier consists of a letter followed by any number of letters and numerals.”, and then assigning a symbolic constant (`TOKEN_IDENTIFIER`) for that kind of token. As we will see, an informal approach is often ambiguous, and a more rigorous approach is needed.

3.2 A Hand-Made Scanner

Figure 3.1 shows how one might write a scanner by hand, using simple coding techniques. To keep things simple, we only consider just a few tokens: `*` for multiplication, `!` for logical-not, `!=` for not-equal, and sequences of letters and numbers for identifiers.

The basic approach is to read one character at a time from the input stream (`fgetc(fp)`) and then classify it. Some single-character tokens are easy: if the scanner reads a `*` character, it immediately returns `TOKEN_MULTIPLY`, and the same would be true for addition, subtraction, and so forth.

However, some characters are part of multiple tokens. If the scanner encounters `!`, that could represent a logical-not operation by itself, or it could be the first character in the `!=` sequence representing not-equal-to.

Upon reading `!`, the scanner must immediately read the next character. If the next character is `=`, then it has matched the sequence `!=` and returns `TOKEN NOT EQUAL`.

But, if the character following `!` is something else, then the non-matching character needs to be *put back* on the input stream using `ungetc`, because it is not part of the current token. The scanner returns `TOKEN NOT` and will consume the put-back character on the next call to `scan_token`.

In a similar way, once a letter has been identified by `isalpha(c)`, then the scanner keeps reading letters or numbers, until a non-matching character is found. The non-matching character is put back, and the scanner returns `TOKEN IDENTIFIER`.

(We will see this pattern come up in every stage of the compiler: an unexpected item doesn't match the current objective, so it must be put back for later. This is known more generally as **backtracking**.)

As you can see, a hand-made scanner is rather verbose. As more token types are added, the code can become quite convoluted, particularly if tokens share common sequences of characters. It can also be difficult for a developer to be certain that the scanner code corresponds to the desired definition of each token, which can result in unexpected behavior on complex inputs. That said, for a small language with a limited number of tokens, a hand-made scanner can be an appropriate solution.

For a complex language with a large number of tokens, we need a more formalized approach to defining and scanning tokens. A formal approach will allow us to have a greater confidence that token definitions do not conflict and the scanner is implemented correctly. Further, a formal approach will allow us to make the scanner compact and high performance – surprisingly, the scanner itself can be the performance bottleneck in a compiler, since every single character must be individually considered.

The formal tools of **regular expressions** and **finite automata** allow us to state very precisely what may appear in a given token type. Then, automated tools can process these definitions, find errors or ambiguities, and produce compact, high performance code.

3.3 Regular Expressions

Regular expressions (REs) are a language for expressing patterns. They were first described in the 1950s by Stephen Kleene [4] as an element of his foundational work in automata theory and computability. Today, REs are found in slightly different forms in programming languages (Perl), standard libraries (PCRE), text editors (vi), command-line tools (grep), and many other places. We can use regular expressions as a compact and formal way of specifying the tokens accepted by the scanner of a compiler, and then automatically translate those expressions into working code. While easily explained, REs can be a bit tricky to use and require some practice in order to achieve the desired results.

Let us define regular expressions precisely:

A **regular expression** s is a string which denotes $L(s)$, a set of strings drawn from an alphabet Σ . $L(s)$ is known as the “language of s .”

$L(s)$ is defined inductively with the following base cases:

- If $a \in \Sigma$ then a is a regular expression and $L(a) = \{a\}$.
- $\varnothing$ is a regular expression and $L(\varnothing)$ contains only the empty string.

Then, for any regular expressions s and t :

1. $s|t$ is a RE such that $L(s|t) = L(s) \cup L(t)$.
2. st is a RE such that $L(st)$ contains all strings formed by the concatenation of a string in $L(s)$ followed by a string in $L(t)$.
3. s^* is a RE such that $L(s^*) = L(s)$ concatenated zero or more times.

Rule #3 is known as the **Kleene closure** and has the highest precedence. Rule #2 is known as **concatenation**. Rule #1 has the lowest precedence and is known as **alternation**. Parentheses can be added to adjust the order of operations in the usual way.

Here are a few examples using just the basic rules. (Note that a finite RE can indicate an infinite set.)

Regular Expression s	Language $L(s)$
hello	{ hello }
d(o i)g	{ dog, dig }
moo*	{ mo, moo, mooo, ... }
(moo)*	{ $\varnothing$, moo, moomoo, moomoomoo, ... }
a(b a)*a	{ aa, aaa, aba, aaaa, aaba, abaa, ... }

The syntax described so far is entirely sufficient to write any regular expression. But, it is also handy to have a few helper operations built on top of the basic syntax:

$s?$ indicates that s is optional.

$s?$ can be written as $(s|\varnothing)$

s^+ indicates that s is repeated one or more times.

s^+ can be written as ss^*

$[a-z]$ indicates any character in that range.

$[a-z]$ can be written as $(a|b|\dots|z)$

$[^x]$ indicates any character except one.

$[^x]$ can be written as $\Sigma - x$

Regular expressions also obey several algebraic properties, which make it possible to re-arrange them as needed for efficiency or clarity:

Associativity:	$a (b c) = (a b) c$
Commutativity:	$a b = b a$
Distribution:	$a(b c) = ab ac$
Idempotency:	$a^{**} = a^{*}$

Using regular expressions, we can precisely state what is permitted in a given token. Suppose we have a hypothetical programming language with the following informal definitions and regular expressions. For each token type, we show examples of strings that match (and do not match) the regular expression.

Informal definition:	<i>An identifier is a sequence of capital letters and numbers, but a number must not come first.</i>
Regular expression:	<code>[A-Z]+([A-Z] [0-9])*</code>
Matches strings:	<code>PRINT</code> <code>MODE5</code>
Does not match:	<code>hello</code> <code>4YOU</code>
Informal definition:	<i>A number is a sequence of digits with an optional decimal point. For clarity, the decimal point must have digits on both left and right sides.</i>
Regular expression:	<code>[0-9]+([0-9]+)?</code>
Matches strings:	<code>123</code> <code>3.14</code>
Does not match:	<code>.15</code> <code>30.</code>
Informal definition:	<i>A comment is any text (except a right angle bracket) surrounded by angle brackets.</i>
Regular expression:	<code><[^>]*></code>
Matches strings:	<code><tricky part></code> <code><<<<look left></code>
Does not match:	<code><this is an <illegal> comment></code>

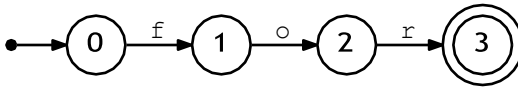
3.4 Finite Automata

A **finite automaton (FA)** is an abstract machine that can be used to represent certain forms of computation. Graphically, an FA consists of a number of states (represented by numbered circles) and a number of edges (represented by labeled arrows) between those states. Each edge is labeled with one or more symbols drawn from an alphabet Σ .

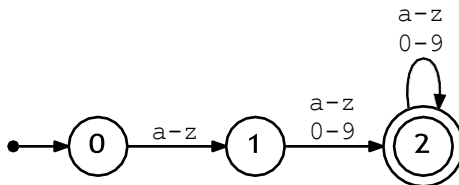
The machine begins in a start state S_0 . For each input symbol presented to the FA, it moves to the state indicated by the edge with the same label

as the input symbol. Some states of the FA are known as **accepting states** and are indicated by a double circle. If the FA is in an accepting state after all input is consumed, then we say that the FA **accepts** the input. We say that the FA **rejects** the input string if it ends in a non-accepting state, or if there is no edge corresponding to the current input symbol.

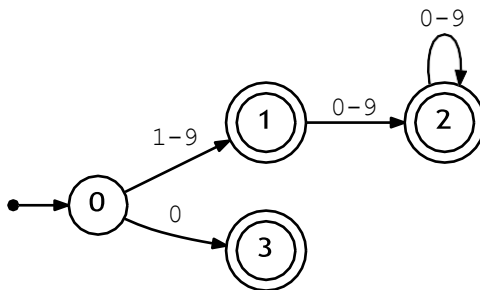
Every RE can be written as an FA, and vice versa. For a simple regular expression, one can construct an FA by hand. For example, here is an FA for the keyword `for`:



Here is an FA for identifiers of the form $[a-z][a-z0-9]^+$



And here is an FA for numbers of the form $([1-9][0-9]^*)|0$



3.4.1 Deterministic Finite Automata

Each of these three examples is a **deterministic finite automaton** (DFA). A DFA is a special case of an FA where every state has no more than one outgoing edge for a given symbol. Put another way, a DFA has no ambiguity: for every combination of state and input symbol, there is exactly one choice of what to do next.

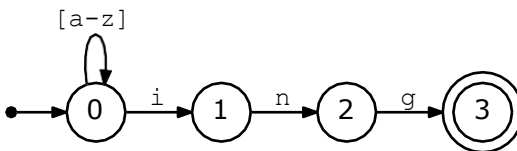
Because of this property, a DFA is very easy to implement in software or hardware. One integer (*c*) is needed to keep track of the current state.

The transitions between states are represented by a matrix ($M[s, i]$) which encodes the next state, given the current state and input symbol. (If the transition is not allowed, we mark it with E to indicate an error.) For each symbol, we compute $c = M[s, i]$ until all the input is consumed, or an error state is reached.

3.4.2 Nondeterministic Finite Automata

The alternative to a DFA is a **nondeterministic finite automaton (NFA)**. An NFA is a perfectly valid FA, but it has an ambiguity that makes it somewhat more difficult to work with.

Consider the regular expression $[a-z]^*ing$, which represents all lower-case words ending in the suffix *ing*. It can be represented with the following automaton:

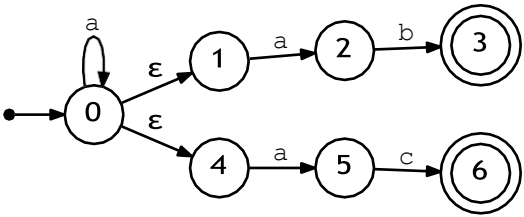


Now consider how this automaton would consume the word *sing*. It could proceed in two different ways. One would be to move to state 0 on *s*, state 1 on *i*, state 2 on *n*, and state 3 on *g*. But the other, equally valid way would be to stay in state 0 the whole time, matching each letter to the $[a-z]$ transition. Both ways obey the transition rules, but one results in acceptance, while the other results in rejection.

The problem here is that state 0 allows for two different transitions on the symbol *i*. One is to stay in state 0 matching $[a-z]$ and the other is to move to state 1 matching *i*.

Moreover, there is no simple rule by which we can pick one path or another. If the input is *sing*, the right solution is to proceed immediately from state zero to state one on *i*. But if the input is *singing*, then we should stay in state zero for the first *ing* and proceed to state one for the second *ing*.

An NFA can also have an ϵ (epsilon) transition, which represents the empty string. This transition can be taken without consuming any input symbols at all. For example, we could represent the regular expression $a^*(ab|ac)$ with this NFA:



This particular NFA presents a variety of ambiguous choices. From state zero, it could consume `a` and stay in state zero. Or, it could take an ϵ to state one or state four, and then consume an `a` either way.

There are two common ways to interpret this ambiguity:

- The **crystal ball interpretation** suggests that the NFA somehow “knows” what the best choice is, by some means external to the NFA itself. In the example above, the NFA would choose whether to proceed to state zero, one, or four before consuming the first character, and it would always make the right choice. Needless to say, this isn’t possible in a real implementation.
- The **many-worlds interpretation** suggests that the NFA exists in all allowable states *simultaneously*. When the input is complete, if any of those states are accepting states, then the NFA has accepted the input. This interpretation is more useful for constructing a working NFA, or converting it to a DFA.

Let us use the many-worlds interpretation on the example above. Suppose that the input string is `aaac`. Initially the NFA is in state zero. Without consuming any input, it could take an epsilon transition to states one or four. So, we can consider its initial state to be all of those states simultaneously. Continuing on, the NFA would traverse these states until accepting the complete string `aaac`:

States	Action
0, 1, 4	consume <code>a</code>
0, 1, 2, 4, 5	consume <code>a</code>
0, 1, 2, 4, 5	consume <code>a</code>
0, 1, 2, 4, 5	consume <code>c</code>
6	accept

In principle, one can implement an NFA in software or hardware by simply keeping track of all of the possible states. But this is inefficient. In the worst case, we would need to evaluate all states for all characters on each input transition. A better approach is to convert the NFA into an equivalent DFA, as we show below.

3.5 Conversion Algorithms

Regular expressions and finite automata are all equally powerful. For every RE, there is an FA, and vice versa. However, a DFA is by far the most straightforward of the three to implement in software. In this section, we will show how to convert an RE into an NFA, then an NFA into a DFA, and then to optimize the size of the DFA.



Figure 3.2: Relationship Between REs, NFAs, and DFAs

3.5.1 Converting REs to NFAs

To convert a regular expression to a nondeterministic finite automaton, we can follow an algorithm given first by McNaughton and Yamada [5], and then by Ken Thompson [6].

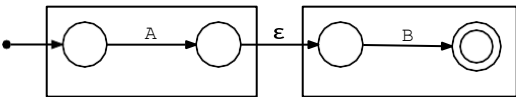
We follow the same inductive definition of regular expression as given earlier. First, we define automata corresponding to the base cases of REs:

The NFA for any character a is: The NFA for an $\varnothing$ transition is:



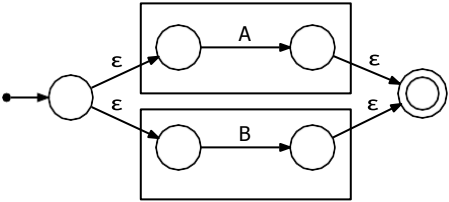
Now, suppose that we have already constructed NFAs for the regular expressions A and B , indicated below by rectangles. Both A and B have a single start state (on the left) and accepting state (on the right). If we write the concatenation of A and B as AB , then the corresponding NFA is simply A and B connected by an $\varnothing$ transition. The start state of A becomes the start state of the combination, and the accepting state of B becomes the accepting state of the combination:

The NFA for the concatenation AB is:



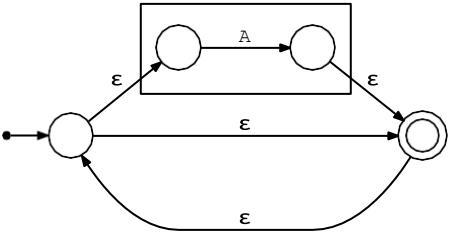
In a similar fashion, the alternation of A and B written as $A \mid B$ can be expressed as two automata joined by common starting and accepting nodes, all connected by $\varnothing$ transitions:

The NFA for the alternation $A \mid B$ is:

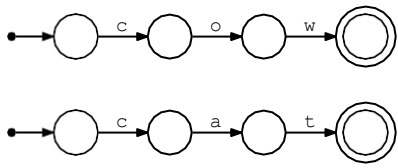


Finally, the Kleene closure A^* is constructed by taking the automaton for A , adding starting and accepting nodes, then adding $\varnothing$ transitions to allow zero or more repetitions:

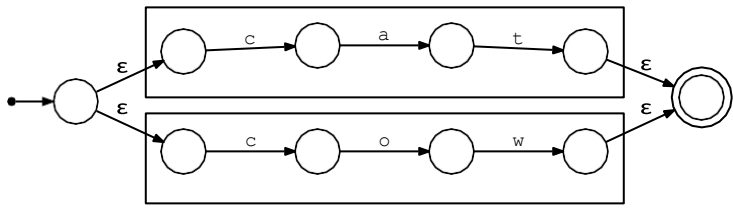
The NFA for the Kleene closure A^* is:



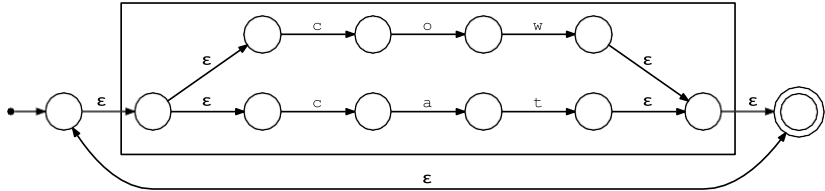
Example. Let's consider the process for an example regular expression $a(cat|cow)^*$. First, we start with the innermost expression `cat` and assemble it into three transitions resulting in an accepting state. Then, do the same thing for `cow`, yielding these two FAs:



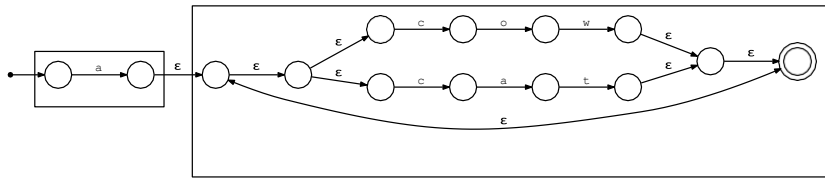
The alternation of the two expressions `cat|cow` is accomplished by adding a new starting and accepting node, with epsilon transitions. (The boxes are not part of the graph, but simply highlight the previous graph components carried forward.)



Then, the Kleene closure $(cat|cow)^*$ is accomplished by adding another starting and accepting state around the previous FA, with epsilon transitions between:



Finally, the concatenation of $a(cat|cow)^*$ is achieved by adding a single state at the beginning for a:



You can easily see that the NFA resulting from the construction algorithm, while correct, is quite complex and contains a large number of epsilon transitions. An NFA representing the tokens for a complete language could end up having thousands of states, which would be very impractical to implement. Instead, we can convert this NFA into an equivalent DFA.

3.5.2 Converting NFAs to DFAs

We can convert any NFA into an equivalent DFA using the technique of **subset construction**. The basic idea is to create a DFA such that each state in the DFA corresponds to multiple states in the NFA, according to the “many-worlds” interpretation.

Suppose that we begin with an NFA consisting of states N and start state N_0 . We wish to construct an equivalent DFA consisting of states D and start state D_0 . Each D state will correspond to multiple N states. First, we define a helper function known as the **epsilon closure**:

Epsilon closure.

$\varphi\text{-closure}(n)$ is the set of NFA states reachable from NFA state n by zero or more φ transitions.

Now we define the subset construction algorithm. First, we create a start state D_0 corresponding to the $\varphi\text{-closure}(N_0)$. Then, for each outgoing character c from the states in D_0 , we create a new state containing the epsilon closure of the states reachable by c . More precisely:

Subset Construction Algorithm.

Given an NFA with states N and start state N_0 , create an equivalent DFA with states D and start state D_0 .

Let $D_0 = \varphi\text{-closure}(N_0)$.

Add D_0 to a list.

While items remain on the list:

 Let d be the next DFA state removed from the list.

 For each character c in Σ :

 Let T contain all NFA states N_k such that:

$N_j \in d$ and $N_j \xrightarrow{c} N_k$

 Create new DFA state $D_i = \varphi\text{-closure}(T)$

 If D_i is not already in the list, add it to the end.

Figure 3.3: Subset Construction Algorithm

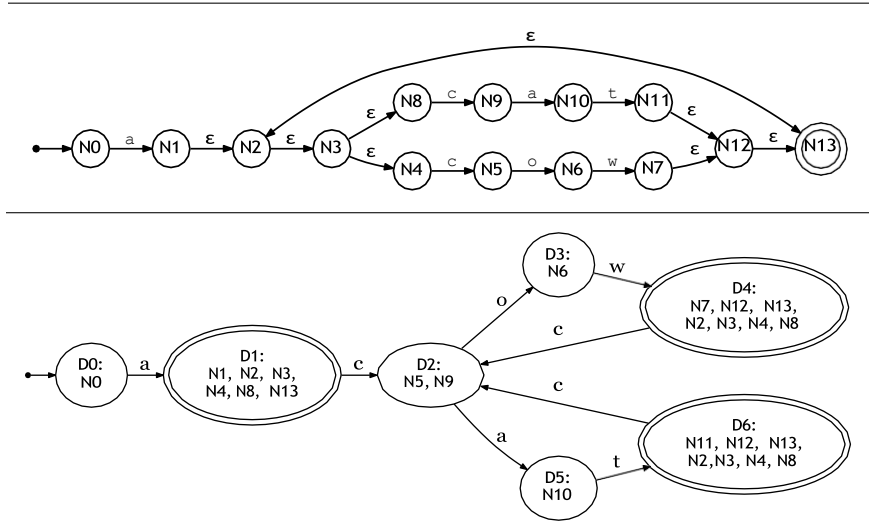


Figure 3.4: Converting an NFA to a DFA via Subset Construction

Example. Let's work out the algorithm on the NFA in Figure 3.4. This is the same NFA corresponding to the RE $a(cat|cow)^*$ with each of the states numbered for clarity.

1. Compute D_0 which is $\varphi\text{-closure}(N_0)$. N_0 has no φ transitions, so $D_0 = \{N_0\}$. Add D_0 to the work list.
2. Remove D_0 from the work list. The character a is an outgoing transition from N_0 to N_1 . $\varphi\text{-closure}(N_1) = \{N_1, N_2, N_3, N_4, N_8, N_{13}\}$ so add all of those to new state D_1 and add D_1 to the work list.
3. Remove D_1 from the work list. We can see that $N_4 \xrightarrow{c} N_5$ and $N_8 \xrightarrow{c} N_9$, so we create a new state $D_2 = \{N_5, N_9\}$ and add it to the work list.
4. Remove D_2 from the work list. Both a and o are possible transitions because of $N_5 \xrightarrow{o} N_6$ and $N_9 \xrightarrow{a} N_{10}$. So, create a new state D_3 for the o transition to N_6 and new state D_5 for the a transition to N_{10} . Add both D_3 and D_5 to the work list.
5. Remove D_3 from the work list. The only possible transition is $N_6 \xrightarrow{w} N_7$ so create a new state D_4 containing the $\varphi\text{-closure}(N_7)$ and add it to the work list.
6. Remove D_5 from the work list. The only possible transition is $N_{10} \xrightarrow{t} N_{11}$ so create a new state D_6 containing $\varphi\text{-closure}(N_{11})$ and add it to the work list.

7. Remove D_4 from the work list, and observe that the only outgoing transition $\subset$ leads to states N_5 and N_9 which already exist as state D_2 , so simply add a transition $D_4 \xrightarrow{\subset} D_2$.
8. Remove D_6 from the work list and, in a similar way, add $D_6 \xrightarrow{\subset} D_2$.
9. The work list is empty, so we are done.

3.5.3 Minimizing DFAs

The subset construction algorithm will definitely generate a valid DFA, but the DFA may possibly be very large (especially if we began with a complex NFA generated from an RE.) A large DFA will have a large transition matrix that will consume a lot of memory. If it doesn't fit in L1 cache, the scanner could run very slowly. To address this problem, we can apply Hopcroft's algorithm to shrink a DFA into a smaller (but equivalent) DFA.

The general approach of the algorithm is to optimistically group together all possibly-equivalent states S into super-states T . Initially, we place all non-accepting S states into super-state T_0 and accepting states into super-state T_1 . Then, we examine the outgoing edges in each state $s \in T_i$. If, a given character c has edges that begin in T_i and end in *different* super-states, then we consider the super-state to be *inconsistent* with respect to c . (Consider an impermissible transition as if it were a transition to T_E , a super-state for errors.) The super-state must then be split into multiple states that are consistent with respect to c . Repeat this process for all super-states and all characters $c \in \Sigma$ until no more splits are required.

DFA Minimization Algorithm.

Given a DFA with states S , create an equivalent DFA with an equal or fewer number of states T .

First partition S into T such that:

T_0 = non-accepting states of S .

T_1 = accepting states of S .

Repeat:

$\forall T_i \in T$:

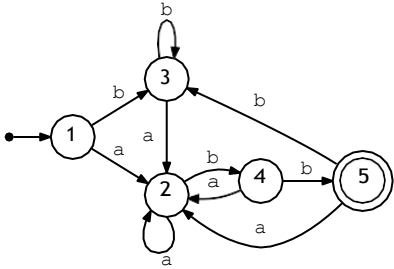
$\forall c \in \Sigma$:

if $T_i \xrightarrow{c} \{ \text{more than one } T \text{ state} \}$,
then split T_i into multiple T states
such that c has the same action in each.

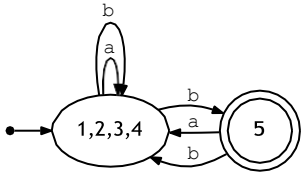
Until no more states are split.

Figure 3.5: Hopcroft's DFA Minimization Algorithm

Example. Suppose we have the following non-optimized DFA and wish to reduce it to a smaller DFA:

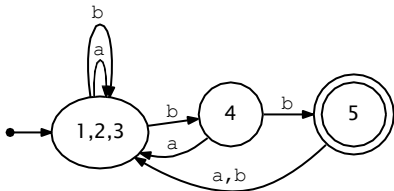


We begin by grouping all of non-accepting states 1, 2, 3, 4 into one super-state and the accepting state 5 into another super-state, like this:

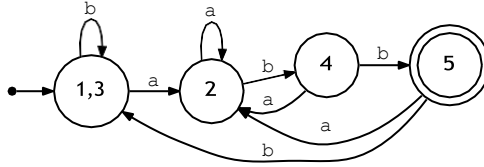


Now, we ask whether this graph is consistent with respect to all possible inputs, by referring back to the original DFA. For example, we observe that, if we are in super-state (1,2,3,4) then an input of *a* always goes to state 2, which keeps us within the super-state. So, this DFA is consistent with respect to *a*. However, from super-state (1,2,3,4) an input of *b* can either stay within the super-state or go to super-state (5). So, the DFA is inconsistent with respect to *b*.

To fix this, we try splitting out one of the inconsistent states (4) into a new super-state, taking the transitions with it:



Again, we examine each super-state for consistency with respect to each input character. Again, we observe that super-state 1,2,3 is consistent with respect to *a*, but not consistent with respect to *b* because it can either lead to state 3 or state 4. We attempt to fix this by splitting out state 2 into its own super-state, yielding this DFA.

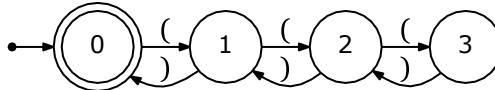


Again, we examine each super-state and observe that each possible input is consistent with respect to the super-state, and therefore we have the minimal DFA.

3.6 Limits of Finite Automata

Regular expressions and finite automata are powerful and effective at recognizing simple patterns in individual words or tokens, but they are not sufficient to analyze all of the structures in a problem. For example, could you use a finite automaton to match an arbitrary number of nested parentheses?

It's not hard to write out an FA that could match, say, up to three pairs of nested parentheses, like this:



But the key word is *arbitrary*! To match any number of parentheses would require an infinite automaton, which is obviously impractical. Even if we were to apply some practical upper limit (say, 100 pairs) the automaton would still be impractically large when combined with all the other elements of a language that must be supported.

For example, a language like Python permits the nesting of parentheses `()` for precedence, curly brackets `{ }` to represent dictionaries, and square brackets `[ ]` to represent lists. An automaton to match up to 100 nested pairs of each in arbitrary order would have 1,000,000 states!

So, we limit ourselves to using regular expressions and finite automata for the narrow purpose of identifying the words and symbols within a problem. To understand the higher level structure of a program, we will instead use parsing techniques introduced in Chapter 4.

```
%{  
    (C Preamble Code)  
%}  
    (Character Classes)  
%%  
    (Regular Expression Rules)  
%%  
    (Additional Code)
```

Figure 3.6: Structure of a Flex File

3.7 Using a Scanner Generator

Because a regular expression precisely describes all the allowable forms of a token, we can use a program to automatically transform a set of regular expressions into code for a scanner. Such a program is known as a **scanner generator**. The program `Lex`, developed at AT&T, was one of the earliest examples of a scanner generator. Vern Paxson translated `Lex` into the C language to create `Flex`, which is distributed under the Berkeley license and is widely used in Unix-like operating systems today to generate scanners implemented in C or C++.

To use Flex, we write a specification of the scanner that is a mixture of regular expressions, fragments of C code, and some specialized directives. The Flex program itself consumes the specification and produces regular C code that can then be compiled in the normal way.

Figure 3.6 gives the overall structure of a Flex file. The first section consists of arbitrary C code that will be placed at the beginning of `scanner.c`, like include files, type definitions, and similar things. Typically, this is used to include a file that contains the symbolic constants for tokens.

The second section declares character classes, which are symbolic short-hands for commonly used regular expressions. For example, you might declare `DIGIT [0-9]`. This class can be referred to later as `{DIGIT}`.

The third section is the most important part. It states a regular expression for each type of token that you wish to match, followed by a fragment of C code that will be executed whenever the expression is matched. In the simplest case, this code returns the type of the token, but it can also be used to extract token values, display errors, or anything else appropriate.

The fourth section is arbitrary C code that will go at the end of the scanner, typically for additional helper functions. A peculiar requirement of Flex is that we must define a function `yywrap()` which returns 1 to indicate that the input is complete at the end of the file. If we wanted to continue scanning in another file, then `yywrap()` would open the next file and return 0.

The regular expression language accepted by Flex is very similar to

that of formal regular expressions discussed above. The main difference is that characters that have special meaning with a regular expression (like parentheses, square brackets, and asterisks) must be escaped with a backslash or surrounded with double quotes. Also, a period (.) can be used to match any character at all, which is helpful for catching error conditions.

Figure 3.7 shows a simple but complete example to get you started. This specification describes just a few tokens: a single character addition (which must be escaped with a backslash), the `while` keyword, an identifier consisting of one or more letters, and a number consisting of one or more digits. As is typical in a scanner, any other type of character is an error, and returns an explicit token type for that purpose.

Flex generates the scanner code, but not a complete program, so you must write a `main` function to go with it. Figure 3.8 shows a simple driver program that uses this scanner. First, the main program must declare as `extern` the symbols it expects to use in the generated scanner code: `yyin` is the file from which text will be read, `yylex` is the function that implements the scanner, and the array `yytext` contains the actual text of each token discovered. Finally, we must have a consistent definition of the token types across the parts of the program, so into `token.h` we put an enumeration describing the new type `token_t`. This file is included in both `scanner.flex` and `main.c`.

Figure 3.10 shows how all the pieces come together. `scanner.flex` is converted into `scanner.c` by invoking `flex -o scanner.c scanner.flex`. Then, both `main.c` and `scanner.c` are compiled to produce object files, which are linked together to produce the complete program.

3.8 Practical Considerations

Handling keywords. In many languages, keywords (such as `while` or `if`) would otherwise match the definitions of identifiers, unless specially handled. There are several solutions to this problem. One is to enter a regular expression for every single keyword into the Flex specification. (These must precede the definition of identifiers, since Flex will accept the first expression that matches.) Another is to maintain a single regular expression that matches all identifiers and keywords. The action associated with that rule can compare the token text with a separate list of keywords and return the appropriate type. Yet another approach is to treat all keywords and identifiers as a single token type, and allow the problem to be sorted out by the parser. (This is necessary in languages like PL/1, where identifiers can have the same names as keywords, and are distinguished by context.)

Tracking source locations. In later stages of the compiler, it is useful for the parser or typechecker to know exactly what line and column number a token was located at, usually to print out a helpful error message.

Contents of File: scanner.flex

```
%{
#include "token.h"
}%
DIGIT [0-9]
LETTER [a-zA-Z]
%%
(" "|\t|\n) /* skip whitespace */
\+          { return TOKEN_ADD; }
while       { return TOKEN_WHILE; }
{LETTER}+   { return TOKEN_IDENT; }
{DIGIT}+    { return TOKEN_NUMBER; }
.           { return TOKEN_ERROR; }
%%
int yywrap() { return 1; }
```

Figure 3.7: Example Flex Specification**Contents of File: main.c**

```
#include "token.h"
#include <stdio.h>

extern FILE *yyin;
extern int yylex();
extern char *yytext;

int main() {
    yyin = fopen("program.c", "r");
    if(!yyin) {
        printf("could not open program.c!\n");
        return 1;
    }

    while(1) {
        token_t t = yylex();
        if(t==TOKEN_EOF) break;
        printf("token: %d  text: %s\n", t, yytext);
    }
}
```

Figure 3.8: Example Main Program

Contents of File: token.h

```
typedef enum {
    TOKEN_EOF=0,
    TOKEN_WHILE,
    TOKEN_ADD,
    TOKEN_IDENT,
    TOKEN_NUMBER,
    TOKEN_ERROR
} token_t;
```

Figure 3.9: Example Token Enumeration

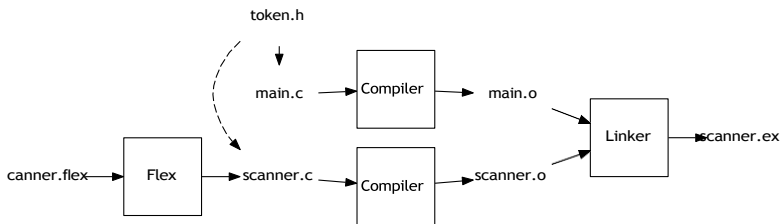


Figure 3.10: Build Procedure for a Flex Program

("Undefined symbol spider at line 153.") This is easily done by having the scanner match newline characters, and increase the line count (but not return a token) each time one is found.

Cleaning tokens. Strings, characters, and similar token types need to be cleaned up after they are matched. For example, `"hello\n"` needs to have its quotes removed and the backslash-n sequence converted to a literal newline character. Internally, the compiler only cares about the actual contents of the string. Typically, this is accomplished by writing a function `string_clean` in the postamble of the Flex specification. The function is invoked by the matching rule before returning the desired token type.

Constraining tokens. Although regular expressions can match tokens of arbitrary length, it does not follow that a compiler must be prepared to accept them. There would be little point to accepting a 1000-letter identifier, or an integer larger than the machine's word size. The typical approach is to set the maximum token length (`YYLMAX` in flex) to a very large value, then examine the token to see if it exceeds a logical limit in the action that matches the token. This allows you to emit an error message that describes the offending token as needed.

Error Handling. The easiest approach to handling errors or invalid input is simply to print a message and exit the program. However, this is unhelpful to users of your compiler – if there are multiple errors, it’s (usually) better to see them all at once. A good approach is to match the minimum amount of invalid text (using the dot rule) and return an explicit token type indicating an error. The code that invokes the scanner can then emit a suitable message, and then ask for the next token.

3.9 Exercises

1. Write regular expressions for the following entities. You may find it necessary to justify what is and is not allowed within each expression:

- (a) English days of the week: Monday, Tuesday, ...
- (b) All integers where every three digits are separated by commas for clarity, such as:
78
1,092
692,098,000
- (c) Internet email addresses like:
"John Doe" <john.doe@gmail.com>
- (d) HTTP Uniform Resource Locators (URLs)
as described by RFC-1738.

2. Write a regular expression for a string containing any number of `X` and single pairs of `< >` and `{ }` which may be nested but not interleaved. For example these strings are allowed:

```
XXX<XX{X}XXX>X
X{X}X<X>X{X}X<X>X
```

But these are not allowed:

```
XXX<X<XX>>XX
XX<XX{XX>XX}XX
```

3. Test the regular expressions you wrote in the previous two problems by translating them into your favorite programming language that has native support for regular expressions. (Perl and Python are two good choices.) Evaluate the correctness of your program by writing test cases that should (and should not) match.
4. Convert these REs into NFAs using Thompson’s construction:

- (a) `for | [a-z]+ | [xb]?[0-9]+`
- (b) `a ( bc*d | ed ) d*`

(c) $(a^*b \mid b^*a \mid ba)^*$

5. Convert the NFAs in the previous problem into DFAs using the subset construction method.
6. Minimize the DFAs in the previous problem by using Hopcroft's algorithm.
7. Write a hand-made scanner for JavaScript Object Notation (JSON) which is described at <http://json.org>. The program should read JSON on the input, and then print out the sequence of tokens observed: LBRACKET, STRING, COLON, etc... Find some large JSON documents online and test your scanner to see if it works.
8. Using Flex, write a scanner for the Java programming language. As above, read in Java source on the input and output token types. Test it out by applying it to a large open source project written in Java.

3.10 Further Reading

1. A.K. Dewdney, "The New Turing Omnibus: Sixty-Six Excursions in Computer Science", Holt Paperbacks, 1992. *An accessible overview of many fundamental problems in computer science – including finite state machines – collected from the author's Mathematical Recreations column in Scientific American.*
2. S. Hollos and J.R. Hollos, "Finite Automata and Regular Expressions: Problems and Solutions", Abrazol Publishing, 2013.
A collection of clever little problems and solutions relating to automata and state machines, if you are looking for more problems to work on.
3. Marvin Minsky, "Computation: Finite and Infinite Machines", Prentice-Hall, 1967.
A classic text offering a more thorough introduction to the theory of finite automata at an undergraduate level.
4. S. Kleene, "Representation of events in nerve nets and finite automata", Automata Studies, C. Shannon and J. McCarthy, editors, Princeton University Press, 1956.
5. R. McNaughton and H. Yamada, "Regular Expressions and State Graphs for Automata", IRE Transactions on Electronic Computers, volume EC-9, number 1, 1960.
<http://dx.doi.org/10.1109/TEC.1960.5221603>
6. K. Thompson, "Programming Techniques: Regular Expression Search Algorithm", Communications of the ACM, volume 11, number 6, 1968.
<http://dx.doi.org/10.1145/363347.363387>

Chapter 4 – Parsing

4.1 Overview

If scanning is like constructing words out of letters, then parsing is like constructing sentences out of words in a natural language. Of course, not every sequence of words makes a valid sentence: “horse aircraft conjugate” is three valid words, but not a meaningful sentence.

To parse a computer program, we must first describe the form of valid sentences in a language. This formal statement is known as a context free grammar (CFG). Because they allow for recursion, CFGs are more powerful than regular expressions and can express a richer set of structures.

While a plain CFG is relatively easy to write, it does not follow that it is easy to parse. An arbitrary CFG can contain ambiguities and other problems that make it difficult to write an automatic parser. Therefore, we consider two subsets of CFGs known as LL(1) and LR(1) grammars.

LL(1) grammars are CFGs that can be evaluated by considering only the current rule and next token in the input stream. This property makes it easy to write a hand-coded parser known as a recursive descent parser. However, a language (and its grammar) must be carefully designed (and occasionally rewritten) in order to ensure that it is an LL(1) grammar. Not all language structures can be expressed as LL(1) grammars.

LR(1) grammars are more general and more powerful than LL(1). Nearly all useful programming languages can be written in LR(1) form. However, the parsing algorithm for LR(1) grammars is more complex and usually cannot be written by hand. Instead, it is common to use a parser generator that will accept an LR(1) grammar and automatically generate the parsing code.

4.2 Context Free Grammars

Let's begin by defining the parts of a CFG.

A **terminal** is a discrete symbol that can appear in the language, otherwise known as a token from the previous chapter. Examples of terminals are keywords, operators, and identifiers. We will use lower-case letters to represent terminals. At this stage, we only need to consider the kind (e.g. integer literal) and not the value (e.g. 456) of a terminal.

A **non-terminal** represents a structure that can occur in a language, but is not a literal symbol. Example of non-terminals are declarations, statements, and expressions. We will use upper-case letters to represent non-terminals: P for program, S for statement, E for expression, etc.

A **sentence** is a valid sequence of terminals in a language, while a **sentential form** is a valid sequence of terminals and non-terminals. We will use Greek symbols to represent sentential forms. For example, α , β , and γ represent (possibly) mixed sequences of terminals and non-terminals. We will use a sequence like $Y_1Y_2\dots Y_n$ to indicate the individual symbols in a sentential form: Y_i may be either a terminal or a non-terminal.

A **context-free grammar (CFG)** is a list of **rules** that formally describe the allowable sentences in a language. The left-hand side of each rule is always a single non-terminal. The right-hand side of a rule is a sentential form that describes an allowable form of that non-terminal. For example, the rule $A \rightarrow xXy$ indicates that the non-terminal A represents a terminal x followed by a non-terminal X and a terminal y . The right hand side of a rule can be $\varnothing$ to indicate that the rule produces nothing. The first rule is special: it is the top-level definition of a program and its non-terminal is known as the **start symbol**.

For example, here is a simple CFG that describes expressions involving addition, integers, and identifiers:

Grammar G_2

1. $P \rightarrow E$
2. $E \rightarrow E + E$
3. $E \rightarrow \text{ident}$
4. $E \rightarrow \text{int}$

This grammar can be read as follows: (1) A complete program consists of one expression. (2) An expression can be any expression plus any expression. (3) An expression can be an identifier. (4) An expression can be an integer literal.

For brevity, we occasionally condense a set of rules with a common left-hand side by combining all of the right hand sides with a logical-or symbol, like this:

$$E \rightarrow E + E | \text{ident} | \text{int}$$

4.2.1 Deriving Sentences

Each grammar describes a (possibly infinite) set of sentences, which is known as the **language** of the grammar. To prove that a given sentence is a member of that language, we must show that there exists a sequence of rule applications that connects the start symbol with the desired sentence. A sequence of rule applications is known as a **derivation** and a double arrow ($\Rightarrow$) is used to show that one sentential form is equal to another by applying a given rule. For example:

- $E \Rightarrow \text{int}$ by applying rule 4 of Grammar G_2 .
- $E + E \Rightarrow E + \text{ident}$ by applying rule 3 of Grammar G_2 .
- $P \Rightarrow \text{int} + \text{ident}$ by applying all rules of Grammar G_2 .

There are two approaches to derivation: top-down and bottom-up. In **top-down derivation**, we begin with the start symbol, and then apply rules in the CFG to expand non-terminals until reaching the desired sentence. For example, **ident + int + int** is a sentence in this language, and here is one derivation to prove it:

Sentential Form	Apply Rule
P	$P \rightarrow E$
E	$E \rightarrow E + E$
E + E	$E \rightarrow \text{ident}$
ident + E	$E \rightarrow E + E$
ident + E + E	$E \rightarrow \text{int}$
ident + int + E	$E \rightarrow \text{int}$
ident + int + int	

In **bottom-up derivation**, we begin with the desired sentence, and then apply the rules backwards until reaching the start symbol. Here is a bottom-up derivation of the same sentence:

Sentential Form	Apply Rule
ident + int + int	$E \rightarrow \text{int}$
ident + int + E	$E \rightarrow \text{int}$
ident + E + E	$E \rightarrow E + E$
ident + E	$E \rightarrow \text{ident}$
E + E	$E \rightarrow E + E$
E	$P \rightarrow E$
P	

Be careful to distinguish between a *grammar* (which is a finite set of rules) and a *language* (which is a set of strings generated by a grammar). It is quite possible for two different grammars to generate the same language, in which case we describe them as having **weak equivalence**.

4.2.2 Ambiguous Grammars

An **ambiguous grammar** allows for more than one possible derivation of the same sentence. Our example grammar is ambiguous because there are two possible derivations for any sentence involving two plus signs. The sentence `ident + int + int` can have the two derivations shown in Figure 4.1.

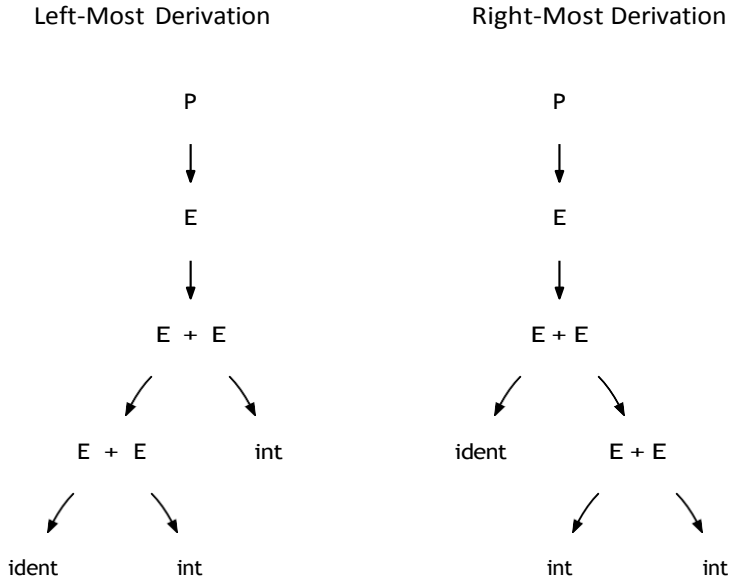


Figure 4.1: Two Derivations of the Same Sentence

Ambiguous grammars present a real problem for parsing (and language design in general) because we do not want a program to have two possible meanings.

Does it matter in this example? It certainly does! In a language like Java, the `+` operator indicates not only addition between integers, but also concatenation between strings. If the identifier is `hello` and the two integers have the value 5, then the left-most derivation would concatenate all three together into `hello55`, while the right-most derivation would compute `5+5=10` and concatenate the result into `hello10`.

Fortunately, it is usually possible to re-write a grammar so that it is not ambiguous. In the common case of binary operators, we can require that one side of the expression be an atomic term (T), like this:

Grammar G_3

1. $P \rightarrow E$
2. $E \rightarrow E + T$
3. $E \rightarrow T$
4. $T \rightarrow \text{ident}$
5. $T \rightarrow \text{int}$

With this change, the grammar is no longer ambiguous, because it only allows a left-most derivation. But also note that it *still accepts the same language as Grammar G_2* . That is, any sentence that can be derived by Grammar G_2 can also be derived by Grammar G_3 , but there exists only one derivation (and one meaning) per sentence. (Proof is left as an exercise to the reader.)

Now suppose that we would like to add more operators to our grammar. If we simply add more rules of the form $E \rightarrow E * T$ and $E \rightarrow E \div T$, we would still have an unambiguous grammar, but it would not follow the rules of precedence in algebra: each operator would be applied from left to right.

Instead, the usual approach is to construct a grammar with multiple levels that reflect the intended precedence of operators. For example, we can combine addition and multiplication by expressing them as a sum of terms (T) that consist of multiplied factors (F), like this:

Grammar G_4

1. $P \rightarrow E$
2. $E \rightarrow E + T$
3. $E \rightarrow T$
4. $T \rightarrow T * F$
5. $T \rightarrow F$
6. $F \rightarrow \text{ident}$
7. $F \rightarrow \text{int}$

Here is another common example that occurs in most programming languages in some form or another. Suppose that an `if` statement has two variations: an if-then which takes an action when an expression is true, and an if-then-else that takes a different action for the true and false cases. We can express this fragment of the language like this:

Grammar G_5

1. $P \rightarrow S$
2. $S \rightarrow \text{if } E \text{ then } S$
3. $S \rightarrow \text{if } E \text{ then } S \text{ else } S$
4. $S \rightarrow \text{other}$

Grammar G_5 is ambiguous because it allows for two derivations of this sentence: `if E then if E then other else other`. Do you see the problem? The `else` part could belong to the outer `if` or to the inner `if`. In most programming languages, the `else` is defined as belonging to the inner `if`, but the grammar does not reflect this.

Do this now:

Write out the two possible parse trees for this sentence:
`if E then if E then other else other`.

4.3 LL Grammars

LL(1) grammars are a subset of CFGs that are easy to parse with simple algorithms. A grammar is LL(1) if it can be parsed by considering only one non-terminal and the next token in the input stream.

To ensure that a grammar is LL(1), we must do the following:

- Remove any ambiguity, as shown above.
- Eliminate any left recursion, as shown below.
- Eliminate any common left prefixes, as shown below.

Once we have taken those steps, then we can prove that it is LL(1) by generating the FIRST and FOLLOW sets for the grammar, and using them to create the LL(1) parse table. If the parse table contains no conflicts, then the grammar is clearly LL(1).

4.3.1 Eliminating Left Recursion

LL(1) grammars cannot contain **left recursion**, which is a rule of the form $A \rightarrow A\alpha$ or, more generally, any rule $A \rightarrow B\beta$ such that $B \Rightarrow A\gamma$ by some sequence of derivations. For example, the rule $E \rightarrow E + T$ is left-recursive because E appears as the first symbol on the right hand side.

You might be tempted to solve the problem by simply re-writing the rule as $E \rightarrow T + E$. While that would avoid left recursion, it would not be an equivalent grammar because it would result in a right-associative plus operator. Also, it would introduce the new problem of a common left prefix, discussed below.

Informally, we must re-write the rules so that the (formerly) recursive rule begins with the leading symbols of its alternatives.

Formally, if you have a grammar of the form:

$$A \rightarrow A\alpha_1 | A\alpha_2 | \dots | \beta_1 | \beta_2 | \dots$$

Substitute with:

$$A \rightarrow \beta_1 A' | \beta_2 A' | \dots$$

$$A' \rightarrow \alpha_1 A' | \alpha_2 A' | \dots | \varnothing$$

Applying this rule to grammar Grammar G_3 , we can re-write it as:

Grammar G_6

1. $P \rightarrow E$
2. $E \rightarrow T E'$
3. $E' \rightarrow + T E'$
4. $E' \rightarrow \varnothing$
5. $T \rightarrow \text{ident}$
6. $T \rightarrow \text{int}$

While Grammar G_6 is perhaps slightly harder for a person to read, it no longer contains left recursion, and it satisfies all the LL(1) properties. A parser considering an E in rule 2 will immediately consider the T non-terminal, and then look at `ident` or `int` on the input to decide between rule 5 and 6. After considering T , the parser moves on to consider E' and can distinguish between rule 3 and 4 by looking for either a `+` or any other symbol on the input.

4.3.2 Eliminating Common Left Prefixes

A simpler problem to solve is grammars that have multiple rules with the same left hand side and a common prefix of tokens on the right hand side. Informally, we simply look for all common prefixes of a given non-terminal, and replace them with one rule that contains the prefix and another that contains the variants.

Formally, look for rules of this form:

$$A \rightarrow \alpha\theta_1 | \alpha\theta_2 | \dots$$

And replace with:

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \theta_1 | \theta_2 | \dots$$

For example, these rules describing an identifier, array reference, and function call all share the same prefix of a single identifier:

Grammar G_7

1. $P \rightarrow E$
2. $E \rightarrow \text{id}$
3. $E \rightarrow \text{id} [E]$
4. $E \rightarrow \text{id} (E)$

If a parser is evaluating E and sees an `id` on the input, that information is not sufficient to distinguish between rules 2, 3, and 4. However, the grammar can be salvaged by factoring out the common prefix, like this:

Grammar G_8

1. $P \rightarrow E$
2. $E \rightarrow \text{id } E'$
3. $E' \rightarrow [E]$
4. $E' \rightarrow (E)$
5. $E' \rightarrow \varnothing$

In this formulation, the parser always consumes an `id` when evaluating an E . If the next token is `[`, then rule 3 is applied. If the next token is `(`, then rule 4 is applied; otherwise, rule 5 is applied.

4.3.3 First and Follow Sets

In order to construct a complete parser for an LL(1) grammar, we must compute two sets, known as FIRST and FOLLOW. Informally, $\text{FIRST}(\alpha)$ indicates the set of terminals (including $\varnothing$) that could potentially appear at the beginning of any derivation of α . $\text{FOLLOW}(A)$ indicates the set of terminals (including $\$$) that could potentially occur after any derivation of the non-terminal A . Given the contents of these sets, an LL(1) parser will always know which rule to pick next.

Here is how to compute FIRST and FOLLOW:

Computing First Sets for a Grammar G

$\text{FIRST}(\alpha)$ is the set of terminals that begin all strings given by α , including $\varnothing$ if $\alpha \Rightarrow \varnothing$.

For Terminals:

For each terminal $a \in \Sigma$: $\text{FIRST}(a) = \{a\}$

For Non-Terminals:

Repeat:

For each rule $X \rightarrow Y_1 Y_2 \dots Y_k$ in a grammar G :

Add a to $\text{FIRST}(X)$

if a is in $\text{FIRST}(Y_1)$

or a is in $\text{FIRST}(Y_n)$ and $Y_1 \dots Y_{n-1} \Rightarrow \varnothing$

If $Y_1 \dots Y_k \Rightarrow \varnothing$ then add $\varnothing$ to $\text{FIRST}(X)$.

until no more changes occur.

For a Sentential Form α :

For each symbol $Y_1 Y_2 \dots Y_k$ in α :

Add a to $\text{FIRST}(\alpha)$

if a is in $\text{FIRST}(Y_1)$

or a is in $\text{FIRST}(Y_n)$ and $Y_1 \dots Y_{n-1} \Rightarrow \varnothing$

If $Y_1 \dots Y_k \Rightarrow \varnothing$ then add $\varnothing$ to $\text{FIRST}(\alpha)$.

Computing Follow Sets for a Grammar G

$\text{FOLLOW}(A)$ is the set of terminals that can come after non-terminal A , including $\$$ if A occurs at the end of the input.

$\text{FOLLOW}(S) = \{\$\}$ where S is the start symbol.

Repeat:

 If $A \rightarrow \alpha B \beta$ then:

 add $\text{FIRST}(\beta)$ (excepting $\varnothing$) to $\text{FOLLOW}(B)$.

 If $A \rightarrow \alpha B$ or $\text{FIRST}(\beta)$ contains $\varnothing$ then:

 add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$.

until no more changes occur.

Here is an example of computing FIRST and FOLLOW for Grammar G_9 :

Grammar G_9

1. $P \rightarrow E$

2. $E \rightarrow T E'$

3. $E' \rightarrow + T E'$

4. $E' \rightarrow \varnothing$

5. $T \rightarrow F T'$

6. $T' \rightarrow * F T'$

7. $T' \rightarrow \varnothing$

8. $F \rightarrow (E)$

9. $F \rightarrow \text{int}$

First and Follow for Grammar G_9

	P	E	E'	T	T'	F
FIRST	(int	(int	+ $\varnothing$	(int	* $\varnothing$	(int
FOLLOW	\$	) \$	) \$	+) \$	+) \$	+ *) \$

Once we have cleaned up a grammar to be LL(1) and computed its FIRST and FOLLOW sets, we are ready to write code for a parser. This can be done by hand or with a table-driven approach.

4.3.4 Recursive Descent Parsing

LL(1) grammars are very amenable to writing simple hand-coded parsers. A common approach is a **recursive descent parser** in which there is one simple function for each non-terminal in the grammar. The body of the function follows the right-hand sides of the corresponding rules: non-terminals result in a call to another parse function, while terminals result in considering the next token.

Three helper functions are needed:

- `scan_token()` returns the next token on the input stream.
- `putback_token(t)` puts an unexpected token back on the input stream, where it will be read again by the next call to `scan_token`.
- `expect_token(t)` calls `scan_token` to retrieve the next token. It returns true if the token matches the expected type. If not, it puts the token back on the input stream and returns false.

Figure 4.2 shows how Grammar G_9 could be written as a recursive descent parser. Note that the parser has one function for each non-terminal: `parse_P`, `parse_E`, etc. Each function returns true (1) if the input matches the grammar, or false (0) otherwise.

Two special cases should be considered. First, if a rule X cannot produce $\varnothing$ and we encounter a token not in $\text{FIRST}(X)$, then we have definitely encountered a parsing error, and we should display a message and return failure. Second, if a rule X *could* produce $\varnothing$ and we encounter a token not in $\text{FIRST}(X)$, then we accept the rule $X \rightarrow \varnothing$ put the token back on the input, and return success. Another rule will expect to consume that token.

There is also the question of what the parser should actually *do* after matching some element of the grammar. In our simple example, the parser simply returns true on a match, and serves only to verify that the input program matches the grammar. If we wished to actually evaluate the expression, each `parse_X` function could compute the result and return it as a `double`. This would effectively give us a simple interpreter for this language. Another approach is for each `parse_X` function to return a data structure representing that node of the parse tree. As each node is parsed, the result is assembled into an abstract syntax tree, with the root returned by `parse_P`.

```
int parse_P() {
    return parse_E() && expect_token(TOKEN_EOF);
}

int parse_E() {
    return parse_T() && parse_E_prime();
}

int parse_E_prime() {
    token_t t = scan_token();
    if(t==TOKEN_PLUS) {
        return parse_T() && parse_E_prime();
    } else {
        putback_token(t);
        return 1;
    }
}

int parse_T() {
    return parse_F() && parse_T_prime();
}

int parse_T_prime() {
    token_t t = scan_token();
    if(t==TOKEN_MULTIPLY) {
        return parse_F() && parse_T_prime();
    } else {
        putback_token(t);
        return 1;
    }
}

int parse_F() {
    token_t t = scan_token();
    if(t==TOKEN_LPAREN) {
        return parse_E() && expect_token(TOKEN_RPAREN);
    } else if(t==TOKEN_INT) {
        return 1;
    } else {
        printf("parse error: unexpected token %s\n",
            token_string(t));
        return 0;
    }
}
```

Figure 4.2: A Recursive-Descent Parser

4.3.5 Table Driven Parsing

An LL(1) grammar can also be parsed using generalized table driven code. A table-driven parser requires a grammar, a parse table, and a stack to represent the current set of non-terminals.

The **LL(1) parse table** is used to determine which rule should be applied for any combination of non-terminal on the stack and next token on the input stream. (By definition, an LL(1) grammar has exactly one rule to be applied for each combination.) To create a parse table, we use the FIRST and FOLLOW sets like this:

LL(1) Parse Table Construction.

Given a grammar G and alphabet Σ , create a parse table $T[A, a]$ that selects a rule for each combination of non-terminal $A \in G$ and terminal $a \in \Sigma$.

For each rule $A \rightarrow \alpha$ in G :

 For each terminal a (excepting φ) in $\text{FIRST}(\alpha)$:

 Add $A \rightarrow \alpha$ to $T[A, a]$.

 If φ is in $\text{FIRST}(\alpha)$:

 For each terminal b (including $\$$) in $\text{FOLLOW}(A)$:

 Add $A \rightarrow \alpha$ to $T[A, b]$.

For example, here is the parse table for Grammar G_9 . Notice that the entries for P , E , T , and F are straightforward: each can only start with `int` or `(`, and so these tokens cause the rules to descend toward F and a choice between rule 8 ($F \rightarrow \text{int}$) and rule 9 ($F \rightarrow (E)$). The entry for E' is a little more complicated: a `+` token results in applying $E' \rightarrow +TE'$, while `)` or `$` indicates $E' \rightarrow \varphi$.

Parse Table for Grammar G_9 :

	int	+	*	(	)	\$
P	1			1		
E	2			2		
E'		3			4	4
T	5			5		
T'		7	6		7	7
F	9			8		

Now we have all the pieces necessary to operate the parser. Informally, the idea is to keep a stack that tracks the current state of the parser. In each step, we consider the top element of the stack and the next token on the input. If they match, then pop the stack, accept the token, and continue. If not, then consult the parse table for the next rule to apply. If we can continue until the end-of-file symbol is matched, then the parse succeeds.

LL(1) Table Parsing Algorithm.
Given a grammar G with start symbol P and parse table T , parse a sequence of tokens and determine whether they satisfy G .

Create a stack S .
Push $\$$ and P onto S .
Let c be the first token on the input.

While S is not empty:
 Let X be the top element of the stack.
 If X matches c :
 Remove X from the stack.
 Advance c to the next token and repeat.
 If X is any other terminal, stop with an error.
 If $T[X, c]$ indicates rule $X \rightarrow \alpha$:
 Remove X from the stack.
 Push symbols α on to the stack and repeat.
 If $T[X, c]$ indicates an error state, stop with an error.

Here is an example of the algorithm applied to the sentence `int * int`:

Stack	Input	Action
P \$	int * int \$	apply 1: $P \Rightarrow E$
E \$	int * int \$	apply 2: $E \Rightarrow T E'$
T E' \$	int * int \$	apply 5: $T \Rightarrow F T'$
F T' E' \$	int * int \$	apply 9: $F \Rightarrow \text{int}$
int T' E' \$	int * int \$	match int
T' E' \$	* int \$	apply 6: $T' \Rightarrow * F T'$
* F T' E' \$	* int \$	match *
F T' E' \$	int \$	apply 9: $F \Rightarrow \text{int}$
int T' E' \$	int \$	match int
T' E' \$	\$	apply 7: $T' \Rightarrow \varnothing$
E' \$	\$	apply 4: $E' \Rightarrow \varnothing$
\$	\$	match \$

4.4 LR Grammars

While LL(1) grammars and top-down parsing techniques are easy to work with, they are not able to represent all of the structures found in many programming languages. For more general-purpose programming languages, we must use an LR(1) grammar and associated bottom-up parsing techniques.

LR(1) is the set of grammars that can be parsed via shift-reduce techniques with a single token of lookahead. LR(1) is a super-set of LL(1) and can accommodate left recursion and common left prefixes which are not permitted in LL(1). This enables us to express many programming constructs in a more natural way. (An LR(1) grammar must still be non-ambiguous, and it cannot have shift-reduce or reduce-reduce conflicts, which we will explain below.)

For example, Grammar G_{10} is an LR(1) grammar:

Grammar G_{10}

1. $P \rightarrow E$

2. $E \rightarrow E + T$

3. $E \rightarrow T$

4. $T \rightarrow id (E)$

5. $T \rightarrow id$

We need to know the FIRST and FOLLOW sets of LR(1) grammars as well, so take a moment now and work out the sets for Grammar G_{10} , using the same technique from section 4.3.3.

	P	E	T
FIRST			
FOLLOW			

4.4.1 Shift-Reduce Parsing

LR(1) grammars must be parsed using the **shift-reduce** parsing technique. This is a bottom-up parsing strategy that begins with the tokens and looks for rules that can be applied to reduce sentential forms into non-terminals. If there is a sequence of reductions that leads to the start symbol, then the parse is successful.

A **shift** action consumes one token from the input stream and pushes it onto the stack. A **reduce** action applies one rule of the form $A \rightarrow \alpha$ from the grammar, replacing the sentential form α on the stack with the non-terminal A . For example, here is a shift-reduce parse of the sentence `id(id+id)` using Grammar G_{10} :

Stack	Input	Action
	id (id + id) \$	shift
id	(id + id) \$	shift
id (	id + id) \$	shift
id (id	+ id) \$	reduce $T \rightarrow id$
id (T	+ id) \$	reduce $E \rightarrow T$
id (E	+ id) \$	shift
id (E +	id) \$	shift
id (E + id	) \$	reduce $T \rightarrow id$
id (E + T	) \$	reduce $E \rightarrow E + T$
id (E	) \$	shift
id (E)	\$	reduce $T \rightarrow id(E)$
T	\$	reduce $E \rightarrow T$
E	\$	reduce $P \rightarrow E$
P	\$	accept

While this example shows that there exists a derivation for the sentence, it does not explain how each action was chosen at each step. For example, in the second step, we might have chosen to reduce `id` to T instead of shifting a left parenthesis. This would have been a bad choice, because there is no rule that begins with T (, but that was not immediately obvious without attempting to proceed further. To make these decisions, we must analyze LR(1) grammars in more detail.

4.4.2 The LR(0) Automaton

An **LR(0) automaton** represents all the possible rules that are currently under consideration by a shift-reduce parser. (The LR(0) automaton is also variously known as the **canonical collection** or the **compact finite state machine** of the grammar.) Figure 4.6 shows a complete automaton for Grammar G_{10} . Each box represents a state in the machine, connected by transitions for both terminals and non-terminals in the grammar.

Each state in the automaton consists of multiple **items**, which are rules augmented by a **dot** (.) that indicates the parser's current position in that rule. For example, the configuration $E \rightarrow E . + T$ indicates that E is currently on the stack, and $+ T$ is a possible next sequence of tokens.

The automaton is constructed as follows. State 0 is created by taking the production for the start symbol ($P \rightarrow E$) and adding a dot at the beginning of the right hand side. This indicates that we expect to see a complete program, but have not yet consumed any symbols. This is known as the **kernel** of the state.

Kernel of State 0

$P \rightarrow . E$

Then, we compute the **closure** of the state as follows. For each item in the state with a non-terminal X immediately to the right of the dot, we add all rules in the grammar that have X as the left hand side. The newly added items have a dot at the beginning of the right hand side.

$P \rightarrow . E$
$E \rightarrow . E + T$
$E \rightarrow . T$

The procedure continues until no new items can be added:

Closure of State 0

$P \rightarrow . E$
$E \rightarrow . E + T$
$E \rightarrow . T$
$T \rightarrow . id (E)$
$T \rightarrow . id$

You can think of the state this way: It describes the initial state of the parser as expecting a complete program in the form of an E . However, an E is known to begin with an E or a T , and a T must begin with an id . All of those symbols could represent the beginning of the program.

From this state, all of the symbols (terminals and non-terminals both) to the right of the dot are possible outgoing transitions. If the automaton takes that transition, it moves to a new state containing the matching items, with the dot moved one position to the right. The closure of the new state is computed, possibly adding new rules as described above.

For example, from state zero, E , T , and id are the possible transitions, because each appears to the right of the dot in some rule. Here are the states for each of those transitions:

Transition on E :

$P \rightarrow E .$ $E \rightarrow E . + T$
--

Transition on T :

$E \rightarrow T .$

Transition on id :

$T \rightarrow id . (E)$ $T \rightarrow id .$
--

Figure 4.3 gives the complete LR(0) automaton for Grammar G_{10} . Take a moment now to trace over the table and be sure that you understand how it is constructed.

No, really. Stop now and study the figure carefully before continuing.

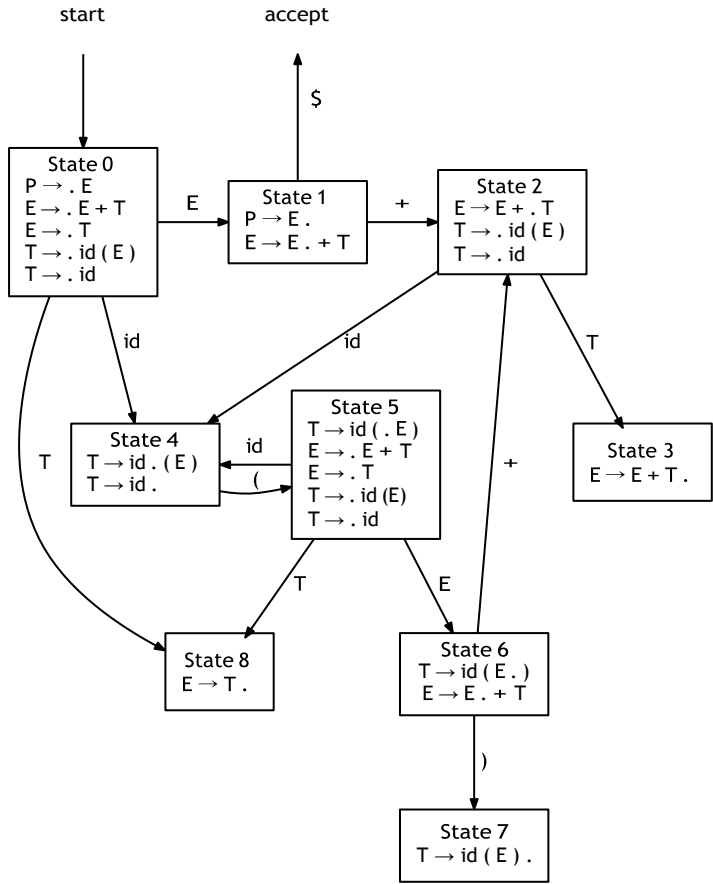


Figure 4.3: LR(0) Automaton for Grammar G_{10}

The LR(0) automaton tells us the choices available at any step of bottom up parsing. When we reach a state containing an item with a dot at the end of the rule, that indicates a possible reduction. A transition on a terminal that moves the dot one position to the right indicates a possible shift. While the LR(0) automaton tells us the available actions at each step, it does not always tell us *which* action to take.¹

Two types of conflicts can appear in an LR grammar:

A **shift-reduce conflict** indicates a choice between a shift action and a reduce action. For example, state 4 offers a choice between shifting a left parenthesis and reducing by rule five:

Shift-Reduce Conflict:

$T \rightarrow \text{id} . (E)$
 $T \rightarrow \text{id} .$

A **reduce-reduce conflict** indicates that two distinct rules have been completely matched, and either one could apply. While Grammar G_{10} does not contain any reduce-reduce conflicts, they commonly occur when a syntactic structure occurs at multiple layers in a grammar. For example, it is often the case that a function invocation can be a statement by itself or an element within an expression. The automaton for such a grammar would contain a state like this:

Reduce-Reduce Conflict:

$S \rightarrow \text{id} (E) .$
 $E \rightarrow \text{id} (E) .$

The LR(0) automaton forms the basis of LR parsing, by telling us which actions are available in each state. But, it does not tell us *which* action to take or how to resolve shift-reduce and reduce-reduce conflicts. To do that, we must take into account some additional information.

¹The 0 in LR(0) indicates that it uses zero lookahead tokens, which is a way of saying that it does not consider the next token before making a reduction. While it is possible to write out a grammar that is strictly LR(0), such a grammar has very limited utility.

Chapter 5 – Semantic Analysis

Now that we have completed construction of the AST, we are ready to begin analyzing the **semantics**, or the actual meaning of a program, and not simply its structure.

Type checking is a major component of semantic analysis. Broadly speaking, the type system of a programming language gives the programmer a way to make verifiable assertions that the compiler can check automatically. This allows for the detection of errors at compile-time, instead of at runtime.

Different programming languages have different approaches to type checking. Some languages (like C) have a rather weak type system, so it is possible to make serious errors if you are not careful. Other languages (like Ada) have very strong type systems, but this makes it more difficult to write a program that will compile at all!

Before we can perform type checking, we must determine the type of each identifier used in an expression. However, the mapping between variable names and their actual storage locations is not immediately obvious. A variable x in an expression could refer to a local variable, a function parameter, a global variable, or something else entirely. We solve this problem by performing **name resolution**, in which each definition of a variable is entered into a **symbol table**. This table is referenced throughout the semantic analysis stage whenever we need to evaluate the correctness of some code.

Once name resolution is completed, we have all the information necessary to check types. In this stage, we compute the type of complex expressions by combining the basic types of each value according to standard conversion rules. If a type is used in a way that is not permitted, the compiler will output an (ideally helpful) error message that will assist the programmer in resolving the problem.

Semantic analysis also includes other forms of checking the correctness of a program, such as examining the limits of arrays, avoiding bad pointer traversals, and examining control flow. Depending on the design of the language, some of these problems can be detected at compile time, while others may need to wait until runtime.

5.1 Overview of Type Systems

Most programming languages assign to every value (whether a literal, constant, or variable) a **type**, which describes the interpretation of the data in that variable. The type indicates whether the value is an integer, a floating point number, a boolean, a string, a pointer, or something else. In most languages, these atomic types can be combined into higher-order types such as enumerations, structures, and variant types to express complex constraints.

The type system of a language serves several purposes:

- **Correctness.** A compiler uses type information provided by the programmer to raise warnings or errors if a program attempts to do something improper. For example, it is almost certainly an error to assign an integer value to a pointer variable, even though both might be implemented as a single word in memory. A good type system can help to eliminate runtime errors by flagging them at compile time instead.
- **Performance.** A compiler can use type information to find the most efficient implementation of a piece of code. For example, if the programmer tells the compiler that a given variable is a constant, then the same value can be loaded into a register and used many times, rather than constantly loading it from memory.
- **Expressiveness.** A program can be made more compact and expressive if the language allows the programmer to leave out facts that can be inferred from the type system. For example, in B-Minor the `print` statement does not need to be told whether it is printing an integer, a string, or a boolean: the type is inferred from the expression and the value is automatically displayed in the proper way.

A programming language (and its type system) are commonly classified on the following axes:

- safe or unsafe
- static or dynamic
- explicit or implicit

In an **unsafe programming language**, it is possible to write valid programs that have wildly undefined behavior that violates the basic structure of the program. For example, in the C programming language, a program can construct an arbitrary pointer to modify any word in memory, and thereby change the data and code of the compiled program. Such power is probably necessary to implement low-level code like an operating system or a driver, but is problematic in general application code.

For example, the following code in C is syntactically legal and will compile, but is unsafe because it writes data outside the bounds of the array `a[]`. As a result, the program could have almost any outcome, including incorrect output, silent data corruption, or an infinite loop.

```
/* This is C code */
int i;
int a[10];
for(i=0;i<100;i++) a[i] = i;
```

In a **safe programming language**, it is not possible to write a program that violates the basic structures of the language. That is, no matter what input is given to a program written in a safe language, it will always execute in a well-defined way that preserves the abstractions of the language. A safe programming language enforces the boundaries of arrays, the use of pointers, and the assignment of types to prevent undefined behavior. Most interpreted languages, like Perl, Python, and Java, are safe languages.

For example, in C#, the boundaries of arrays are checked at runtime, so that running off the end of an array has the predictable effect of throwing an `IndexOutOfRangeException`:

```
/* This is C-sharp code */
a = new int[10];
for(int i=0;i<100;i++) a[i] = i;
```

In a **statically typed language**, all typechecking is performed at compile-time, long before the program runs. This means that the program can be translated into basic machine code without retaining any of the type information, because all operations have been checked and determined to be safe. This yields the most high performance code, but it does eliminate some kinds of convenient programming idioms.

Static typing is often used to distinguish between integer and floating point operations. While operations like addition and multiplication are usually represented by the same symbols in the source language, they are implemented with fundamentally different machine code. For example, in the C language on X86 machines, `(a+b)` would be translated to an `ADDL` instruction for integers, but an `FADD` instruction for floating point values. To know which instruction to apply, we must first determine the type of `a` and `b` and deduce the intended meaning of `+`.

In a **dynamically typed language**, type information is available at runtime and stored in memory alongside the data that it describes. As the program executes, the safety of each operation is checked by comparing the types of each operand. If types are observed to be incompatible, then the program must halt with a runtime type error. This also allows for

code that can explicitly examine the type of a variable. For example, the `instanceof` operator in Java allows one to test for types explicitly:

```
/* This is Java code */

public void sit( Furniture f ) {
    if (f instanceof Chair) {
        System.out.println("Sit up straight!");
    } else if ( f instanceof Couch ) {
        System.out.println("You may slouch.");
    } else {
        System.out.println("You may sit normally.");
    }
}
```

In an **explicitly typed language**, the programmer is responsible for indicating the types of variables and other items in the code explicitly. This requires more effort on the programmer's part but reduces the possibility of unexpected errors. For example, in an explicitly typed language like C, the following code might result in an error or warning, due to the loss of precision when assigning a floating point to an integer:¹

```
/* This is C code */
int x = 32.5;
```

Explicit typing can also be used to prevent assignment between variables that have the same underlying representation, but different meanings. For example, in C and C++, pointers to different types have the same implementation (a pointer) but it makes no sense to interchange them. The following should generate an error or at least a warning:

```
/* This is C code */
int *i;
float *f = i;
```

In an **implicitly typed language**, the compiler will infer the type of variables and expressions (to the degree possible) without explicit input from the programmer. This allows for programs to be more compact but can result in accidental behavior. For example, recent C++ standards allow a variable to be declared with automatic type `auto`, like this:

```
/* This is C++11 code */
auto x = 32.5;
cout << x << endl;
```

¹Not all C compilers will generate a warning, but they should!

The compiler determines that 32.5 has type `double`, and therefore `x` must also have type `double`. In a similar way, the output operator `<<` is defined to have a certain behavior on integers, another behavior on strings, and so forth. In this case, the compiler already determined that the type of `x` is `double` and so it chooses the variant of `<<` that operates on doubles.

5.2 Designing a Type System

To describe the type system of a language, we must explain its atomic types, its compound types, and the rules for assigning and converting between types.

The **atomic types** of a language are the simple types used to describe individual variables that are typically (though not always) stored in single registers in assembly language: integers, floating point numbers, boolean values, and so forth. For each atomic type, it is necessary to clearly define the range that is supported. For example, integers may be signed or unsigned, be 8 or 16 or 32 or 64 bits; floating point numbers could be 32 or 40 or 64 bits; characters could be ASCII or Unicode.

Many languages allow for **user-defined types** in which the programmer defines a new type that is implemented using an atomic type but gives it a new meaning by restricting the range. For example, in Ada, you might define new types for days and months:

```
-- This is Ada code
type Day is range 1..31;
type Month is range 1..12;
```

This is useful because variables and functions dealing with days and months are now kept separate, preventing you from accidentally assigning one to another, or for giving the value 13 to a variable of type `Month`.

C has a similar feature, but it is much weaker: `typedef` declares a new name for a type, but doesn't have any means of restricting the range, and doesn't prevent you from making assignments between types that share the same base type:

```
/* This is C code */
typedef int Month;
typedef int Day;

/* Assigning m to d is allowed in C,
   because they are both integers. */

Month m = 10;
Day d = m;
```

Enumerations are another kind of user-defined type in which the programmer indicates a finite set of symbolic values that a variable can contain. For example, if you are working with uncertain boolean variables in Rust, you might declare:

```
/* This is Rust code */
enum Fuzzy { True, False, Uncertain };
```

Internally, an enumeration value is simply an integer, but it makes the source code more readable, and also allows the compiler to prevent the programmer from assigning an illegal value. Once again, the C language allows you to declare enumerations, but doesn't prevent you from mixing and matching integers and enumerations.

The **compound types** of a language combine together existing types into more complex aggregations. You are surely familiar with a **structure type** (or **record type**) that groups together several values into a larger whole. For example, you might group together latitude and longitude to treat them as a single `coordinates` structure:

```
/* This is Go code */
type coordinates struct {
    latitude float64
    longitude float64
}
```

Less frequently used are **union types** in which multiple symbols occupy the same memory. For example, in C, you can declare a union type `of number` that contains an overlapping float and integer:

```
/* This is C code */
union number {
    int i;
    float f;
};

union number n;
n.i = 10;
n.f = 3.14;
```

In this case, `n.i` and `n.f` occupy the same memory. If you assign 10 to `n.i` and read it back, you will see 10 as expected. However, if you assign 10 to `n.i` and read back `n.f`, you will likely observe a garbage value, depending on how exactly the two values are mapped into memory. Union types are occasionally handy when implementing operating system features such as device drivers, because hardware interfaces often re-use the same memory locations for multiple purposes.

Some languages provide a **variant type** which allows the programmer to explicitly describe a type with multiple variants, each with different fields. This is similar to the concept of a union type, but prevents the programmer from performing unsafe accesses. For example, Rust allows us to create a variant type representing an expression tree:

```
/* This is Rust code */
enum Expression {
    ADD{ left: Expression, right: Expression },
    MULTIPLY{ left: Expression, right: Expression },
    INTEGER{ value: i32 },
    NAME{ name: string }
}
```

This variant type is tightly controlled so that it is difficult to use incorrectly. For an Expression of type `ADD`, it has `left` and `right` fields which can be used in the expected way. For an Expression of type `NAME`, the `name` field can be used. The other fields are simply not available unless the appropriate type is selected.

Finally, we must define what happens when unlike types are used together. Suppose that an integer `i` is assigned to a floating point `f`. A similar situation arises when an integer is passed to a function expecting a floating point as an argument. There are several possibilities for what a language may do in this case:

- **Disallow the assignment.** A very strict language (like B-Minor) could simply emit an error and prevent the program from compiling! Perhaps it simply makes no sense to make the assignment, and the compiler is saving the programmer from a grievous error. If the assignment is *really* desired, it could be accomplished by requiring that the programmer call a built-in conversion function (e.g. `IntToFloat`) that accepts one type and returns another.
- **Perform a bitwise copy.** If the two variables have the same underlying storage size, the unlike assignment could be accomplished by just copying the bits in one variable to the location of the other. This is *usually* a bad idea, since there is no guarantee that one data type has any meaning in the other context. But it does happen in a few select cases, such as when assigning different pointer types in C.
- **Convert to an equivalent value.** For certain types, the compiler may have built-in conversions that change the value to the desired type implicitly. For example, it is common to implicitly convert between integers and floating points, or between signed and unsigned integers. But this does not mean the operation is safe! An implied conversion can lose information, resulting in very tricky bugs.

- **Interpret the value in a different way.** In some cases, it may be desirable to convert the value into some other value that is not equivalent but still useful for the programmer. For example, in Perl, when a list is copied to a scalar context, the *length* of the list is placed in the target variable, rather than the content of the list.

```
@days = ("Monday", "Tuesday", "Wednesday", ... );
@a = @days; # copies the array to array a
$b = @days; # puts the length of the array into b
```

5.3 The B-Minor Type System

The B-Minor type system is safe, static, and explicit. As a result, it is fairly compact to describe, straightforward to implement, and eliminates a large number of programming errors. However, it may be more strict than some languages, so there will be a large number of errors that we must detect.

B-Minor has the following atomic types:

- 5.3.1 `integer` - A 64 bit signed integer.
- 5.3.2 `boolean` - Limited to symbols `true` or `false`.
- 5.3.3 `char` - Limited to ASCII values.
- 5.3.4 `string` - ASCII values, null terminated.
- 5.3.5 `void` - Only used for a function that returns no

value. And the following compound types:

- 5.3.6 `array [size] type`
- 5.3.7 `function type ( a: type, b: type, ... )`

And here are the type rules that must be enforced:

- 5.3.8 A value may only be assigned to a variable of the same type.
- 5.3.9 A function parameter may only accept a value of the same type.
- 5.3.10 The type of a `return` statement must match the function return type.
- 5.3.11 All binary operators must have the same type on the left and right hand sides.
- 5.3.12 The equality operators `!=` and `==` may be applied to any type except `void`, `array`, or `function` and always return `boolean`.

- 5.3.13 The comparison operators `< <= >=`
`>` may only be applied to integer values and always return boolean.
- 5.3.14 The boolean operators `! && ||` may only be applied to boolean values and always return boolean.
- 5.3.15 The arithmetic operators `+ - * / % ^ ++ --` may only be applied to integer values and always return integer.

5.4 The Symbol Table

The **symbol table** records all of the information that we need to know about every declared variable (and other named items, like functions) in the program. Each entry in the table is a `struct symbol` which is shown in Figure 7.1.

```
struct symbol {
    symbol_t kind;
    struct type *type;
    char *name;
    int which;
};

typedef enum {
    SYMBOL_LOCAL,
    SYMBOL_PARAM,
    SYMBOL_GLOBAL
} symbol_t;
```

Figure 7.1: The Symbol Structure

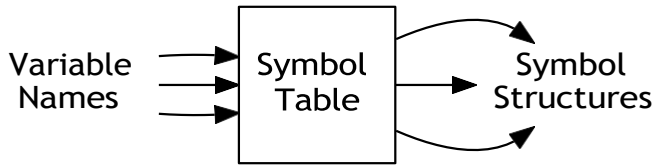
The `kind` field indicates whether the symbol is a local variable, a global variable, or a function parameter. The `type` field points to a type structure indicating the type of the variable. The `name` field gives the name (obviously), and the `which` field gives the ordinal position of local variables and parameters. (More on that later.)

As with all the other data structures we have created so far, we must have a factory function like this:

```
struct symbol * symbol_create( symbol_t kind,
                              struct type *type,
                              char *name ) {
    struct symbol *s = malloc(sizeof(*s));
    s->kind = kind;
    s->type = type;
    s->name = name;
    return s;
}
```

To begin semantic analysis, we must create a suitable `symbol` structure for each variable declaration and enter it into the symbol table.

Conceptually, the symbol table is just a map between the name of each variable, and the symbol structure that describes it:



However, it's not *quite* that simple, because most programming languages allow the same variable name to be used multiple times, as long as each definition is in a distinct **scope**. In C-like languages (including B-Minor) there is a global scope, a scope for function parameters and local variables, and then nested scopes everywhere curly braces appear.

For example, the following B-Minor program defines the symbol `x` three times, each with a different type and storage class. When run, the program should print `10 hello false`.

```
x: integer = 10;

f: function void ( x: string ) =
{
    print x, "\n";
    {
        x: boolean = false;
        print x, "\n";
    }
}

main: function void () =
{
    print x, "\n";
    f("hello");
}
```


To accommodate these multiple definitions, we will structure our symbol table as a stack of hash tables, as shown in Figure 7.2. Each hash table maps the names in a given scope to their corresponding symbols. This allows a symbol (like x) to exist in multiple scopes without conflict. As we proceed through the program, we will push a new table every time a scope is entered, and pop a table every time a scope is left.

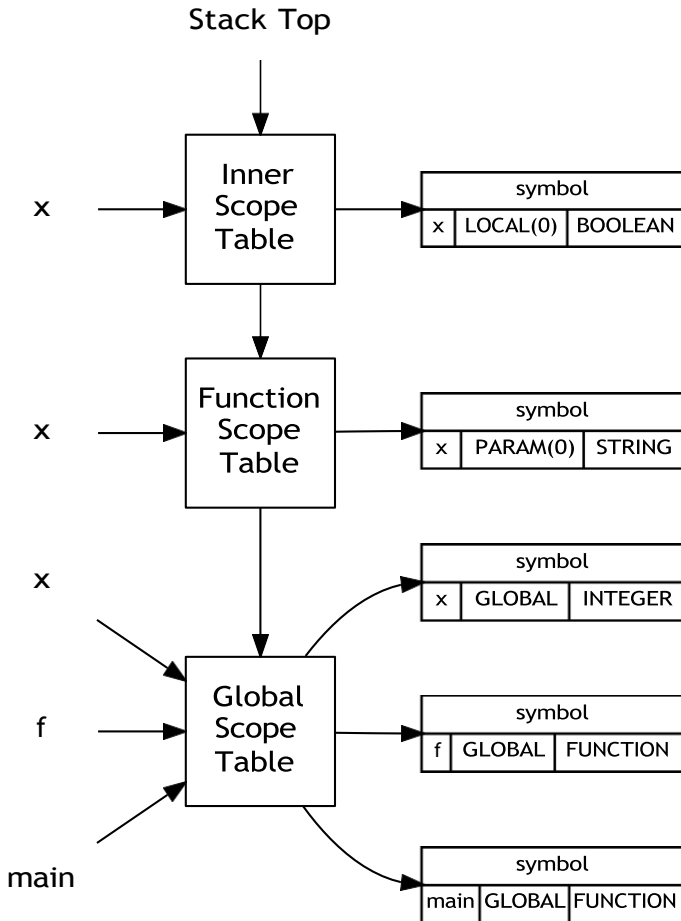


Figure 5.2: A Nested Symbol Table

```
void scope_enter();
void scope_exit();
int  scope_level();

void scope_bind( const char *name, struct symbol *sym );
struct symbol *scope_lookup( const char *name );
struct symbol *scope_lookup_current( const char *name );
```

Figure 7.3: Symbol Table API

To manipulate the symbol table, we define six operations in the API given in Figure 7.3. They have the following meanings:

- `scope_enter()` causes a new hash table to be pushed on the top of the stack, representing a new scope.
- `scope_exit()` causes the topmost hash table to be removed.
- `scope_level()` returns the number of hash tables in the current stack. (This is helpful to know whether we are at the global scope or not.)
- `scope_bind(name, sym)` adds an entry to the topmost hash table of the stack, mapping `name` to the symbol structure `sym`.
- `scope_lookup(name)` searches the stack of hash tables from top to bottom, looking for the first entry that matches `name` exactly. If no match is found, it returns null.
- `scope_lookup_current(name)` works like `scope_lookup` except that it only searches the topmost table. This is used to determine whether a symbol has already been defined in the current scope.

5.5 Name Resolution

With the symbol table in place, we are now ready to match each use of a variable name to its matching definition. This process is known as **name resolution**. To implement name resolution, we will write a `resolve` method for each of the structures in the AST, including `decl_resolve()`, `stmt_resolve()` and so forth.

Collectively, these methods must iterate over the entire AST, looking for variable declarations and uses. Wherever a variable is declared, it must be entered into the symbol table and the `symbol` structure linked into the AST. Wherever a variable is used, it must be looked up in the symbol table, and the `symbol` structure linked into the AST. Of course, if a symbol is declared twice in the same scope, or used without declaration, then an appropriate error message must be emitted.

We will begin with declarations, as shown in Figure 7.4. Each `decl` represents a variable declaration of some kind, so `decl_resolve` will create a new `symbol`, and then bind it to the name of the declaration in the current scope. If the declaration represents an expression (`d->value` is not null) then the expression should be resolved. If the declaration represents a function (`d->code` is not null) then we must create a new scope and resolve the parameters and the code.

Figure 7.4 gives some sample code for resolving declarations. As always in this book, consider this starter code in order to give you the basic idea. You will have to make some changes in order to accommodate all the features of the language, handle errors cleanly, and so forth.

In a similar fashion, we must write `resolve` methods for each structure in the AST. `stmt_resolve()` (not shown) must simply call the appropriate `resolve` on each of its sub-components. In the case of a `STMT_BLOCK`, it must also enter and leave a new scope. `param_list_resolve()` (also not shown) must enter a new variable declaration for each parameter of a function, so that those definitions are available to the code of a function.

To perform name resolution on the entire AST, you may simply invoke `decl_resolve()` once on the root node of the AST. This function will traverse the entire tree by calling the necessary sub-functions.

```

void decl_resolve( struct decl *d )
{
    if(!d) return;

    symbol_t kind = scope_level() > 1 ?
                    SYMBOL_LOCAL : SYMBOL_GLOBAL;

    d->symbol = symbol_create(kind,d->type,d->name);

    expr_resolve(d->value);
    scope_bind(d->name,d->symbol);

    if(d->code) {
        scope_enter();
        param_list_resolve(d->type->params);
        stmt_resolve(d->code);
        scope_exit();
    }

    decl_resolve(d->next);
}

```

Figure 7.4: Name Resolution for Declarations

```

void expr_resolve( struct expr *e )
{
    if(!e) return;

    if( e->kind==EXPR_NAME ) {
        e->symbol = scope_lookup(e->name);
    } else {
        expr_resolve( e->left );
        expr_resolve( e->right );
    }
}

```

Figure 5.5: Name Resolution for Expressions

5.6 Implementing Type Checking

Before checking expressions, we need some helper functions for checking and manipulating type structures. Here is pseudo-code for checking equality, copying, and deleting types:

```
boolean type_equals( struct type *a, struct type *b )
{
    if( a->kind == b->kind ) {
        if( a and b are atomic types ){
            Return true
        } else if ( both are array ) {
            Return true if subtype is recursively equal
        } else if ( both are function ) {
            Return true if both subtype and params
            are recursively equal
        }
    } else {
        Return false
    }
}

struct type * type_copy( struct type *t )
{
    Return a duplicate copy of t, making sure
    to duplicate subtype and params recursively.
}

void type_delete( struct type *t )
{
    Free all the elements of t recursively.
}
```

Next, we construct a function `expr_ttypecheck` that will compute the proper type of an expression, and return it. To simplify our code, we assert that `expr_ttypecheck`, if called on a non-null `expr`, will always return a newly-allocated `type` structure. If the expression contains an invalid combination of types, then `expr_ttypecheck` will print out an error, but return a valid type, so that the compiler can continue on and find as many errors as possible.

The general approach is to perform a recursive, post-order traversal of the expression tree. At the leaves of the tree, the type of the node simply corresponds to the kind of the expression node: an integer literal has integer type, a string literal has string type, and so on. If we encounter a variable name, the type can be determined by following the `symbol` pointer

to the symbol structure, which contains the type. This type is copied and returned to the parent node.

For interior nodes of the expression tree, we must compare the type of the left and right subtrees, and determine if they are compatible with the rules indicated in Section 7.3. If not, we emit an error message and increment a global error counter. Either way, we return the appropriate type for the operator. The types of the left and right branches are no longer needed and can be deleted before returning.

Here is the basic code structure:

```
struct type * expr_typecheck( struct expr *e )
{
    if(!e) return 0;

    struct type *lt = expr_typecheck(e->left);
    struct type *rt = expr_typecheck(e->right);

    struct type *result;

    switch(e->kind) {
        case EXPR_INTEGER_LITERAL:
            result = type_create(TYPE_INTEGER,0,0);
            break;
        case EXPR_STRING_LITERAL:
            result = type_create(TYPE_STRING,0,0);
            break;

        /* more cases here */

    }

    type_delete(lt);
    type_delete(rt);

    return result;
}
```

Let's consider the cases for a few operators in detail. Arithmetic operators can only be applied to integers, and always return an integer type:

```
case EXPR_ADD:
    if( lt->kind!=TYPE_INTEGER ||
        rt->kind!=TYPE_INTEGER ) {
        /* display an error */
    }
    result = type_create(TYPE_INTEGER,0,0);
    break;
```

The equality operators can be applied to most types, as long as the types are equal on both sides. These always return boolean.

```
case EXPR_EQ:
case EXPR_NE:
    if(!type_equals(lt,rt)) {
        /* display an error */
    }
    if(lt->kind==TYPE_VOID ||
        lt->kind==TYPE_ARRAY ||
        lt->kind==TYPE_FUNCTION) {
        /* display an error */
    }
    result = type_create(TYPE_BOOLEAN,0,0);
    break;
```

An array dereference like `a[i]` requires that `a` be an array, `i` be an integer, and returns the subtype of the array:

```
case EXPR_DEREF:
    if(lt->kind==TYPE_ARRAY) {
        if(rt->kind!=TYPE_INTEGER) {
            /* error: index not an integer */
        }
        result = type_copy(lt->subtype);
    } else {
        /* error: not an array */
        /* but we need to return a valid type */
        result = type_copy(lt);
    }
    break;
```

Most of the hard work in typechecking is done in `expr typecheck`, but we still need to implement typechecking on declarations, statements, and the other elements of the AST. `decl typecheck`, `stmt typecheck`

and the other typechecking methods simply traverse the AST, compute the type of expressions, and then check them against declarations and other constraints as needed.

For example, `decl_typecheck` simply confirms that variable declarations match their initializers and otherwise typechecks the body of function declarations:

```
void decl_typecheck( struct decl *d )
{
    if(d->value) {
        struct type *t;
        t = expr_typecheck(d->value);
        if(!type_equals(t,d->symbol->type)) {
            /* display an error */
        }
    }
    if(d->code) {
        stmt_typecheck(d->code);
    }
}
```

Statements must be typechecked by evaluating each of their components, and then verifying that types match where needed. After the type is examined, it is no longer needed and may be deleted. For example, if-then statements require that the control expression have boolean type:

```
void stmt_typecheck( struct stmt *s )
{
    struct type *t;
    switch(s->kind) {
        case STMT_EXPR:
            t = expr_typecheck(s->expr);
            type_delete(t);
            break;
        case STMT_IF_THEN:
            t = expr_typecheck(s->expr);
            if(t->kind!=TYPE_BOOLEAN) {
                /* display an error */
            }
            type_delete(t);
            stmt_typecheck(s->body);
            stmt_typecheck(s->else_body);
            break;
        /* more cases here */
    }
}
```


5.7 Error Messages

Compilers in general are notorious for displaying terrible error messages. Fortunately, we have developed enough code structure that it is straightforward to display an informative error message that explains exactly what types were discovered, and what the problem is.

For example, this bit of B-Minor code has a mess of type problems:

```
s: string = "hello";
b: boolean = false;
i: integer = s + (b<5);
```

Most compilers would emit an unhelpful message like this:

```
error: type compatibility in expression
```

But, your project compiler can very easily have much more detailed error messages like this:

```
error: cannot compare a boolean (b) to an integer (5)
error: cannot add a boolean (b<5) to a string (s)
```

It's just a matter of taking some care in printing out each of the expressions and types involved when a problem is found:

```
printf("error: cannot add a ");
type_print(lt);
printf(" ");
expr_print(e->left);
printf(") to a ");
type_print(rt);
printf(" ");
expr_print(e->right);
printf("\n");
```

Chapter 6 – Memory Organization

6.1 Introduction

Before digging into the translation of intermediate code to assembly language, we must discuss how the internal memory of a running program is laid out. Although a process is free to use memory in any way that it likes, a convention has developed that divides the areas of a program into logical segments, each with a different internal management strategy.

6.2 Logical Segmentation

A conventional program sees memory as a linear sequence of words, each with a numeric address starting at zero, and increasing up to some large number (e.g. 4GB on a 32-bit processor.)

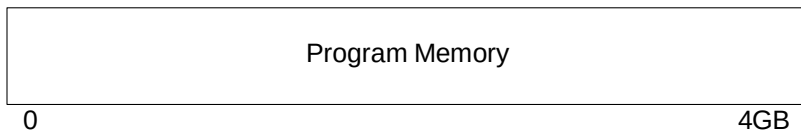


Figure 6.1: Flat Memory Model

In principle, the CPU is free to use memory in any way it sees fit. Code and data could be scattered and intermixed across memory in any order that is convenient. It is even technically possible for a CPU to modify the memory containing its code while it is running. It goes without saying that programming in this fashion would be complex, confusing, and difficult to debug.

Instead, program memory is commonly laid out by separating it into **logical segments**. Each segment is a sequential address range, dedicated to a particular purpose within the program. The segments are typically

laid out in this order:

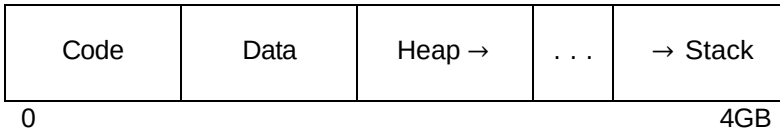


Figure 9.2: Logical Segments

- The **code segment** (also known as the **text segment**) contains the machine code of the program, corresponding to the bodies of functions in a C program.
- The **data segment** contains the global data of the program, corresponding to the global variables in a C program. The data segment may further be sub-divided into read-write data (variables) and read-only data (constants).
- The **heap segment** contains the heap, which is the area of memory that is managed dynamically at runtime by `malloc` and `free` in a C program, or `new` and `delete` in other languages. The top of the heap is historically known as the **break**.
- The **stack segment** contains the stack, which records the current execution state of the program as well as the local variables currently in use.

Typically, the heap grows “up” from lower addresses to higher addresses, while the stack grows “down” from higher to lower. In between the two segments is an invalid region of memory that is unused until overtaken by one segment or the other.

On a simple computer such as an embedded machine or microcontroller, logical segments are nothing more than an organizational convention: nothing stops the program from using memory improperly. If the heap grows too large, it can run into the stack segment (or vice versa), the program will crash (if you are lucky) or suffer silent data corruption (if you are unlucky).

On a computer with an operating system that employs multiprogramming and memory protection, the situation is better. Each process running in the OS has its own private memory space, with the illusion of starting at address zero and extending to a high address. As a result, each process can access its own memory arbitrarily but is prevented from accessing or modifying other processes. Within its own space, each process lays out its own code, data, heap, and stack segments.

In some operating systems, when a program is initially loaded into memory, permissions are set on each range of memory corresponding to its purpose: memory corresponding to each segment can be set appropri-

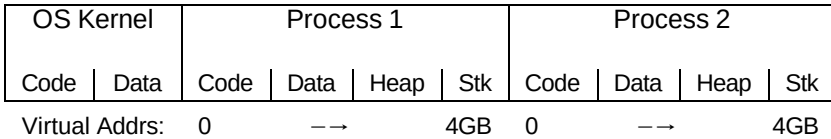


Figure 9.3: Multiprogrammed Memory Layout

ately: read-write for data, heap, and stack; read-only for constants; read-execute for code; and none for the unused area.

The permissions on the logical segments also protect a process from damaging itself in certain ways. For example, code cannot be modified at runtime because it is marked read/execute, while items on the heap cannot be executed, because they are marked read/write. (To be clear, this only prevents against accidents, not malice; a program can always ask the operating system to change the permissions on one of its own pages. For an example, look up the `mprotect` system call in Unix.)

If a process attempts to access memory in a prohibited way or attempts to access the unused area, a **page fault** occurs. This forces a transfer of control to the OS, which considers the process and the faulting address. If the access indicates a violation of the logical segmentation of the program, then the process is forcibly killed with the error message **segmentation fault**.¹

Initially, a process is given a small amount of memory for the heap segment, which it manages internally to implement `malloc` and `free`. If this area is exhausted and the program still needs more, it must explicitly request it from the operating system. On traditional Unix systems, this is done with the `brk` system call, which requests that the heap segment be extended to a new address. If the OS agrees, then it will allocate new pages at the beginning of the invalid area, effectively extending the heap segment. If the OS does not agree, then `brk` will return an error code, causing `malloc` to return an error (indicated as a null pointer) which the program must handle.

The stack has a similar problem, in that it must be able to grow down. It is not so easy for a program to determine exactly when more stack is needed, because that happens whenever a new function is invoked, or new local variables are allocated. Instead, modern operating systems maintain a **guard page** at the top of the invalid area, adjacent to the current stack. When a process attempts to extend the stack into the invalid area, a page fault occurs, and control is transferred to the OS. If the OS sees that the faulting address is in the guard page, it can simply allocate more pages for the stack, set the page permissions appropriately, and move the guard page down to the new top of the invalid area.

Of course, there are limits on how big the heap and the stack may grow;

¹And now you understand this mysterious Unix term!

every OS implements policies controlling how much memory any process or user may consume. If any of these policies are violated, the OS may decline to extend the memory of the process.

The idea of breaking a program into segments is so powerful and useful that it was common for many decades to have the concept implemented in hardware. (If you have taken a class in computer architecture and operating systems, you have probably studied this in some detail.) The basic idea is that the CPU maintains a table of segments, recording the starting address and length, along with the permissions associated with each segment. The operating system would typically set up a hardware segment to correspond to the logical organization just described.

Although hardware segmentation was widely used in operating systems through the 1980s, it has been largely replaced by paging, which was seen as simpler and more flexible. Processor vendors have responded by removing support for hardware segmentation in new designs. For example, every generation of the Intel X86 architecture from the 8086 up to the Pentium supported segmentation in 32-bit protected mode. The latest 64-bit architectures provide only paging facilities, and no segmentation. Logical segmentation continues as a useful way to organize programs in memory.

Let's continue by looking at each of the logical segments in more detail.

9.1 Heap Management

The heap contains memory that is managed dynamically at runtime. The OS does not control the internal organization of the heap, except to limit its total size. Instead, the internal structure of the heap is managed by the standard library or other runtime support software that is automatically linked into a program. In a C program, the functions **malloc** and **free** allocate and release memory on the heap, respectively. In C++, **new** and **delete** have the same effect. Other languages manipulate the heap implicitly when objects and arrays are created and deleted.

The simplest implementation of **malloc** and **free** is to treat the entire heap as one large linked list of memory regions. Each entry in the list records the state of the region (free or in use), the size of the region, and has pointers to the previous and next regions. Here's what that might look like in C:

```
struct chunk {
    enum { FREE, USED } state;
    int size;
    struct chunk *next;
    struct chunk *prev;
    char data[0];
};
```

(Note that we declared `data` as an array of length zero. This is a little trick that allows us to treat `data` as a variable length array, provided that the underlying memory is actually present.)

Under this scheme, the initial state of the heap is simply one entry in a linked list:

FREE	1000	data
prev	next	

Suppose that the user calls `malloc(100)` to allocate 100 bytes of memory. `malloc` will see that the (single) chunk of memory is free, but much larger than the requested size. So, it will split it into one small chunk of 100 bytes and one larger chunk with the remainder. This is accomplished by simply writing a new chunk header into the data area after 100 bytes. Then, connect them together into a linked list:

USED	100	data	FREE	900	data
prev	next		prev	next	

Once the list has been modified, `malloc` returns the address of the data element within the chunk, so that the user can access it directly. It doesn't return the linked list node itself, because the user doesn't need to know about the implementation details. If there is no chunk available that is large enough to satisfy the current request, then the process must ask the OS to extend the heap by calling `brk`.

When the user calls `free` on a chunk of memory, the state of the chunk in the linked list is marked `FREE`, and then merged with adjacent nodes, if they are also free.

(Incidentally, now you can see why it is dangerous for a program to modify memory carelessly outside a given memory chunk. Not only could it affect other chunks, but it could damage the linked list itself, resulting in wild behavior on the next `malloc` or `free`!)

If the program always frees memory in the opposite order that it was allocated, then the heap will be nicely split into allocated and free memory. However, that isn't what happens in practice: memory can be allocated and freed in any order. Over time, the heap can degenerate into a mix of oddly sized chunks of allocated and freed memory. This is known as **memory fragmentation**.

Excessive fragmentation can result in waste: if there are many small chunks available, but none of them large enough to satisfy the current `malloc`, then the process has no choice but to extend the heap, leaving the small pieces unused. This increases pressure on total virtual memory consumption in the operating system.

In a language like C, memory chunks cannot be moved while in use, and so fragmentation cannot be fixed after it has already occurred. How-

ever, the memory allocator has some limited ability to avoid fragmentation by choosing the location of new allocations with care. Some simple strategies are easy to imagine and have been studied extensively:

- **Best Fit.** On each allocation, search the entire linked list and find the *smallest* free chunk that is larger than the request. This tends to leave large spaces available but generates tiny leftover free fragments that are too small to be used.
- **Worst Fit.** On each allocation, search the entire linked list and find the *largest* free chunk that is larger than the request. Somewhat counterintuitively, this method tends to reduce fragmentation by avoiding the creation of tiny unusable fragments.
- **First Fit.** On each allocation, search the linked list from the beginning, and find the *first* fragment (large or small) that satisfies the request. This performs less work than Best Fit or Worst Fit, but performs an increasing amount of work as the linked list increases in size.
- **Next Fit.** On each allocation, search the linked list from the last examined location, and find the *next* fragment (large or small) that satisfies the request. This minimizes the amount of work done on each allocation, while distributing allocations throughout the heap.

For general purpose allocators where one cannot make assumptions about application behavior, the conventional wisdom is that Next Fit results in good performance with an acceptable level of fragmentation.

9.2 Stack Management

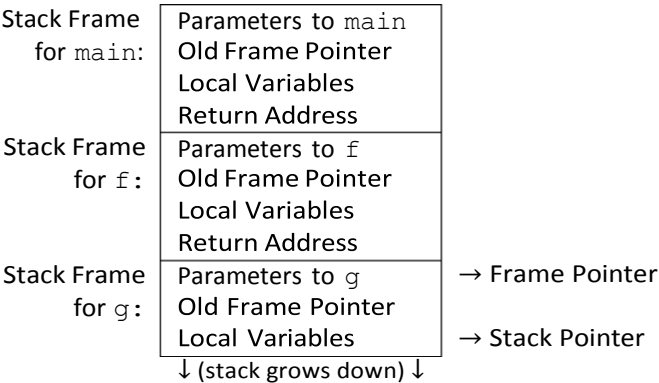
The **stack** is used to record the current state of the running program. Most CPUs have a specialized register – the **stack pointer** – which stores the address where the next item will be pushed or popped. Because the stack grows down from the top of memory, there is a confusing convention: pushing an item on the stack causes the stack pointer to move to a lower numbered address, while popping an item off the stack causes the stack pointer to move to a higher address. The “top” of the stack is actually at the lowest address!

Each invocation of a function occupies a range of memory in the stack, known as a **stack frame**. The stack frame contains the parameters and the local variables used by that function. When a function is called, a new stack frame is pushed; when the function returns, the stack frame is popped, and execution continues in the caller’s stack frame.

Another specialized register known as the **frame pointer** (or sometimes **base pointer**) indicates the beginning of the current frame. Code

within a function relies upon the frame pointer to identify the location of the current parameters and local variables.

For example, suppose that the `main` function calls function `f`, and then `f` calls `g`. If we stop the program in the middle of executing `g`, the stack would look like this:



The order and details of the elements in a stack frame differ somewhat between CPU architectures and operating systems. As long as both the caller and the callee agree on what goes in the stack frame, then any function may call another, even if they were written in different languages, or built by different compilers.

The agreement on the contents of the activation record is known as a **calling convention**. This is typically written out in a detailed technical document that is used by the designers of compilers, operating systems, and libraries to ensure that code is mutually interoperable.

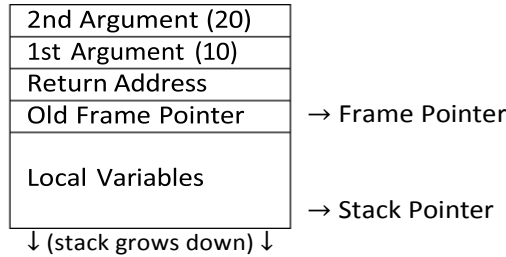
There are two broad categories of calling conventions, with many opportunities for variation in between. One is to put the arguments to a function call on the stack, and the other is to place them in registers.

9.2.1 Stack Calling Convention

The conventional approach to calling a function is to push the arguments to that function on the stack (in reverse order), and then to jump to the address of the function, leaving behind a return address on the stack. Most CPUs have a specialized `CALL` instruction for this purpose. For example, the assembly code to call `f(10, 20)` could be as simple as this:

```
PUSH $20
PUSH $10
CALL f
```

When `f` begins executing, it saves the old frame pointer currently in effect and makes space for its own local variables. As a result, the stack frame for `f(10, 20)` looks like this:



To access its arguments or local variables, `f` must load them from memory relative to the frame pointer. As you can see, the function arguments are found at fixed positions *above* the frame pointer, while local variables are found *below* the frame pointer.²

9.2.2 Register Calling Convention

An alternate approach to calling a function is to put the arguments into registers, and then call the function. For example, let us suppose that our calling convention indicates that registers `%R10`, `%R11`, etc are to be used for arguments. Under this calling convention, the assembly code to invoke `f(10, 20)` might look like this:

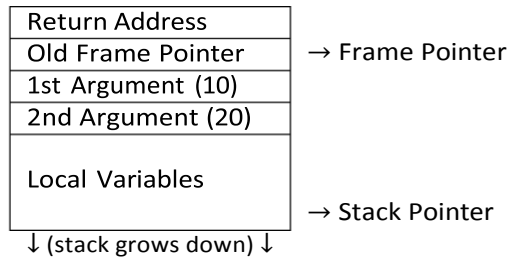
```
MOVE $10 -> %R10
MOVE $20 -> %R11
CALL f
```

When `f` begins executing, it still must save the old frame pointer and make room for local variables. It doesn't have to load arguments from the stack; it simply expects the values in `%R10` and `%R11` and can compute on them right away. This could confer a significant speed advantage by avoiding memory accesses.

But, what if `f` is a complex function that needs to invoke other functions? It will still need to save the current values of the argument registers, in order to free them up for its own use.

To allow for this possibility, the stack frame for `f` must leave space for the arguments, in case they must be saved. The calling convention must define the location of the arguments, and they are typically stored *below* the return address and old frame pointer, like this:

²The arguments are pushed in reverse order in order to allow the possibility of a variable number of arguments. Argument 1 is always two words above the frame pointer, argument 2 is always three words above, and so on.



What happens if the function has more arguments than there are registers set aside for arguments? In this case, the additional arguments are pushed on to the stack, as in the stack calling convention.

In the big picture, the choice between stack and register calling conventions doesn't matter much, except that all parties must agree on the details of the convention. The register calling convention has the slight advantage that a **leaf function** (a function that does not call other functions) can compute a simple result without accessing memory. Typically, the register calling convention is used on architectures that have a large number of registers that might otherwise go unused.

It is possible to mix the conventions in a single program, as long as both caller and callee are informed of the distinction. For example, the Microsoft X86 compilers allow keywords in function prototypes to select a convention: `cdecl` selects the stack calling convention, while `fastcall` uses registers for the first two arguments.

9.3 Locating Data

For each kind of data in a program, there must be an unambiguous method of locating that data in memory. The compiler must generate an **address computation** using the basic information available about the symbol. The computation varies with the storage class of the data:

- **Global data** has the easiest address computation. In fact, the compiler doesn't usually compute global addresses, but rather passes the *name* of each global symbol to the assembler, which then selects the address computation. In the simplest case, the assembler will generate an **absolute address** giving the exact location of the data in program memory.

However, the simple approach isn't necessarily efficient, because an absolute address is a full word (e.g. 64 bits), the same size as an instruction in memory. This means that the assembler must use several instructions (RISC) or multi-word instructions (CISC) to load the address into a register. Assuming that most programs don't use the entire address space, it isn't usually necessary to use the entire word.

An alternative is to use a **base-relative address** that consists of a base address given by a register plus a fixed offset given by the assembler.

For example, global data addresses could be given by a register indicating the beginning of the data segment, plus a fixed offset, while a function address could be given by a register indicating the beginning of the code segment plus a fixed offset. Such an approach can be used in dynamically loaded libraries, when the location of the library is not fixed in advance, but the location of a function within the library is known.

Yet another approach is to use a **PC-relative address** in which the exact distance in bytes between the referring instruction and the target data is computed, and then encoded into the instruction. This works as long as the relative distance is small enough (e.g. 16 bits) to fit into the address field of the instruction. This task is performed by the assembler and is usually invisible to the programmer.

- **Local data** works differently. Because local variables are stored on the stack, a given local variable does not necessarily occupy the same absolute address each time it is used. If a function is called recursively, there may be multiple instances of a given local variable in use simultaneously! For this reason, local variables are always specified as an offset relative to the current frame pointer. (The offset may be positive or negative, depending on the function calling convention.) Function parameters are just a special case of local variables: a parameter's position on the stack is given precisely by its ordinal position in the parameters.
- **Heap data** can only be accessed by way of pointers that are stored as global or local variables. To access data on the heap, the compiler must generate an address computation for the pointer itself, then dereference the pointer to reach the item on the heap.

So far, we have only considered atomic data types that are easily stored in a single word of memory: booleans, integers, floats, and so forth. However, any of the more complex data types can be placed in any of the three storage classes, and require some additional handling.

An array can be stored in global, local, or heap memory, and the beginning of the array is found by one of the methods above. An element in the array is found by multiplying the index by the size of the items in the array, and adding that to the address of the array itself:

```
address(a[i]) = address(a) + sizeof(type) * i
```

The more interesting question is how to deal with the length of the array itself. In an unsafe language like C, the simple approach is to simply do nothing: if the program happens to run off the end of the array, the compiler will happily compute an address outside the array bounds, and chaos results. For some applications where performance is paramount, the simplicity of this approach trumps any increase in safety.

A safer approach is to store the length of the array at the base address of the array itself. Then, the compiler may generate code that checks the actual index against the array bounds before generating the address. This prevents any sort of runtime accident by the programmer. However, the downside is performance. Every time the programmer mentions `a[i]`, the resulting code must contain this:

1. Compute address of array `a`.
2. Load length of `a` into a register.
3. Compare array index `i` to register.
4. If `i` is outside of array bounds, raise an exception.
5. Otherwise, compute address of `a[i]` and continue.

This pattern is so common that some computer architectures provide dedicated support for array bounds checking. The Intel X86 architecture, (which we will examine in detail in the next chapter) provides a unique `BOUND` instruction, whose only purpose is to compare a value against two array bound values, and then raise a unique “Array Bounds Exception” if it falls outside.

Structures have similar considerations. In memory, a structure is very much like an array, except that it can contain items of irregular size. To access an item within a structure, the compiler must generate an address computation of the beginning of the structure, and then add an offset corresponding to the name of item (known as the **structure tag**) within the structure. Of course, it is not necessary to perform bounds checking since the offsets are fixed at compile time.

For complex nested data structures, the address computation necessary to find an individual element can become quite complicated. For example, consider this bit of code to represent a deck of cards:

```
struct card {
    int suit;
    int rank;
};

struct deck {
    int is_shuffled;
    struct card cards[52];
};

struct deck d;

d.cards[10].rank = 10;
```

To compute `d.cards[10].rank`, the compiler must first generate an address computation for `d`, depending on whether it is a local or global variable. From there, the offset of `cards` is added, then the offset of the tenth item, then the offset of `rank` within the card. The complete address computation is:

```
address(d.card[10].rank) =
    address(d)
    + offset(cards)
    + sizeof(struct card)*10
    + offset(rank)
```

9.4 Program Loading

Before a program begins executing in memory, it first exists as a file on disk, and there must be a convention for loading it into memory. There are a variety of **executable formats** for organizing a program on disk, ranging from very simple to very complex. Here are a few examples to give you the idea.

The simplest computer systems simply store an executable as a **binary blob** on disk. The program code, data, and initial state of the heap and stack are simply dumped into one file without distinction. To run the program, the OS must simply load the contents of the file into memory, and then jump to the first location of the program to begin execution.

This approach is about as simple as one can imagine. It does work, but it has several limitations. One is that the format wastes space on uninitialized data. For example, if the program declares a large global array where each element has the value zero, then every single zero in that array will be stored in the file. Another is that the OS has no insight into how the program intends to use memory, so it is unable to set permissions on each logical segment, as discussed above. Yet another is that the binary blob has no identifying information to show that it is an executable.

However, the binary blob approach is still occasionally used in places where programs are small and simplicity is paramount. For example, the very first boot stage of a PC operating system reads in a single sector from the boot hard disk containing a binary blob, which then carries out the second stage of booting. Embedded systems often have very small programs measured in a few kilobytes, and rely on binary blobs.

An improved approach used in classic Unix systems for many years is the **a.out** executable format.³ There are many slight variations on the format, but they all share the same basic structure. The executable file consists of a short header structure, followed by the text, initialized data, and symbol table:

³The first Unix assembler sent its output to a file named `a.out` by default. In the absence of any other name for the format, the name stuck.

Header	Text	Data	Symbols
--------	------	------	---------

The header structure itself is just a few bytes that allow the operating system to interpret the rest of the file:

Magic Number
Text Section Size
Data Section Size
BSS Size
Symbol Table Size
Entry Point

The **magic number** is a unique integer that clearly defines the file as an executable: if the file does not begin with this magic number, the OS will not even attempt to execute it. Different magic numbers are defined for executables, unlinked object files, and shared libraries. The **text size** field indicates the number of bytes in the text section that follows the header. The **data size** field indicates the amount of *initialized* data that appears in the file, while the **BSS size** field indicates the amount of *uninitialized data*.

⁴

The uninitialized data need not be stored in the file. Instead it is simply allocated in memory as part of the data segment when the program is loaded. The **symbol table** in the executable lists each of the variable and function names used in the program along with their locations in the code and data segment; this permits a debugger to interpret the meaning of addresses. Finally, the **entry point** gives the address of the starting point of the program (typically `main`) in the text segment. This allows the starting point to be something other than the first address in the program.

The `a.out` format is a big improvement over a binary blob, and is still supported and usable today in many operating systems. However, it isn't quite powerful enough to support many of the features needed by modern languages, particularly dynamically loaded libraries.

The **Extensible Linking Format (ELF)** is widely used today across many operating systems to represent executables, object files, and shared libraries. Like `a.out`, an ELF file has multiple sections representing code, data, and `bss`, but it can have an arbitrary number of additional sections for debugging data, initialization and finalization code, metadata about the tools used, and so forth. The number of *sections* in the file outnumbers the *segments* in memory, and so a **section table** in the ELF file indicates how multiple sections are to be mapped into a single segment.

⁴BSS stands for "Block Started by Symbol" and first appeared in an assembler for IBM 704 in the 1950s.

File Header
Program Header
Code Section
Data Section
Read-Only Section
...
Section Header

9.5 Further Reading

1. Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau, “Operating Systems: Three Easy Pieces”, Arpaci-Dusseau Books, 2015.
<http://www.ostep.org>
Operating systems is usually the course in which memory management is covered in great detail. If you need a refresher on memory allocators (or anything else in operating systems), check out this online textbook.
2. John R. Levine, “Linkers and Loaders”, Morgan Kaufmann, 1999.
This book provides a detailed look at linkers and loaders, which is an often-overlooked topic that falls in cracks between compilers and operating systems. A solid understanding of linking is necessarily to create and use libraries effectively.
3. Paul R. Wilson, “Uniprocessor Garbage Collection Techniques”, Lecture Notes in Computer Science, volume 637, 1992.
<https://link.springer.com/chapter/10.1007/BFb0017182>
This widely-read article gives an accessible overview of the key techniques of garbage collection, which is an essential component of the runtime of modern dynamic languages.

Chapter 7 – Assembly Language

7.1 Introduction

In order to build a compiler, you must have a working knowledge of at least one kind of assembly language. And, it helps to see two or more variations of assembly, so as to fully appreciate the distinctions between architectures. Some of these differences, such as register structure, are quite fundamental, while some of the differences are merely superficial.

We have observed that many students seem to think that assembly language is rather obscure and complicated. Well, it is true that the complete manual for a CPU is extraordinarily thick, and may document hundreds of instructions and obscure addressing modes. However, it's been our experience that it is really only necessary to learn a small subset of a given assembly language (perhaps 30 instructions) in order to write a functional compiler. Many of the additional instructions and features exist to handle special cases for operating systems, floating point math, and multi-media computing. You can do almost everything needed with the basic subset.

We will look at two different CPU architectures that are in wide use today: X86 and ARM. The Intel X86 is a CISC architecture that has evolved since the 1970s from 8-bit to 64-bit and is now the dominant chip in personal computers, laptops, and high performance servers. The ARM processor is a RISC architecture that began life as a 32-bit chip for the personal computer market, and is now the dominant chip for low-power and embedded devices such as mobile phones and tablets.

This chapter will give you a working knowledge of the basics of each architecture, but you will need a good reference to look up more details. We recommend that you consult the Intel Software Developer Manual [1] and the ARM Architecture Reference Manual [3] for the complete details. (Note that each section is meant to be parallel and self-contained, so some explanatory material is repeated for both X86 and ARM.)

7.2 Open Source Assembler Tools

A given assembly language can have multiple dialects for the same CPU, depending on whether one uses the assembler provided by the chip vendor, or other open source tools. For consistency, we will give examples in the assembly dialect supported by the GNU compiler and assembler, which are known as `gcc` and `as` (or sometimes `gas`).

A good way to get started is to view the assembler output of the compiler for a C program. To do this, run `gcc` with the `-S` flag, and the compiler will produce assembly output rather than a binary program. On Unix-like systems, assembly code is stored in files ending with `.s`, which indicates “source” file.

If you run `gcc -S hello.c -o hello.s` on this C program:

```
#include <stdio.h>

int main( int argc, char *argv[] )
{
    printf("hello %s\n", "world");
    return 0;
}
```

then you should see output similar to this in `hello.s`

```
.file    "test.c"
.data
.LC0:
    .string "hello %s\n"
.LC1:
    .string "world"
.text
.global main
main:
    PUSHQ    %rbp
    MOVQ     %rsp, %rbp
    SUBQ     $16, %rsp
    MOVQ     %rdi, -8(%rbp)
    MOVQ     %rsi, -16(%rbp)
    MOVQ     $.LC0, %rax
    MOVQ     $.LC1, %rsi
    MOVQ     %rax, %rdi
    MOVQ     $0, %rax
    CALL     printf
    MOVQ     $0, %rax
    LEAVE
    RET
```

(There are many valid ways to compile `hello.c` and so the output of your compiler may be somewhat different.)

Regardless of the CPU architecture, the assembly code has three different kinds of elements:

Directives begin with a dot and indicate structural information useful to the assembler, linker, or debugger, but are not in and of themselves assembly instructions. For example, `.file` simply records the name of the original source file to assist the debugger. `.data` indicates the start of the data segment of the program, while `.text` indicates the start of the program segment. `.string` indicates a string constant within the data section, and `.global main` indicates that the label `main` is a global symbol that can be accessed by other code modules.

Labels end with a colon and indicate by their position the association between names and locations. For example, the label `.LC0:` indicates that the immediately following string should be called `.LC0`. The label `main:` indicates that the instruction `PUSHQ %rbp` is the first instruction of the main function. By convention, labels beginning with a dot are temporary local labels generated by the compiler, while other symbols are user-visible functions and global variables. The labels do not become part of the resulting machine code *per se*, but they are present in the resulting object code for the purposes of linking, and in the eventual executable, for purposes of debugging.

Instructions are the actual assembly code like `(PUSHQ %rbp)`, typically indented to visually distinguish them from directives and labels. Instructions in GNU assembly are not case sensitive, but we will generally upper-case them, for consistency.

To take this `hello.s` and turn it into a runnable program, just run `gcc`, which will figure out that it is an assembly program, assemble it, and link it with the standard library:

```
% gcc hello.s -o hello
% ./hello
hello world
```

It is also interesting to compile the assembly code into object code, and then use the `nm` utility to display the symbols ("names") present in the code:

```
% gcc hello.s -c -o hello.o
% nm hello.o
0000000000000000 T main
                 U printf
```

This displays the information available to the linker. `main` is present in the text (T) section of the object, at location zero, and `printf` is undefined

(U), since it must be obtained from the standard library. But none of the labels like `.LC0` appear because they were not declared as `.global`.

As you are learning assembly language, take advantage of an existing compiler: write some simple functions to see what `gcc` generates. This can give you some starting points to identify new instructions and techniques to use.

7.3 X86 Assembly Language

X86 is a generic term that refers to the series of microprocessors descended from (or compatible with) the Intel 8088 processor used in the original IBM PC, including the 8086, 80286, '386, '486, and many others. Each generation of CPUs added new instructions and addressing modes from 8-bit to 16-bit to 32-bit, all while retaining backwards compatibility with old code. A variety of competitors (such as AMD) produced compatible chips that implemented the same instruction set.

However, Intel broke with tradition in the 64-bit generation by introducing a new brand (Itanium) and architecture (IA64) that was *not* backwards compatible with old code. Instead, it implemented a new concept known as Very Long Instruction Word (VLIW) in which multiple concurrent operations were encoded into a single word. This had the potential for significant speedups due to instruction-level parallelism but represented a break with the past.

AMD stuck with the old ways and produced a 64-bit architecture (AMD64) that *was* backwards compatible with both Intel and AMD chips. While the technical merits of both approaches were debatable, the AMD approach won in the marketplace, and Intel followed by producing its own 64-bit architecture (Intel64) that was compatible with AMD64 and its own previous generation of chips. X86-64 is the generic name that covers both AMD64 and Intel64 architectures.

X86-64 is a fine example of CISC (complex instruction set computing). There are a very large number of instructions with many different sub-modes, some of them designed for very narrow tasks. However, a small subset of instructions will let us accomplish a lot.

- **Registers and Data Types**

X86-64 has sixteen (almost) general purpose 64-bit integer registers:

<code>%rax</code>	<code>%rbx</code>	<code>%rcx</code>	<code>%rdx</code>	<code>%rsi</code>	<code>%rdi</code>	<code>%rbp</code>	<code>%rsp</code>
<code>%r8</code>	<code>%r9</code>	<code>%r10</code>	<code>%r11</code>	<code>%r12</code>	<code>%r13</code>	<code>%r14</code>	<code>%r15</code>

These registers are *almost* general purpose because earlier versions of the processors intended for each register to be used for a specific purpose, and not all instructions could be applied to every register. The names of

A Note on AT&T Syntax versus Intel Syntax

Note that the GNU tools use the traditional AT&T syntax, which is used across many processors on Unix-like operating systems, as opposed to the Intel syntax typically used on DOS and Windows systems. The following instruction is given in AT&T syntax:

```
MOVQ %RSP, %RBP
```

`MOVQ` is the name of the instruction, and the percent signs indicate that `RSP` and `RBP` are registers. In the AT&T syntax, the source is always given first, and the destination is always given second.

In other places (such as the Intel manual), you will see the Intel syntax, which (among other things) dispenses with the percent signs and *reverses* the order of the arguments. For example, this is the same instruction in the Intel syntax:

```
MOVQ RBP, RSP
```

When reading manuals and web pages, be careful to determine whether you are looking at AT&T or Intel syntax: look for the percent signs!

the lower eight registers indicate the purpose for which each was originally intended: for example, `%rax` is the accumulator.

As the design developed, new instructions and addressing modes were added to make the various registers almost equal. A few remaining instructions, particularly related to string processing, require the use of `%rsi` and `%rdi`. In addition, two registers are reserved for use as the stack pointer (`%rsp`) and the base pointer (`%rbp`). The final eight registers are numbered and have no specific restrictions.

The architecture has expanded from 8 to 64 bits over the years, and so each register has some internal structure. The lowest 8 bits of the `%rax` register are an 8-bit register `%al`, and the next 8 bits are known as `%ah`. The low 16 bits are collectively known as `%ax`, the low 32-bits as `%eax`, and the whole 64 bits as `%rax`.

Figure 10.1: X86 Register Structure



The numbered registers %r8-%r15 have the same structure, but a slightly different naming scheme:

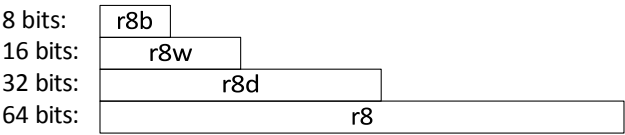


Figure 10.2: X86 Register Structure

To keep things simple, we will focus our attention on the 64-bit registers. However, most production compilers use a mix of modes: a byte can represent a boolean; a longword is usually sufficient for integer arithmetic, since most programs don’t need integer values above 2^{32} ; and a quadword is needed to represent a memory address, enabling up to 16EB (exa-bytes) of virtual memory.

• **Addressing Modes**

The `MOV` instruction moves data between registers and to and from memory in a variety of different modes. A single letter suffix determines the size of data to be moved:

Suffix	Name	Size
B	BYTE	1 byte (8 bits)
W	WORD	2 bytes (16 bits)
L	LONG	4 bytes (32 bits)
Q	QUADWORD	8 bytes (64 bits)

`MOVB` moves a byte, `MOVW` moves a word, `MOVL` moves a long, `MOVQ` moves a quad-word.¹ Generally, the size of the locations you are moving to and from must match the suffix. In some cases, you can leave off the suffix, and the assembler will infer the right size. However, this can have unexpected consequences, so we will make a habit of using the suffix.

The arguments to `MOV` can have one of several addressing modes.

- A **global value** is simply referred to by an unadorned name such as `x` or `printf`, which the assembler translates into an absolute address or an address computation.
- An **immediate value** is a constant value indicated by a dollar sign such as `$56`, and has a limited range, depending on the instruction in use.
- A **register value** is the name of a register such as `%rbx`.

¹Careful: These terms are not portable. A *word* has a different size on different machines.

- An **indirect value** refers to a value by the address contained in a register. For example, `(%rsp)` refers to the value pointed to by `%rsp`.
- A **base-relative** value is given by adding a constant to the name of a register. For example, `-16(%rcx)` refers to the value at the memory location sixteen bytes below the address indicated by `%rcx`. This mode is important for manipulating stacks, local values, and function parameters, where the start of an object is given by a register.
- A **complex** address is of the form $D(R_A, R_B, C)$ which refers to the value at address $R_A + R_B * C + D$. Both R_A and R_B are general purpose registers, while C can have the value 1, 2, 4, or 8, and D can be any integer displacement. This mode is used to select an item within an array, where R_A gives the base of the array, R_B gives the index into the array, C gives the size of the items in the array, and D is an offset relative to that item.

Here is an example of using each kind of addressing mode to load a 64-bit value into `%rax`:

Mode	Example
Global Symbol	<code>MOVQ x, %rax</code>
Immediate	<code>MOVQ \$56, %rax</code>
Register	<code>MOVQ %rbx, %rax</code>
Indirect	<code>MOVQ (%rsp), %rax</code>
Base-Relative	<code>MOVQ -8(%rbp), %rax</code>
Complex	<code>MOVQ -16(%rbx,%rcx,8), %rax</code>

For the most part, the same addressing modes may be used to store data into registers and memory locations. There are some exceptions. For example, it is not possible to use base-relative for both arguments of `MOV`: `MOVQ -8(%rbx), -8(%rbx)`. To see exactly what combinations of addressing modes are supported, you must read the manual pages for the instruction in question.

In some cases, you may want to load the *address* of a variable instead of its value. This is handy when working with strings or arrays. For this purpose, use the `LEA` (load effective address) instruction, which can perform the same address computations as `MOV`:

Mode	Example
Global Symbol	<code>LEAQ x, %rax</code>
Base-Relative	<code>LEAQ -8(%rbp), %rax</code>
Complex	<code>LEAQ -16(%rbx,%rcx,8), %rax</code>

- **Basic Arithmetic**

You will need four basic arithmetic instructions for your compiler: integer addition, subtraction, multiplication, and division.

ADD and SUB have two operands: a source and a destructive target. For example, this instruction:

```
ADDQ %rbx, %rax
```

adds %rbx to %rax, and places the result in %rax, overwriting what might have been there before. This requires a little care, so that you don't accidentally clobber a value that you might want to use later. For example, you could translate $c = a+b+b$; like this:

```
MOVQ a, %rax
MOVQ b, %rbx
ADDQ %rbx, %rax
ADDQ %rbx, %rax
MOVQ %rax, c
```

The IMUL instruction is a little unusual, because multiplying two 64-bit integers results in a 128-bit integer, in the general case. IMUL takes its argument, multiplies it by the contents of %rax, and then places the low 64 bits of the result in %rax and the high 64 bits in %rdx. (This is implicit: %rdx is not mentioned in the instruction.)

For example, suppose that you wish to translate $c = b * (b+a)$; , where a, and b, and c are global integers. Here is one possible translation:

```
MOVQ a, %rax
MOVQ b, %rbx
ADDQ %rbx, %rax
IMULQ %rbx
MOVQ %rax, c
```

The IDIV instruction does the same thing, except backwards: it starts with a 128 bit integer value whose low 64 bits are in %rax and high 64 bits in %rdx, and divides it by the value given in the instruction. The quotient is placed in %rax and the remainder in %rdx. (If you want to implement the modulus instruction instead of division, just use the value of %rdx.)

To set up a division, you must make sure that both registers have the necessary sign-extended value. If the dividend fits in the lower 64 bits, but is negative, then the upper 64 bits must all be ones to complete the two's-complement representation. The CQO instruction serves the very specific purpose of sign-extending %rax into %rdx for division.

For example, to divide a by five:


```

MOVQ a, %rax    # set the low 64 bits of the dividend
CQO             # sign-extend %rax into %rdx
IDIVQ $5        # divide %rdx:%rax by 5,
                # leaving result in %rax

```

The instructions `INC` and `DEC` increment and decrement a register destructively. For example, the statement `a = ++b` could be translated as:

```

MOVQ b, %rax
INCQ %rax
MOVQ %rax, b
MOVQ %rax, a

```

The instructions `AND`, `OR`, and `XOR` perform destructive *bitwise* boolean operations on two values. Bitwise means that the operation is applied to each individual bit in the operands, and stored in the result.

So, `AND $0101B $0110B` would yield the result `$0100B`. In a similar way, the `NOT` instruction inverts each bit in the operand. For example, the bitwise C expression `c = (a & ~b) ;` could be translated like this:

```

MOVQ a, %rax
MOVQ b, %rbx
NOTQ %rbx
ANDQ %rax, %rbx
MOVQ %rbx, c

```

Be careful here: these instructions *do not* implement logical boolean operations according to the C representation that you are probably familiar with. For example, if you define “false” to be the integer zero, and “true” to be any non-zero value. In that case, `$0001` is true, but `NOT $0001B` is `$1110B`, which is also true! To implement that correctly, you need to use `CMP` with conditionals described below.²

Like the `MOV` instruction, the various arithmetic instructions can work on a variety of addressing modes. However, for your compiler project, you will likely find it most convenient to use `MOV` to load values in and out of registers, and then use only registers to perform arithmetic.

²Alternatively, you could use the bitwise operators as logical operators if you give `true` the integer value -1 (all ones) and `false` the integer value zero.

• **Comparisons and Jumps**

Using the JMP instruction, we may create a simple infinite loop that counts up from zero using the %rax register:

```
MOVQ $0, %rax
loop: INCQ %rax
      JMP loop
```

To define more useful structures such as terminating loops and if-then statements, we must have a mechanism for evaluating values and changing program flow. In most assembly languages, these are handled by two different kinds of instructions: compares and jumps.

All comparisons are done with the CMP instruction. CMP compares two different registers and then sets a few bits in the internal EFLAGS register, recording whether the values are the same, greater, or lesser. You don't need to look at the EFLAGS register directly. Instead a selection of conditional jumps examine the EFLAGS register and jump appropriately:

Instruction	Meaning
JE	Jump if Equal
JNE	Jump if Not Equal
JL	Jump if Less
JLE	Jump if Less or Equal
JG	Jump if Greater
JGE	Jump if Greater or Equal

For example, here is a loop to count %rax from zero to five:

```
MOVQ $0, %rax
loop: INCQ %rax
      CMPQ $5, %rax
      JLE loop
```

And here is a conditional assignment: if global variable x is greater than zero, then global variable y gets ten, else twenty:

```
MOVQ x, %rax
CMPQ $0, %rax
JLE .L1
.L0:
MOVQ $10, %rbx
JMP .L2
.L1:
MOVQ $20, %rbx
.L2:
MOVQ %rbx, y
```

Note that jumps require the compiler to define target labels. These labels must be unique and private within one assembly file, but cannot be seen outside the file unless a `.global` directive is given. Labels like `.L0`, `.L1`, etc, can be generated by the compiler on demand.

- ***The Stack***

The stack is an auxiliary data structure used primarily to record the function call history of the program along with local variables that do not fit in registers. By convention, the stack grows *downward* from high values to low values. The `%rsp` register is known as the **stack pointer** and keeps track of the bottom-most item on the stack.

To push `%rax` onto the stack, we must subtract 8 (the size of `%rax` in bytes) from `%rsp` and then write to the location pointed to by `%rsp`:

```
SUBQ $8, %rsp
MOVQ %rax, (%rsp)
```

Popping a value from the stack involves the opposite:

```
MOVQ (%rsp), %rax
ADDQ $8, %rsp
```

To discard the most recent value from the stack, just move the stack pointer the appropriate number of bytes :

```
ADDQ $8, %rsp
```

Of course, pushing to and popping from the stack referred to by `%rsp` is so common, that the two operations have their own instructions that behave exactly as above:

```
PUSHQ %rax
POPQ  %rax
```

Note that, in 64-bit code, `PUSH` and `POP` are limited to working with 64-bit values, so a manual `MOV` and `ADD` must be used if it is necessary to move smaller items to/from the stack.

- **Calling a Function**

Prior to the 64-bit architecture described here, a simple stack calling convention was used: arguments were pushed on the stack in reverse order, then the function was invoked with `CALL`. The called function looked for the arguments on the stack, did its work, and returned the result in `%eax`. The caller then removed the arguments from the stack.

However, 64-bit code uses a register calling convention, in order to exploit the larger number of available registers in the X86-64 architecture.³ This convention is known as the **System V ABI** [2] and is written out in a lengthy technical document. The complete convention is quite complicated, but this summary handles the basic cases:

Figure 10.3: Summary of System V ABI Calling Convention

- The first six integer arguments (including pointers and other types that can be stored as integers) are placed in the registers `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, and `%r9`, in that order.
- The first eight floating point arguments are placed in the registers `%xmm0`-`%xmm7`, in that order.
- Arguments in excess of those registers are pushed onto the stack.
- If the function takes a variable number of arguments (like `printf`) then the `%rax` register must be set to the number of floating point arguments.
- The return value of the function is placed in `%rax`.

In addition, we also need to know how the remaining registers are handled. A few are **caller-saved**, meaning that the calling function must save those values before invoking another function. Others are **callee-saved**, meaning that a function, when called, must save the values of those registers, and restore them on return. The argument and result registers need not be saved at all. Figure 10.4 shows these requirements.

To invoke a function, we must first compute the arguments and place them in the desired registers. Then, we must push the two caller-saved registers (`%r10` and `%r11`) on the stack, to save their values. We then issue the `CALL` instruction, which pushes the current instruction pointer on to the stack then jumps to the code location of the function. Upon return from the function, we pop the two caller-saved registers off of the stack, and look for the return value of the function in the `%rax` register.

³Note that there is nothing *stopping* you from writing a compiler that uses a stack calling convention. But if you want to invoke functions compiled by others (like the standard library) then you need to stick to the convention already in use.

Here is an example. The following C program:

```
int x=0;
int y=10;

int main()
{
    x = printf("value: %d\n",y);
}
```

could be translated to this:

```
.data
x:
    .quad 0
y:
    .quad 10
str:
    .string "value: %d\n"

.text
.global main
main:
    MOVQ $str, %rdi # first argument in %rdi: string
    MOVQ y, %rsi # second argument in %rsi: y
    MOVQ $0, %rax # there are zero float args

    PUSHQ %r10 # save the caller-saved regs
    PUSHQ %r11

    CALL printf # invoke printf

    POPQ %r11 # restore the caller-saved regs
    POPQ %r10

    MOVQ %rax, x # save the result in x

    RET # return from main function
```

Figure 10.4: System V ABI Register Assignments

Register	Purpose	Who Saves?
%rax	result	not saved
%rbx	scratch	callee saves
%rcx	argument 4	not saved
%rdx	argument 3	not saved
%rsi	argument 2	not saved
%rdi	argument 1	not saved
%rbp	base pointer	callee saves
%rsp	stack pointer	callee saves
%r8	argument 5	not saved
%r9	argument 6	not saved
%r10	scratch	CALLER saves
%r11	scratch	CALLER saves
%r12	scratch	callee saves
%r13	scratch	callee saves
%r14	scratch	callee saves
%r15	scratch	callee saves

- ***Defining a Leaf Function***

Because function arguments are passed in registers, it is easy to write a **leaf function** that computes a value without calling any other functions. For example, code for the following function:

```
square: function integer ( x: integer ) =
{
    return x*x;
}
```

Could be as simple as this:

```
.global square
square:
    MOVQ %rdi, %rax # copy first argument to %rax
    IMULQ %rax      # multiply it by itself
                   # result is already in %rax
    RET            # return to caller
```

Unfortunately, this won't work for a function that wants to invoke other functions, because we haven't set up the stack properly. A more complex approach is needed for the general case.

• *Defining a Complex Function*

A complex function must be able to invoke other functions and compute expressions of arbitrary complexity, and then return to the caller with the original state intact. Consider the following recipe for a function that accepts three arguments and uses two local variables:

```
.global func
func:
    pushq %rbp          # save the base pointer
    movq %rsp, %rbp     # set new base pointer

    pushq %rdi          # save first argument on the stack
    pushq %rsi          # save second argument on the stack
    pushq %rdx          # save third argument on the stack

    subq $16, %rsp      # allocate two more local variables

    pushq %rbx          # save callee-saved registers
    pushq %r12
    pushq %r13
    pushq %r14
    pushq %r15

    ### body of function goes here ###

    popq %r15           # restore callee-saved registers
    popq %r14
    popq %r13
    popq %r12
    popq %rbx

    movq %rbp, %rsp     # reset stack pointer
    popq %rbp           # recover previous base pointer
    ret                 # return to the caller
```

There is a lot to keep track of here: the arguments given to the function, the information necessary to return, and space for local computations. For this purpose, we use the base register pointer `%rbp`. Whereas the stack pointer `%rsp` points to the end of the stack where new data will be pushed, the base pointer `%rbp` points to the start of the values used by this function. The space between `%rbp` and `%rsp` is known as the **stack frame** for this function call.

There is one more complication: each function needs to use a selection of registers to perform computations. However, what happens when one function is called in the middle of another? We do not want any registers currently in use by the caller to be clobbered by the called function. To prevent this, each function must save and restore all of the registers that it uses by pushing them onto the stack at the beginning, and popping them off of the stack before returning. According to Figure 10.4, each function must preserve the values of `%rsp`, `%rbp`, `%rbx`, and `%r12-%r15` when it completes.

Here is the stack layout for `func`, defined above:

Contents	Address	
old <code>%rip</code> register	<code>8(%rbp)</code>	
old <code>%rbp</code> register	<code>(%rbp)</code>	→ <code>%rbp</code> points here
argument 0	<code>-8(%rbp)</code>	
argument 1	<code>-16(%rbp)</code>	
argument 2	<code>-24(%rbp)</code>	
local variable 0	<code>-32(%rbp)</code>	
local variable 1	<code>-40(%rbp)</code>	
saved register <code>%rbx</code>	<code>-48(%rbp)</code>	
saved register <code>%r12</code>	<code>-56(%rbp)</code>	
saved register <code>%r13</code>	<code>-64(%rbp)</code>	
saved register <code>%r14</code>	<code>-72(%rbp)</code>	
saved register <code>%r15</code>	<code>-80(%rbp)</code>	→ <code>%rsp</code> points here

Figure 10.5: Example X86-64 Stack Layout

Note that the base pointer (`%rbp`) locates the start of the stack frame. So, within the body of the function, we may use base-relative addressing against the base pointer to refer to both arguments and locals. The arguments to the function follow the base pointer, so argument zero is at `-8(%rbp)`, argument one at `-16(%rbp)`, and so forth. Past those are local variables to the function at `-32(%rbp)` and then saved registers at `-48(%rbp)`. The stack pointer points to the last item on the stack. If we use the stack for additional purposes, data will be pushed to further negative values. (Note that we have assumed all arguments and variables are 8 bytes large: different types would result in different offsets.)

Here is a complete example that puts it all together. Suppose that you have a B-minor function defined as follows:

```
compute: function integer
    ( a: integer, b: integer, c: integer ) =
{
    x:integer = a+b+c;
    y:integer = x*5;
    return y;
}
```

A complete translation of the function is on the next page. The code given is correct, but rather conservative. As it turned out, this particular function didn't need to use registers `%rbx` and `%r15`, so it wasn't necessary to save and restore them. In a similar way, we could have kept the arguments in registers without saving them to the stack. The result could have been computed directly into `%rax` rather than saving it to a local variable. These optimizations are easy to make when writing code by hand, but not so easy when writing a compiler.

For your first attempt at building a compiler, your code created will (probably) not be very efficient if each statement is translated independently. The preamble to a function must save all the registers, because it does not know *a priori* which registers will be used later. Likewise, a statement that computes a value must save it back to a local variable, because it does not know beforehand whether the local will be used as a return value. We will explore these issues later in Chapter 12 on optimization.

Figure 10.6: Complete X86 Example

```

.global compute
compute:
##### preamble of function sets up stack
pushq %rbp          # save the base pointer
movq %rsp, %rbp     # set new base pointer to rsp

pushq %rdi          # save first argument (a) on the stack
pushq %rsi          # save second argument (b) on the stack
pushq %rdx          # save third argument (c) on the stack

subq $16, %rsp      # allocate two more local variables

pushq %rbx          # save callee-saved registers
pushq %r12
pushq %r13
pushq %r14
pushq %r15

##### body of function starts here
movq -8(%rbp), %rbx  # load each arg into a register
movq -16(%rbp), %rcx
movq -24(%rbp), %rdx

addq %rdx, %rcx      # add the args together
addq %rcx, %rbx
movq %rbx, -32(%rbp) # store the result into local 0 (x)

movq -32(%rbp), %rbx # load local 0 (x) into a register.
movq $5, %rcx        # load 5 into a register
movq %rbx, %rax      # move argument in rax
imulq %rcx            # multiply them together
movq %rax, -40(%rbp) # store the result in local 1 (y)

movq -40(%rbp), %rax # move local 1 (y) into the result

##### epilogue of function restores the stack
popq %r15            # restore callee-saved registers
popq %r14
popq %r13
popq %r12
popq %rbx

movq %rbp, %rsp      # reset stack to base pointer.
popq %rbp            # restore the old base pointer

ret                  # return to caller

```

7.4 ARM Assembly

The ARM processor architecture has a history almost as long as the X86 architecture. It originated as the 32-bit **Acorn RISC Machine** used in the **Acorn Archimedes** personal computer in 1987, and has since evolved into a wide-used low-power CPU employed in many embedded and mobile systems, now known as the **Advanced RISC Machine (ARM)**. The evolving architecture has been implemented by a number of chip vendors working from a common architecture definition. The most recent versions of the architecture are ARMv7-A (32-bit) and ARMv8-A (64-bit.) This chapter will focus on the 32-bit architecture, with some remarks on differences in the 64-bit architecture at the end.

ARM is an example of a **Reduced Instruction Set Computer (RISC)** rather than a **Complex Instruction Set Computer (CISC)**. Compared to X86, ARM relies on a smaller set of instructions which are simpler to pipeline or run in parallel, reducing the complexity and energy consumption of the chip. ARM is sometimes considered “partially” RISC due to a few exceptions. For example, the difference in the time to execute some ARM instructions makes the pipeline imperfect, the inclusion of a barrel shifter for preprocessing brings forward more complex instructions, and conditional execution decreases some of the potential instructions executed and leads to less branching instructions so less energy used by the processor. We will mainly be focusing on the elements of the instruction set which allow us to do the most in a compiler, leaving the more complex aspects and optimizations of the programming language for later.

- **Registers and Data Types**

ARM-32 has a set of 16 general purpose registers, `r0-r15`, with the following conventions for use:

Name	Alias	Purpose
r0		General Purpose Register
r1		General Purpose Register
...		...
r10		General Purpose Register
r11	fp	Frame Pointer
r12	ip	Intra-Procedure-Call Scratch Register
r13	sp	Stack Pointer
r14	lr	Link Register (Return Address)
r15	pc	Program Counter

In addition to general purpose registers, there are two registers that cannot be directly accessed: the Current Program Status Register (CPSR) and the Saved Program Status Register (SPSR). These hold the results of comparison operations, as well as privileged data regarding the process

state. A user-level program cannot access these directly, but they can be set as a side effect of some operations.

ARM uses the following suffixes to indicate data sizes. Note that these have *different* meanings than in X86 assembly! If no suffix is given, the assembler assumes an unsigned word operand. Signed types are used to provide appropriate sign-extension when loading a small data type into a larger register. There is no register naming structure for anything below a word.

Data Type	Suffix	Size
Byte	B	8 bits
Halfword	H	16 bits
Word	W	32 bits
Double Word	-	64 bits
Signed Byte	SB	8 bits
Signed Halfword	SH	16 bits
Signed Word	SW	32 bits
Double Word	-	64 bits

• **Addressing Modes**

ARM makes the use of two different classes of instructions to move data between registers and between registers and memory. `MOV` copies data and constants between registers, while `LDR` (load) and `STR` (store) instructions are used to move data between registers and memory.

The `MOV` instruction is used to move a known immediate value into a given register or move another register into the first register. In ARM, immediate values are denoted by a `#`. However, these immediate values must be 16-bits or less. If they are greater, the `LDR` instruction must be used instead. Most ARM instructions indicate the destination register on the left and the source register on the right. (`STR` is an exception.) So for moving data between immediate values and registers we would have the following:

Mode	Example
Immediate	<code>MOV r0, #3</code>
Register	<code>MOV r1, r0</code>

The mnemonic letter for each data type can be appended to the `MOV` instruction allowing us to be sure of which is being transferred and how that data is being transferred. If not, the assembler assumes an entire word.

To move values in and out of memory, use the Load (`LDR`) and Store (`STR`) instructions, both of which indicate the source or destination register as the first argument, and the source or destination memory address as the second argument. In the simplest case, the address is given by a register and indicated in brackets:

Figure 10.7: ARM Addressing Modes

Address Mode	Example
Literal	LDR Rd, =0xABCD1234
Absolute Address	LDR Rd, =label
Register Indirect	LDR Rd, [Ra]
Pre-indexing - Immediate	LDR Rd, [Ra, #4]
Pre-indexing - Register	LDR Rd, [Ra, Ro]
Pre-indexing - Immediate & Writeback	LDR Rd, [Ra, #4]!
Pre-indexing - Register & Writeback	LDR Rd, [Ra, Ro]!
Post-indexing - Immediate	LDR Rd, [Ra], #4
Post-indexing - Register	LDR Rd, [Ra], Ro

```
LDR Rd, [Ra]
STR Rs, [Ra]
```

In this case, Rd denotes the destination register, Rs denotes the source register and Ra denotes the register containing the address. (Note that the memory address must be aligned to the data type: a byte can load a value from any address, a half-word from every even address, and so on.)

Both LDR and STR support a variety of addressing modes, shown in Figure 10.7. First, LDR can be used to load a literal value (or label address) of a full 32-bits into a register. (See the next section for a full explanation of this.) Unlike X86, there is no single instruction that loads a value from a given memory address. To accomplish this, you must first load the address into a register, and then perform a register-indirect load:

```
LDR r1, =x
LDR r2, [r1]
```

A number of more complex addressing modes are available which facilitate the implementation of pointers, arrays, and structures in high-level programs. Pre-indexing modes add a constant (or register) to a base register, and then load from the computed address:

```
LDR r1, [r2, #4] ; Load from address r2 + 4
LDR r1, [r2, r3] ; Load from address r2 + r3
```

It is also possible to write-back to the base register by appending a bang (!) character. This indicates that the computed address should be saved in the base register after the address is loaded:

```
LDR r1, [r2, #4]! ; Load from r2 + 4 then r2 += 4
LDR r1, [r2, r3]! ; Load from r2 + r3 then r2 += r3
```

Post-indexing modes do the same thing, but in the reverse order. First, the load is performed from the base register, then the base register is incremented:

```
LDR r1, [r2], #4 ; Load from r2 then r2 += 4
LDR r1, [r2], r3 ; Load from r2 then r2 += r3
```

These complex pre-indexing and post-indexing modes make it possible to have single-instruction implementations of idiomatic C operations like `b = a++`. The corresponding modes are also available to the `STR` instruction.

Absolute addresses (and other large literals) are somewhat more complicated in ARM. Because every instruction must fit into a 32-bit word, it is not possible to fit a 32-bit address into an instruction, alongside the opcode. Instead, large literals must be stored in a **literal pool**, which is a small region of data inside the code section of the program. A literal can be loaded from a pool with a PC-relative load instruction, which can reference ± 4096 bytes from the loading instruction. This results in several small literal pools being scattered throughout the program, so that each one is close to the load instruction that uses it.

The ARM assembler generally hides this complexity from the user. When a label or large literal is prefixed with an equals sign, this indicates to the assembler that the marked value should be placed into a literal pool, and a corresponding PC-relative instruction emitted instead.

For example, the following instructions, which load the address of `x` into `r1`, then the value of `x` into `r2`:

```
LDR r1, =x
LDR r2, [r1]
```

Will be expanded into this load of the address of `x` from an adjacent literal pool, followed by loading the value of `x`:

```
LDR r1, .L1
LDR r2, [r1]
B .end
.L1:
    .word x
.end:
```

• **Basic Arithmetic**

ARM provides three-address arithmetic instructions on registers. The `ADD` and `SUB` instructions specify the result register as the first argument, and compute on the second and third arguments. The third operand may be an 8-bit constant, or a register with an optional shift applied. The variants

with carry-in enabled will add the C bit of the CPSR to the result. All four take an optional S suffix which will set the condition flags (including carry) on completion.

Instruction	Example
Add	ADD Rd, Rm, Rn
Add with carry-in	ADC Rd, Rm, Rn
Subtract	SUB Rd, Rm, Rn
Subtract with carry-in	SBC Rd, Rm, Rn

Multiplication works much the same way, except that multiplying two 32-bit numbers could result in a 64-bit value. The ordinary `MUL` discards the high bits of the results, while `UMULL` puts the 64-bit result in two 32-bit registers. The signed variant `SMULL` will sign extend the high register as needed.

Instruction	Example
Multiplication	MUL Rd, Rm, Rn
Unsigned Long Multiplication	UMULL RdHi, RdLo, Rm, Rn
Signed Long Multiplication	SMULL RdHi, RdLo, Rm, Rn

There is no division instruction in ARM, because it cannot be carried out in a single pipelined cycle. Instead, when division is needed, it is accomplished by invoking an external function in a standard library. This is left as an exercise for the reader.

The logical instructions are very similar in structure to the arithmetic instructions. We have the bitwise-and, bitwise-or, bitwise-exclusive-or and bitwise-bit-clear, which is the equivalent of a bitwise-and of the first value and the inverted second value. The move-not `MVN` instruction performs a bitwise-not while moving from one register to another.

Instruction	Example
Bitwise-And	AND Rd, Rm, Rn
Bitwise-Or	ORR Rd, Rm, Rn
Bitwise-Xor	EOR Rd, Rm, Rn
Bitwise-Bit-Clear	BIC Rd, RM, Rn
Move-Not	MVN Rd, Rn

• Comparisons and Branches

The `CMP` instruction compares two values and sets the N (negative) and Z (zero) flags in the CPSR, to be read by later instructions. In the case of comparing a register and an immediate value, the immediate must be the second operand:

```
CMP Rd, Rn
CMP Rd, #imm
```

Figure 10.8: ARM Branch Instructions

Opcode	Meaning		
B	Branch Always	BL	Branch and Link
BX	Branch and Exchange	BLX	Branch-Link-Exchange
BEQ	Equal	BVS	Overflow Set
BNE	Not Equal	BVC	Overflow Clear
BGT	Greater Than	BHI	Higher (unsigned >)
BGE	Greater Than or Equal	BHS	Higher or Same (uns. >=)
BLT	Less Than	BLO	Lower (unsigned <)
BLE	Less Than or Equal	BLS	Lower or Same (uns. <=)
BMI	Negative	BPL	Positive or Zero

In addition, an "S" can be appended to the arithmetic instructions to update the CPSR in a similar way. For example, SUBS will subtract two values, store the result, and update the CPSR.

A variety of branch instructions consult the previously-set values of the CPSR, and then jump to a given label, if the appropriate flags are set. An unconditional branch is specified with simply B.

For example, to count from zero to five:

```
MOV r0, #0
loop: ADD r0, r0, 1
      CMP r0, #5
      BLT loop
```

And to conditionally assign a global variable y ten if x is greater than 0 and 20 if it is not

```
LDR r0, =x
LDR r0, [r0]
CMP r0, #0
BGT .L1
.L0:
    MOV r0, #20
    B .L2
.L1:
    MOV r0, #10
.L2:
    LDR r1, =y
    STR r0, [r1]
```

The branch-and-link (BL) instruction, is used to implement function calls. BL sets the link register to be the address of the next instruction, and then jumps to the given label. The link register is then used as the return

address at the conclusion of the function. The `BX` instruction branches to the address given in a register, and is most frequently used to return from a function call by branching to the link register. `BLX` performs a branch-and-link to the address given by a register, and can be used to invoke function pointers, virtual methods, or any other indirect jump.

A special feature of the ARM instruction set is **conditional execution**. A 4-bit field in each instruction word indicates one of 16 possible conditions that must hold, otherwise the instruction is ignored. The various types of conditional branch shown above are simply a plain branch (`B`) instruction with the various conditions applied. The same two letter suffixes can be applied to almost any instruction.

For example, suppose we have the following code fragment, which increments either `a` or `b`, depending on which one is smaller:

```
if(a<b) { a++; } else { b++; }
```

Instead of implementing this as control flow using branches and labels, we can simply make each of the two additions conditional upon a previous comparison. Whichever condition holds true will be executed, and the other skipped. Assuming that `a` and `b` are held in `r0` and `r1` respectively:

```
CMP    r0, r1
ADDLT  r0, r0, #1
ADDGE  r1, r1, #1
```

• **The Stack**

The stack is an auxiliary data structure used primarily to record the function call history of the program along with local variables that do not fit in registers. By convention, the stack grows *downward* from high values to low values. The `sp` register is known as the **stack pointer** and keeps track of the bottom-most item on the stack.

To push the `r0` register onto the stack, we must subtract the size of the register from `sp`, and then store `r0` into the location pointed to by `sp`:

```
SUB sp, sp, #4
STR r0, [sp]
```

Alternatively, this can be done with a single instruction making use of pre-indexing and writeback:

```
STR r0, [sp, #-4]!
```

The `PUSH` pseudo-instruction accomplishes the same thing, but can also move any number of registers (encoded as a bitmask) to the stack. Curly braces are used to indicate the list of registers to push:

Figure 10.9: Summary of ARM Calling Convention

- The first four arguments are placed in registers r0, r1, r2 and r3.
- Additional arguments are pushed on the stack in reverse order.
- The caller must save r0-r3 and r12, if needed.
- the caller must always save r14, the link register.
- The **callee** must save r4-r11, if needed.
- The result is placed in r0.

```
PUSH {r0,r1,r2}
```

Popping a value off the stack involves the opposite:

```
LDR r0, [sp]  
ADD sp, sp, #4
```

Once again this can be done with a single instruction:

```
LDR r0, [sp], #4
```

And, to pop a set of registers all at once:

```
POP {r0,r1,r2}
```

Unlike X86, any data items ranging from a byte to a double-word can be pushed on to the stack, as long as data alignment is respected.

- ***Calling a Function***

ARM uses a register calling convention described by the ARM-Thumb Procedure Call Standard (ATPCS) [4], which is summarized in Figure 10.9.

To call a function, place the desired arguments in the registers r0-r3, save the (current) value of the link register, and then use the BL instruction to jump to the function. Upon return, restore the previous value of the link register, and examine the result in register r0.

For example, the following C function:

```
int x=0;  
int y=10;  
int main() {  
    x = printf("value: %d\n",y);  
}
```

Figure 10.10: ARM Register Assignments

Register	Purpose	Who Saves?
r0	argument 0 / result	not saved
r1	argument 1	CALLER saves
r2	argument 2	CALLER saves
r3	argument 3	CALLER saves
r4	scratch	callee saves
...	...	...
r10	scratch	callee saves
r11	frame pointer	callee saves
r12	intraprocedure	CALLER saves
r13	stack pointer	callee saves
r14	link register	CALLER saves
r15	program counter	saved in link register

Would become the following in ARM:

```
.data
x: .word 0
y: .word 10
S0: .ascii "value: %d\012\000"
.text
main:
    LDR r0, =S0 @ Load address of S0
    LDR r1, =y   @ Load address of y
    LDR r1, [r1] @ Load value of y

    PUSH {ip,lr} @ Save registers
    BL printf    @ Call printf
    POP {ip,lr}  @ Restore registers

    LDR r1, =x   @ Load address of x
    STR r0, [r1] @ Store return value in x
.end
```

- **Defining a Leaf Function**

Because function arguments are passed in registers, it is easy to write a **leaf function** that computes a value without calling any other functions. For example, code for the following function:

```
square: function integer ( x: integer ) =
{
    return x*x;
```

```
}
```

Could be as simple as this:

```
.global square
square:
    MUL    r0, r0, r0    @ multiply argument by itself
    BX     lr            @ return to caller
```

Unfortunately, this won't work for a function that wants to invoke other functions, because we haven't set up the stack properly. A more complex approach is needed for the general case.

- ***Defining a Complex Function***

A complex function must be able to invoke other functions and compute expressions of arbitrary complexity, and then return to the caller with the original state intact. Consider the following recipe for a function that accepts three arguments and uses two local variables:

```
func:
    PUSH {fp}            @ save the frame pointer
    MOV fp, sp           @ set the new frame pointer
    PUSH {r0,r1,r2}      @ save the arguments on the stack
    SUB sp, sp, #8       @ allocate two more local variables
    PUSH {r4-r10}        @ save callee-saved registers

    @@@ body of function goes here @@@

    POP {r4-r10}         @ restore callee saved registers
    MOV sp, fp           @ reset stack pointer
    POP {fp}             @ recover previous frame pointer
    BX     lr            @ return to the caller
```

Through this method, we ensure that we are able to save all the values in the registers into the stack and ensure that no data will be lost. The stack, once this has been done, looks very similar to that of the X86 stack, just with some extra callee-saved registers stored on the stack.

Here is a complete example that puts it all together. Suppose that you have a B-minor function defined as follows:

```
compute: function integer
    ( a: integer, b: integer, c: integer ) =
{
    x: integer = a+b+c;
    y: integer = x*5;
    return y;
}
```

Figure 10.11: Example ARM Stack Frame

Contents	Address	
Saved r12	[fp, #8]	
Old LR	[fp, #4]	
Old Frame Pointer	[fp]	→ fp points here
Argument 2	[fp, #-4]	
Argument 1	[fp, #-8]	
Argument 0	[fp, #-12]	
Local Variable 1	[fp, #-16]	
Local Variable 0	[fp, #-20]	
Saved r10	[fp, #-24]	
Saved r9	[fp, #-28]	
Saved r8	[fp, #-32]	
Saved r7	[fp, #-36]	
Saved r6	[fp, #-40]	
Saved r5	[fp, #-44]	
Saved r4	[fp, #-48]	→ sp points here

A complete translation of the function is on the next page. Note that this is one of many ways to construct a valid stack frame for a function definition. Other approaches are valid, as long as the function uses the stack frame consistently. For example, the callee could first push all of the argument and scratch registers on the stack, and then allocate space for local variables below that. (The reverse process must be used on function exit, of course.)

Another common approach is for the callee `PUSH {fp, ip, lr, pc}` on to the stack, before pushing arguments and local variables. While not strictly required for implementing the function, it provides additional debugging information in the form of a **stack backtrace** so that a debugger can look backwards through the call stack and easily reconstruct the current execution state of the program.

The code given is correct, but rather conservative. As it turned out, this particular function didn't need to use registers `r4` and `r5`, so it wasn't necessary to save and restore them. In a similar way, we could have kept the arguments in registers without saving them to the stack. The result could have been computed directly into `r0` rather than saving it to a local variable. These optimizations are easy to make when writing code by hand, but not so easy when writing a compiler.

For your first attempt at building a compiler, your code created will (probably) not be very efficient if each statement is translated independently. The preamble to a function must save all the registers, because it does not know *a priori* which registers will be used later. Likewise, a statement that computes a value must save it back to a local variable, because

Figure 10.12: Complete ARM Example

```

.global compute
compute:
##### preamble of function sets up stack
PUSH {fp}           @ save the frame pointer
MOV  fp, sp         @ set the new frame pointer
PUSH {r0,r1,r2}     @ save the arguments on the stack
SUB  sp, sp, #8     @ allocate two more local variables
PUSH {r4-r10}       @ save callee-saved registers

##### body of function starts here
LDR  r0, [fp,#-12]   @ load argument 0 (a) into r0
LDR  r1, [fp,#-8]    @ load argument 1 (b) into r1
LDR  r2, [fp,#-4]    @ load argument 2 (c) into r2
ADD  r1, r1, r2      @ add the args together
ADD  r0, r0, r1
STR  r0, [fp,#-20]   @ store the result into local 0 (x)
LDR  r0, [fp,#-20]   @ load local 0 (x) into a register.
MOV  r1, #5          @ move 5 into a register
MUL  r2, r0, r1      @ multiply both into r2
STR  r2, [fp,#-16]   @ store the result in local 1 (y)
LDR  r0, [fp,#-16]   @ move local 1 (y) into the result

##### epilogue of function restores the stack
POP  {r4-r10}       @ restore callee saved registers
MOV  sp, fp         @ reset stack pointer
POP  {fp}           @ recover previous frame pointer
BX   lr             @ return to the caller

```

it does not know beforehand whether the local will be used as a return value. We will explore these issues later in Chapter 12 on optimization.

- **64-bit Differences**

The 64-bit ARMv8-A architecture provides two execution modes: The A32 mode supports the 32-bit instruction set described above, and the A64 mode supports a new 64-bit execution model. This permits a 64-bit CPU with a supporting operating system to execute a mix of 32-bit and 64-bit programs simultaneously. Though not binary compatible with A32, the A64 model follows much of the same architectural principles, with a few key changes:

Word Size. A64 instructions are still a fixed size of 32 bits, however, registers and address computations are 64 bits.

Registers. A64 has 32 64-bit registers, named $x0-x31$. $x0$ is a dedicated zero register: when read, it always provides the value zero, when written, there is no effect. $x1-x15$ are general purpose registers, $x16$ and $x17$ are for interprocess communication, $x29$ is the frame pointer, $x30$ is the link register and $x31$ is the stack pointer. (The program counter is not directly accessible from user code.) Instead of using a data type suffix, a 32-bit value may be indicated by naming a register as $w\#$.

Instructions. A64 instructions are largely the same as A32, using the same mnemonics, with a few differences. Conditional predicates are no longer part of every instruction. Instead, all conditional codes must perform an explicit `CMP` and then a conditional branch. The `LDM/STM` instructions and pseudo-instructions `PUSH/POP` are not available and must be replaced with a sequence of explicit loads and stores. (This can be made more efficient by using `LDP/STP` which load and store pairs of registers.)

Calling Convention. To invoke a function, the first eight arguments are placed in registers $x0-x7$, and the remainder are pushed on to the stack. The caller must preserve registers $x9-x15$ and $x30$ while the callee must preserve $x19-x29$. The (scalar) return value is placed in $x0$, while extended return values are pointed to by $x8$.

7.5 Further Reading

This chapter has given you a brief orientation to the core features of the X86 and ARM architectures, enough to write simple programs in the most direct way. However, you will certainly need to look up specific details of individual instructions in order to better understand their options and limitations. Now you are ready to read the detailed reference manuals and keep them handy while you construct your compiler:

1. Intel64 and IA-32 Architectures Software Developer Manuals. Intel Corp., 2017.
<http://www.intel.com/content/www/us/en/processors/architectures-software-developer-manuals.html>
2. System V Application Binary Interface, Jan Hubicka, Andreas Jaeger, Michael Matz, and Mark Mitchell (editors), 2013.
<https://software.intel.com/sites/default/files/article/402129/mpx-linux64-abi.pdf>
3. ARM Architecture Reference Manual ARMv8. ARM Limited, 2017.
https://static.docs.arm.com/ddi0487/bb/DDI0487B_b_armv8_arm.pdf.
4. The ARM-THUMB Procedure Call Standard. ARM Limited, 2000.
<https://developer.arm.com/documentation/ihl0042/f/>

Chapter 8 – Code Generation

8.1 Introduction

Congratulations, you have made it to the final stage of the compiler! After scanning and parsing the source code, constructing the AST, performing type checking, and generating an intermediate representation, we are now ready to generate some code.

To start, we are going to take a naïve approach to code generation, in which we consider each element of the program in isolation. Each expression and statement will be generated as a standalone unit, without reference to its neighbors. This is easy and straightforward, but it is conservative and will lead to a large amount of non-optimal code. But it will work, and give you a starting point for thinking about more sophisticated techniques.

The examples will focus on X86-64 assembly code, but they are not hard to adapt to ARM and other assembly languages as needed. As with previous stages, we will define a method for each element of a program. `decl_codegen` will generate code for a declaration, calling `stmt_codegen` for a statement, `expr_codegen` for an expression, and so on. These relationships are shown in Figure 11.1.

Once you have learned this basic approach to code generation, you will be ready for the *following* chapter, in which we consider more complex methods for generating more highly optimized code.

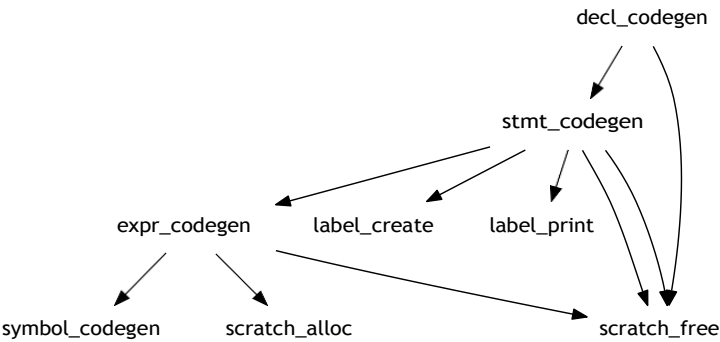
8.2 Supporting Functions

Before generating some code, we need to set up a few supporting functions to keep track of a few details. To generate expressions, you will need some **scratch registers** to hold intermediate values between operators. Write three functions with the following interface:

```
int scratch_alloc();
void scratch_free( int r );
const char * scratch_name( int r );
```

Looking back at Chapter 10, you can see that we set aside each register for a purpose: some are for function arguments, some for stack frame

Figure 11.1: Code Generation Functions



management, and some are available for scratch values. Take those scratch registers and put them into a table like this:

r	0	1	2	3	4	5	6
name	%rbx	%r10	%r11	%r12	%r13	%r14	%r15
inuse	X		X				

Then, write `scratch_alloc` to find an unused register in the table, mark it as in use, and return the register number `r`. `scratch_free` should mark the indicated register as available. `scratch_name` should return the name of a register, given its number `r`. Running out of scratch registers is possible, but unlikely, as we will see below. For now, if `scratch_alloc` cannot find a free register, just emit an error message and halt.

Next, we will need to generate a large number of unique, anonymous labels that indicate the targets of jumps and conditional branches. Write two functions to generate and display labels:

```
int label_create();
const char * label_name( int label );
```

`label_create` simply increments a global counter and returns the current value. `label_name` returns that label in a string form, so that label 15 is represented as `".L15"`.

Finally, we need a way of mapping from the symbols in a program to the assembly language code representing those symbols. For this, write a function to generate the address of a symbol:

```
const char * symbol_codegen( struct symbol *s );
```

This function returns a string which is a fragment of an instruction, representing the address computation needed for a given symbol. Write `symbol_codegen` to first examine the scope of the symbol. Global variables are easy: the name in assembly language is the same as in the source language. If you have a `symbol` structure representing the global variable `count:integer`, then `symbol_codegen` should simply return `count`.

Symbols that represent local variables and function parameters should instead return an address computation that yields the position of that local variable or parameter on the stack. The groundwork for this was laid in the typechecking phase, where you assigned each parameter and each local variable a unique sequence number.

For example, suppose you have this function definition:

```
f: function void ( x: integer, y: integer ) =
{
    z: integer = 10;
    return x + y + z;
}
```

In this case, `x` has parameter position zero, `y` has position one, and `z` is local variable zero. Now look back at Figure 10.5, which shows the stack layout on the X86-64 processor. Parameter position zero is at the address `-8(%rbp)`, parameter one is at `-16(%rbp)`, and then local variable zero follows after that at `-24(%rbp)`.

Given that, you can now extend `symbol_codegen` to return a string describing the precise stack address of local variables and parameters, knowing only its position in the stack frame.

11.1 Generating Expressions

The basic approach to generating assembly code for an expression is to perform a post-order traversal of the AST or DAG, and emit one or more instructions for each node. The main idea is to keep track of the registers in which each intermediate value is stored. To do this, add a `reg` field to the AST or DAG node structure, which will hold the number of a register returned by `scratch_alloc`. As you visit each node, emit an instruction and place into the `reg` field the number of the register containing that value. When the node is no longer needed, call `scratch_free` to release it.

Suppose we want to generate X86 code for the following DAG, where *a*, *b* and *c* are global integers:

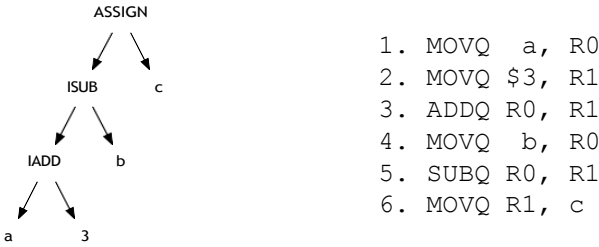


Figure 11.2: Example of Generating X86 Code from a DAG

A post-order traversal would visit the nodes in the following order:

1. Visit the *a* node. Call `scratch_alloc` to allocate a new register (0) and save that in `node->reg`. Then emit the instruction `MOVQ a, R0` to load the value into register zero.¹
2. Visit the *3* node. Call `scratch_alloc` to allocate a new register (1) and emit the instruction `MOVQ $3, R1`.
3. Visit the *IADD* node. By examining the two children of this node, we can see that their values are stored in registers *R0* and *R1*, so we emit an instruction that adds them together: `ADDQ R0, R1`. This is a destructive two-address instruction which leaves the result in *R1*. *R0* is no longer needed, so we call `scratch_free(0)`.
4. Visit the *b* node. Call `scratch_alloc` to allocate a new register (0) and emit `MOVQ b, R0`.
5. Visit the *ISUB* node. Emit `SUBQ R0, R1`, leaving the result in *R1*, and freeing register *R0*.
6. Visit the *c* node, but don't emit anything, because it is the target of the assignment.
7. Visit the *ASSIGN* node and emit `MOVQ R1, c`.

¹Note that the actual name of register *R0* is `scratch_name(0)`, which is `%rbx`. To keep the example clear, we will call them *R0*, *R1*, etc. for now.

And here is the same code with the actual register names provided by `scratch_name`:

```
MOVQ    a, %rbx
MOVQ    $3, %r10
ADDQ    %r10, %rbx
MOVQ    b, %rbx
SUBQ    %rbx, %r10
MOVQ    %r10, c
```

Here is how to implement it in code. Write a function called `expr_codegen` that first recursively calls `expr_codegen` for its left and right children. This will cause each child to generate code such that the result will be left in the register number noted in the `reg` field. The current node then generates code using those registers, and frees the registers it no longer needs. Figure 11.3 gives a skeleton for this approach.

A few additional refinements are needed to the basic process.

First, not all symbols are simple global variables. When a symbol forms part of an instruction, use `symbol_codegen` to return the string that gives the specific address for that symbol. For example, if `a` was the first parameter to the function, then the first instruction in the sequence would have looked like this instead:

```
MOVQ    -8(%rbp), %rbx
```

Second, some nodes in the DAG may require multiple instructions, so as to handle peculiarities of the instruction set. You will recall that the X86 `IMUL` only takes one argument, because the first argument is always `%rax` and the result is always placed in `%rax` with the overflow in `%rdx`. To perform the multiply, we must move one child register into `%rax`, multiply by the other child register, and then move the result from `%rax` to the destination scratch register. For example, the expression `(x*10)` would translate as this:

```
MOV    $10, %rbx
MOV    x,   %r10
MOV    %rbx, %rax
IMUL   %r10
MOV    %rax, %r11
```

Of course, this also means that `%rax` and `%rdx` cannot be used for other purposes while a multiply is in progress. Since we have a large number of scratch registers to work with, we will just not use `%rdx` for any other purpose in our basic code generator.

```

void expr_codegen( struct expr *e )
{
    if(!e) return;

    switch(e->kind) {

        // Leaf node: allocate register and load value.

        case EXPR_NAME:
            e->reg = scratch_alloc();
            printf("MOVQ %s, %s\n",
                symbol_codegen(e->symbol),
                scratch_name(e->reg));
            break;

        // Interior node: generate children, then add them.

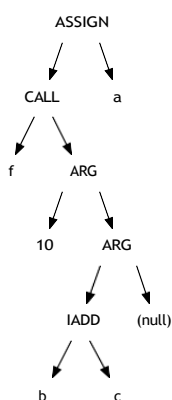
        case EXPR_ADD:
            expr_codegen(e->left);
            expr_codegen(e->right);
            printf("ADDQ %s, %s\n",
                scratch_name(e->left->reg),
                scratch_name(e->right->reg));
            e->reg = e->right->reg;
            scratch_free(e->left->reg);
            break;

        case EXPR_ASSIGN:
            expr_codegen(e->left);
            printf("MOVQ %s, %s\n",
                scratch_name(e->left->reg),
                symbol_codegen(e->right->symbol));
            e->reg = e->left->reg;
            break;

        . . .
    }
}

```

Figure 8.3: Expression Generation Skeleton



```

MOVQ $10, %rbx
MOVQ b, %r10
MOVQ c, %r11
ADDQ %r10, %r11
MOVQ %r11, %rsi
MOVQ %rbx, %rdi
PUSHQ %r10
PUSHQ %r11
CALL f
POPQ %r11
POPQ %r10
MOVQ %rax, %rbx
MOVQ %rbx, a
  
```

Figure 8.4: Generating Code for a Function Call

Third, how do we invoke a function? Recall that a function is represented by a single CALL node, with each of the arguments in an unbalanced tree of ARG nodes. Figure 11.4 gives an example of a DAG representing the expression $a = f(10, b+c)$.

The code generator must evaluate each of the ARG nodes, computing the value of each left hand child. If the machine has a stack calling convention, then each ARG node corresponds to a PUSH onto the stack. If the machine has a register calling convention, generate all of the arguments, then copy each one into the appropriate argument register after all arguments are generated. Then, emit CALL on the function name, after saving any caller-saved registers. When the function call returns, move the value from the return register (`%rax`) into a newly allocated scratch register and restore the caller-saved registers.

Finally, note carefully the side effects of expressions. Every expression has a **value** which is computed and left in a scratch register for its parent node to consider. Some expressions also have **side effects** which are actions in addition to the value. With some operators, it is easy to overlook one or the other!

For example, the expression $(x=10)$ yields a *value* of 10, which means you can use that expression anywhere a value is expected. This is what allows you to write $y=x=10$ or $f(x=10)$. The expression also has the *side effect* of storing the value 10 into the variable x . When you generate code for $x=10$ assignment, be sure to carry out the side effect of storing 10 into x (that's obvious) but also retain the value 10 in the register that represents the value of the expression.

11.2 Generating Statements

Now that we have encapsulated expression generation into a single function `expr_codegen`, we can begin to build up larger code structures that rely upon expressions. `stmt_codegen` will create code for all control flow statements. Begin by writing a skeleton for `stmt_codegen` like this:

```
void stmt_codegen( struct stmt *s )
{
    if(!s) return;

    switch(s->kind) {
        case STMT_EXPR:
            ...
            break;
        case STMT_DECL:
            ...
            break;
        ...
    }
    stmt_codegen(s->next);
}
```

Figure 8.5: Statement Generator Skeleton

Now consider each kind of statement in turn, starting with the easy cases. If the statement describes a declaration `STMT_DECL` of a local variable, then just delegate that by calling `decl_codegen`:

```
case STMT_DECL:
    decl_codegen(s->decl);
    break;
```

A statement that consists of an expression (`STMT_EXPR`) simply requires that we call `expr_codegen` on that expression, and then free the scratch register containing the top-level value of the expression. (In fact, every time `expr_codegen` is called, a scratch register should be freed.)

```
case STMT_EXPR:
    expr_codegen(s->expr);
    scratch_free(s->expr->reg);
    break;
```

A `return` statement must evaluate an expression, move it into the designated register for return values `%rax`, and then jump to the function epilogue, which will unwind the stack and return to the call point. (See below for more details about the prologue.)


```

case STMT_RETURN:
    expr_codegen(s->expr);
    printf("MOV %s, %%rax\n", scratch_name(s->expr->reg));
    printf("JMP .%s_epilogue\n", function_name);
    scratch_free(s->expr->reg);
    break;

```

(The careful reader will notice that this code needs to know the name of the function that contains this statement. You will have to figure out a way to pass that information down.)

Control flow statements are more interesting. It's useful to first consider what we want the output assembly language to look like, and then work backwards to get the code for generating it.

Here is a template for a conditional statement:

```

if ( expr ) {
    true-statements
} else {
    false-statements
}

```

To express this in assembly, we must evaluate the control expression so that its value resides in a known register. A `CMP` expression is then used to test if the value is equal to zero (false). If the expression is false, then we must jump to the false branch of the statement with a `JE` (jump-if-equal) statement. Otherwise, we continue through to the true branch of the statement. At the end of the true statement, we must `JMP` over the else body to the end of the statement.

```

expr
CMP $0, register
JE false-label
true-statements
JMP done-label
false-label :
false-statements
done-label :

```

Once you have a skeleton of the desired code, writing the code generator is easy. First, generate two new labels, then call `expr_codegen` for each expression, `stmt_codegen` for each statement, and substitute the few additional instructions as needed to make the overall structure.

```
case STMT_IF:
    int else_label = label_create();
    int done_label = label_create();
    expr_codegen(s->expr);
    printf("CMP $0, %s\n", scratch_name(s->expr->reg));
    scratch_free(s->expr->reg);
    printf("JE %s\n", label_name(else_label));
    stmt_codegen(s->body);
    printf("JMP %s\n", label_name(done_label));
    printf("%s:\n", label_name(else_label));
    stmt_codegen(s->else_body);
    printf("%s:\n", label_name(done_label));
    break;
```

A similar approach can be used to generate loops. Here is the source template of a for-loop:

```
for ( init-expr ; expr ; next-expr ) {
    body-statements
}
```

And here is the corresponding assembly template. First, evaluate the initializing expression. Then, upon each iteration of the loop, evaluate the control expression. If false, jump to the end of the loop. If not, execute the body of the loop, then evaluate the next expression.

```
init-expr
top-label:
expr
CMP $0, register
JE done-label
body-statements
next-expression
JMP top-label
done-label:
```

Writing the code generator is left as an exercise for the reader. Just keep in mind that each of the three expressions in a for-loop can be omitted. If

the *init-expr* or the *next-expr* are omitted, they have no effect. If the *expr* is omitted, it is assumed to be true.²

Many languages have loop-control constructs like `continue;` and `break;`. In these cases, the compiler must keep track of the labels associated with the current loop being generated, and convert these into a `JMP` to the top label, or the done-label respectively.

The `print` statement in B-Minor is a special case of an imperative statement with variant behavior depending on the type of the expression to be printed. For example, the following `print` statement must generate slightly different code for the integer, boolean, and string to be printed:

```
i: integer = 10;
b: boolean = true;
s: string = "\n";
print i, b, s;
```

Obviously, there is no simple assembly code corresponding to the display of an integer. In this case, a common approach is to reduce the task into an abstraction that we already know. The printing of integers, booleans, strings, etc., can be delegated to function calls that explicitly perform those actions. The generated code for `print i, b, s` is then equivalent to this:

```
print_integer(i);
print_boolean(b);
print_string(s);
```

So, to generate a `print` statement, we simply generate the code for each expression to be printed, determine the type of the expression with `expr_typecheck` and then emit the corresponding function call.

Of course, each of these functions must be written and then linked into each instance of a B-Minor program, so they are available when needed. These functions, and any others necessary, are collectively known as the **runtime library** for B-Minor programs. As a general rule, the more high-level a programming language, the more runtime support is needed.

²Yes, each of the three components of a for-loop are expressions. It is customary that the first has a side effect of initialization (`i=0`), the second is a comparison (`i<10`), and the third has a side effect to generate the next value (`i++`), but they are all just plain expressions.

8.3 Conditional Expressions

Now that you know how to generate control flow statements, we must return to one aspect of expression generation. Conditional expressions (less-than, greater-than, equals, etc.) compare two values and return a boolean value. They most frequently appear in control flow expressions but can also be used as simple values, like this:

```
b: boolean = x < y;
```

The problem is that there is no single instruction that simply performs the comparison and places the boolean result in a register. Instead, you must go the long way around and make a control flow construct that compares the two expressions, then constructs the desired result.

For example, if you have a conditional expression like this:

<i>left-expr</i>	<	<i>right-expr</i>
------------------	---	-------------------

then generate code according to this template:

<i>left-expr</i>
<i>right-expr</i>

```

CMP left-register, right-register
JLT true-label
MOV false, result-register
JMP done-label
true-label:
  MOV true, result-register
done-label:
```

Of course, for different conditional operators, use a different jump instruction in the appropriate place. With slight variations, you can use the same approach to implement the ternary conditional operator ($x?a:b$) found in many languages.

A funny outcome of this approach is that if you generate code for an if-statement like `if (x>y) { . . . }` in the obvious way, you will end up with *two* conditional structures in the assembly code. The first conditional computes the result of $x>y$ and places that in a register. The second conditional compares that result against zero and then jumps to the true or false branch of the statement. With a little careful coding, you can check for this common case and generate a single conditional statement that evaluates the expression and uses one conditional jump to implement the statement.

8.3 Generating Declarations

Finally, emitting the entire program is a matter of traversing each declaration of code or data and emitting its basic structure. Declarations can fall into three cases: global variable declarations, local variable declarations, and global function declarations. (B-Minor does not permit local function declarations.)

Global data declarations are simply a matter of emitting a label along with a suitable directive that reserves the necessary space, and an initializer, if needed. For example, these B-Minor declarations at global scope:

```
i: integer = 10;
s: string = "hello";
b: array [4] boolean = {true, false, true, false};
```

Should yield these output directives:

```
.data
i: .quad 10
s: .string "hello"
b: .quad 1, 0, 1, 0
```

Note that a global variable declaration can only be initialized by a constant value (and not a general expression) precisely because the data section of the program can only contain constants (and not code). If the programmer accidentally put code into the initializer, then the typechecker should have discovered that and raised an error before code generation began.

Emitting a local variable declaration is much easier. (This only happens when `decl_codegen` is called by `stmt_codegen` inside of a function declaration.) Here, you can assume that space for the local variable has already been established by the function prologue, so no stack manipulations are necessary. However, if the variable declaration has an initializing expression (`x:integer=y*10;`) then you must generate code for the expression, store it in the local variable, and free the register.

Function declarations are the final piece. To generate a function, you must emit a label with the function's name, followed by the function prologue. The prologue must take into account the number of parameters and local variables, making the appropriate amount of space on the stack. Next comes the body of the function, followed by the function epilogue. The epilogue should have a unique label so that `return` statements can easily jump there.

8.4 Exercises

- 8.4.1 Write a legal expression that would exhaust the six available scratch registers, if using the technique described in this chapter. In general, how many registers are needed to generate code for an arbitrary expression tree?
- 8.4.2 When using a register calling convention, why is it necessary to generate values for *all* the function arguments before moving the values into the argument registers?
- 8.4.3 Can a global variable declaration have a non-constant initializing expression? Explain why.
- 8.4.4 Suppose B-Minor included a `switch` statement. Sketch out two different assembly language templates for implementing a `switch`.
- 8.4.5 Write the complete code generator for the X86-64 architecture, as outlined in this chapter.
- 8.4.6 Write several test programs to test all the aspects of B-Minor then use your compiler to build, test, and run them.
- 8.4.7 Compare the assembly output of your compiler on your test programs to the output of a production compiler like `gcc` on equivalent programs written in C. What differences do you see?
- 8.4.8 Add an extra code generator to your compiler that emits a different assembly language like ARM or an intermediate representation like LLVM. Describe any changes in approach that were necessary.

